ĐẠI HỌC BÁCH KHOA HÀ NỘI



ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ HỆ THỐNG BÁO CHÁY KHÔNG DÂY TÍCH HỢP TRONG HỆ SINH THÁI NHÀ THÔNG MINH

NGUYỄN TRẦN TIẾN
TIEN.NT222184@sis.hust.edu.vn

Ngành KT Điều khiển & Tự động hóa

Đã ký số ngày 29/06/20

Giảng viên hướng dẫn: TS. Lê Công Cường

Chữ ký của GVHD

Khoa: Tự động hóa
Trường: Điện – Điện tử

HÀ NỘI, 06/2026

BỘ GIÁO DỤC & ĐÀO TẠO
ĐH BÁCH KHOA HÀ NỘI

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do - Hạnh phúc

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Nguyễn Trần Tiến

MSSV: 20222184

Khóa: K67

Trường: Điện- Điện tử

Ngành: Kỹ thuật Điều khiển và Tự động hóa

1. **Tên đề tài:**

Thiết kế hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh

2. **Nội dung đề tài:**

Nghiên cứu, thiết kế và xây dựng một hệ thống báo cháy không dây có khả năng tích hợp trong hệ sinh thái nhà thông minh, hoạt động ổn định, đáng tin cậy và tiết kiệm năng lượng. Các nội dung chính của đề tài bao gồm:

- Tìm hiểu cơ sở lý thuyết về mạng truyền thông không dây công suất thấp và các giao thức trao đổi dữ liệu cho hệ thống IoT.
- Phân tích yêu cầu và thiết kế phần cứng cho các nút cảm biến cùng thiết bị trung tâm của hệ thống.
- Xây dựng phần mềm thu thập, truyền nhận và xử lý dữ liệu; thiết kế giao diện giám sát và cơ chế cảnh báo từ xa.
- Triển khai, thử nghiệm và đánh giá hoạt động của hệ thống trong các kịch bản thực tế.

3. **Thời gian giao đề tài:** 11/03/2026

4. **Thời gian hoàn thành:** 28/06/2026

Hà Nội, ngày 28 tháng 06 năm 2026

CÁN BỘ HƯỚNG DẪN

TS. LÊ CÔNG CƯỜNG

Lời cam đoan

Em xin cam đoan đồ án tốt nghiệp "Thiết kế hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh" là công trình nghiên cứu của riêng em, được thực hiện dưới sự hướng dẫn của TS. Lê Công Cường. Các số liệu và kết quả trong đồ án là trung thực, mọi tài liệu tham khảo đều được trích dẫn đầy đủ. Em xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

I hereby declare that this graduation project, "Design of a Wireless Fire Alarm System Integrated into a Smart Home Ecosystem", is my own work, carried out under the supervision of Dr. Le Cong Cuong. All data and results presented are truthful, and all references are properly cited. I take full responsibility for this declaration.

Lời cảm ơn

Em xin gửi lời cảm ơn chân thành đến TS. Lê Công Cường đã tận tình hướng dẫn em trong suốt quá trình thực hiện đồ án. Em cũng xin cảm ơn các thầy, cô Khoa Tự động hóa và Trường Đại học Bách khoa Hà Nội đã trang bị cho em nền tảng kiến thức vững chắc, cùng gia đình và bạn bè đã luôn động viên, hỗ trợ em hoàn thành đồ án này.

I would like to express my sincere gratitude to Dr. Le Cong Cuong for his dedicated guidance throughout this project. I also thank the lecturers of the Faculty of Automation and Hanoi University of Science and Technology for providing me with a solid foundation of knowledge, as well as my family and friends for their constant encouragement and support in completing this project.

Tóm tắt nội dung đồ án

Đồ án xây dựng một hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh nhằm phát hiện và cảnh báo cháy sớm, hoạt động ổn định và tiết kiệm năng lượng. Hệ thống dùng mạng không dây kết nối các nút cảm biến với thiết bị trung tâm và đám mây; dữ liệu môi trường (nhiệt độ, độ ẩm, chất lượng không khí, khói) được thu thập, giám sát thời gian thực trên giao diện web và cảnh báo từ xa qua tin nhắn. Kết quả thực nghiệm cho thấy hệ thống truyền nhận ổn định, phản hồi nhanh và tiết kiệm năng lượng, đáp ứng các yêu cầu đặt ra và có khả năng mở rộng. Qua đồ án, sinh viên nắm vững kiến thức về hệ thống IoT, thiết kế phần cứng - phần mềm và lập trình nhúng.

This project develops a wireless fire alarm system integrated into a smart home ecosystem for early fire detection and warning, operating reliably and energy-efficiently. The system uses a wireless network to connect sensor nodes with a central device and the cloud; environmental data (temperature, humidity, air quality and smoke) is collected, monitored in real time on a web interface, and sent as remote alerts via text messages. Experimental results show stable transmission, fast response and low power consumption, meeting the stated requirements with good scalability. Through this project, the student has gained solid knowledge of IoT systems, hardware–software design and embedded programming.

Sinh viên thực hiện
Ký và ghi rõ họ tên

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU CHUNG 1
1.1 Bối cảnh và lý do chọn đề tài 1
1.2 Mục tiêu của hệ thống 1
1.3 Đối tượng và phạm vi ứng dụng 1
1.4 Nhiệm vụ thiết kế 2
1.5 Phương pháp nghiên cứu và phát triển 3
1.5.1 Phương pháp nghiên cứu lý thuyết 3
1.5.2 Phương pháp thiết kế hệ thống 3
1.5.3 Phương pháp thực nghiệm và phát triển 4
1.5.4 Phương pháp đánh giá kết quả 4
1.5.5 Công cụ và môi trường phát triển 4
1.6 Cấu trúc của báo cáo 5
1.7 Kết luận chương 5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 6
2.1 Giới thiệu về giao thức Thread và mạng lưới mesh cho hệ thống IoT 6
2.1.1 Lý do lựa chọn giao thức Thread 6
2.1.2 Kiến trúc mạng Thread 8
2.1.3 Giao thức 6LoWPAN và định tuyến 10
2.2 Giao thức CoAP và MQTT trong truyền thông IoT 11
2.2.1 Lý do lựa chọn giao thức CoAP và MQTT 11
2.2.2 Giao thức MQTT 12
2.2.3 Giao thức CoAP 14
2.2.4 So sánh MQTT và CoAP 16
2.3 Vi điều khiển và nền tảng phát triển 16
2.3.1 ESP32-H2 16
2.3.2 ESP32 WROOM 32 22
2.4 Cảm biến và cơ chế hoạt động 27
2.4.1 Cảm biến nhiệt độ độ ẩm SHT31 27
2.4.2 Cảm biến chất lượng không khí CCS811 32
2.4.3 Cảm biến khói RE46C194 + IR LED + Photodiode 37
2.5 Kết luận chương 42
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 43

3.1 Sơ đồ khối tổng thể 43
3.2 Yêu cầu chức năng và phi chức năng 44
3.2.1 Yêu cầu chức năng 44
3.2.2 Yêu cầu phi chức năng 45
3.3 Tính toán công suất tiêu thụ và ước lượng thời lượng pin cho SED 46
3.3.1 Cơ chế tiết kiệm năng lượng cho SED 46
3.3.2 Tính toán công suất tiêu thụ và ước lượng thời lượng pin 47
3.4 Thiết kế phần cứng 48
3.4.1 Sơ đồ mạch Border Router 48
3.4.2 Sơ đồ mạch cho Router 49
3.4.3 Sơ đồ mạch cho SED 50
3.5 Thiết kế phần mềm 51
3.5.1 Lưu đồ hoạt động tổng quát 51
3.5.2 Giao tiếp, cấu hình cho SHT31 và xử lý ngưỡng nhiệt độ 55
3.5.3 Giao tiếp, cấu hình với CCS811 55
3.5.4 Đóng gói dữ liệu JSON và gửi CoAP 55
3.5.5 Xử lý nhận CoAP trên Border Router và chuyển tiếp MQTT 56
3.6 Thiết kế giao diện giám sát và cảnh báo 58
3.6.1 Mục tiêu và các yêu cầu kỹ thuật đối với giao diện hệ thống 58
3.6.2 Mô hình luồng dữ liệu thời gian thực tại giao diện (Data Flow) 59
3.6.3 Giải pháp bố cục và tổ chức không gian hiển thị (Wireframe/Mockup) 59
3.6.4 Tích hợp cảnh báo từ xa qua Telegram 60
3.7 Kết luận chương 60
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 61
4.1 Môi trường phát triển 61
4.1.1 Cài đặt ESP-IDF 61
4.1.2 Cài đặt các thư viện phụ thuộc 62
4.2 Lưu đồ thuật toán cho các thiết bị trong hệ thống 62
4.2.1 Lưu đồ thuật toán Border Router 62
4.2.2 Lưu đồ thuật toán RCP 63
4.2.3 Lưu đồ thuật toán Router 65
4.2.4 Lưu đồ thuật toán SED 65
4.3 Thiết lập mạng Thread 66

4.3.1 Đồng bộ datasheet 66
4.3.2 Ghép nối SED và Router 67
4.4 Triển khai xử lý thông số từ các cảm biến 67
4.4.1 Cảm biến SHT31 67
4.4.2 Cảm biến CCS811 68
4.5 Triển khai truyền nhận dữ liệu giữa các thiết bị 69
4.5.1 Triển khai truyền nhận dữ liệu giữa SED và Router 69
4.5.2 Triển khai truyền nhận dữ liệu giữa Router và Border Router 69
4.6 Cấu hình MQTT Broker 70
4.6.1 Triển khai trên HiveMQ 70
4.6.2 Triển khai trên Border Router 70
4.7 Triển khai Web Dashboard 71
4.7.1 Quy trình quản lý mã nguồn và triển khai CI/CD 71
4.7.2 Triển khai cấu trúc không gian hiển thị (HTML/CSS) 71
4.7.3 Thiết lập kết nối và xử lý luồng sự kiện MQTT (JavaScript) 72
4.7.4 Thuật toán kiểm tra ngưỡng cảnh báo và trực quan hóa đồ thị 72
4.8 Triển khai cảnh báo qua Telegram 72
4.9 Biên dịch và nạp firmware 73
4.10 Kết luận chương 74
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 75
5.1 Kết quả giao tiếp mạng Thread 75
5.1.1 Thời gian join mạng của SED 75
5.1.2 Độ ổn định kết nối 75
5.2 Kết quả đọc cảm biến 76
5.3 Kết quả trong các trạng thái kiểm thử 77
5.4 Kết quả cảnh báo qua Telegram 78
5.5 Hiệu quả tiết kiệm năng lượng 79
5.6 Đánh giá về độ trễ và độ tin cậy của hệ thống 79
5.7 Khả năng mở rộng 79
5.8 Kết luận chương 80
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 81
6.1 Tổng kết những kết quả đạt được 81
6.2 Hạn chế và nguyên nhân 81
6.3 Đề xuất hướng phát triển tương lai 82

6.3.1 Tích hợp học máy để dự đoán cháy 82
6.3.2 Mở rộng thêm các loại cảm biến 82
6.3.3 Nâng cấp giao diện quản lý đa nền tảng 82
6.4 Kết luận chương 83
KẾT LUẬN 84
1. Kết luận 84
2. Hướng phát triển của đồ án trong tương lai 84
TÀI LIỆU THAM KHẢO 85

DANH MỤC HÌNH VẼ

Hình 2.1 Sơ đồ kiến trúc phân cấp trong mạng Thread 8
Hình 2.2 Sơ đồ phân loại các thiết bị trong mạng Thread 9
Hình 2.3 Mô hình cấu trúc mạng truyền thông MQTT 13
Hình 2.4 Mô hình tương tác giữa Client và Server sử dụng CoAP 15
Hình 2.5 Module ESP32-H2 Supermini 18
Hình 2.6 Sơ đồ nguyên lý vi điều khiển ESP32-H2 20
Hình 2.7 Module ESP32 WROOM 32 38 pins out 24
Hình 2.8 Cảm biến SHT31 29
Hình 2.9 Sơ đồ nguyên lý cảm biến SHT31 29
Hình 2.10 Cảm biến CCS811 33
Hình 2.11 Sơ đồ nguyên lý cảm biến CCS811 34
Hình 2.12 IC cảm biến RE46C194 39
Hình 2.13 Sơ đồ nguyên lý cảm biến RE46C194 39
Hình 3.1 Sơ đồ khối tổng thể hệ thống 43
Hình 3.2 Khối nguồn Border Router 49
Hình 3.3 Khối hạ áp 3V3 Border Router 49
Hình 3.4 Khối vi điều khiển Border Router 49
Hình 3.5 Khối nguồn Router 50
Hình 3.6 Khối hạ áp 3V3 Router 50
Hình 3.7 Khối MCU và cảm biến cho Router 50
Hình 3.8 Khối nguồn SED 51
Hình 3.9 Khối MCU và cảm biến SED 51
Hình 3.10 Lưu đồ hệ thống giai đoạn khởi tạo 52
Hình 3.11 Lưu đồ hệ thống giai đoạn giám sát nền 53
Hình 3.12 Lưu đồ hệ thống giai đoạn phát hiện sự cố 54
Hình 3.13 Lưu đồ luồng dữ liệu từ Border Router 58
Hình 4.1 Lưu đồ thuật toán ESP32 WROOM 32 làm Border Router 63
Hình 4.2 Lưu đồ thuật toán ESP32-H2 chạy RCP 64
Hình 4.3 Lưu đồ thuật toán Router 65
Hình 4.4 Lưu đồ thuật toán SED 66
Hình 4.5 Đẩy dữ liệu code lên Github 71
Hình 4.6 Thiết lập kết nối MQTT Broker với Render 72
Hình 4.7 Triển khai cấu hình trên Railway 73
Hình 5.1 Trạng thái của SED trong mạng Thread 75
Hình 5.2 Dữ liệu đẩy lên MQTT broker hiển thị trên HiveMQ 76
Hình 5.3 Mô phỏng hệ thống khi Router mất kết nối 76

Hình 5.4 Kết quả đọc cảm biến tại SED 77
Hình 5.5 Giao diện Web Dashboard chuyển trạng thái cảnh báo nguy hiểm màu đỏ khi nhiệt độ vượt ngưỡng 77
Hình 5.6 Giao diện Web Dashboard chuyển trạng thái cảnh báo nguy hiểm màu đỏ khi eCO_2 vượt ngưỡng 78
Hình 5.7 Kết quả gửi từ Telegram 78

DANH MỤC BẢNG BIỂU

Bảng 2.1 So sánh Thread với các công nghệ không dây phổ biến 7
Bảng 2.2 So sánh các giao thức truyền tin phổ biến 12
Bảng 2.3 So sánh MQTT và CoAP 16
Bảng 2.4 So sánh các vi điều khiển hỗ trợ mạng Thread 17
Bảng 2.5 Các chân module ESP32-H2 Supermini 19
Bảng 2.6 Thông số kỹ thuật ESP32-H2 Supermini 21
Bảng 2.7 So sánh các vi điều khiển chạy Border Router 23
Bảng 2.8 Các chân module ESP32 WROOM 32 25
Bảng 2.9 Thông số kỹ thuật ESP32 WROOM 32 26
Bảng 2.10 So sánh các cảm biến nhiệt độ 28
Bảng 2.11 Các chân cảm biến SHT31 29
Bảng 2.12 Tập lệnh cảm biến SHT31 31
Bảng 2.13 So sánh các cảm biến chất lượng không khí 33
Bảng 2.14 Sơ đồ chân cảm biến CCS811 34
Bảng 2.15 Thông số kỹ thuật cảm biến CCS811 35
Bảng 2.16 Các chế độ đo trên cảm biến CCS811 36
Bảng 2.17 Các thanh ghi của CCS811 37
Bảng 2.18 So sánh các cảm biến khói 38
Bảng 2.19 Thông số kỹ thuật cảm biến RE46C194 40

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks (IPv6 trên mạng không dây công suất thấp)
2	ADC	Analog-to-Digital Converter (Bộ chuyển đổi tương tự–số)
3	BLE	Bluetooth Low Energy (Bluetooth năng lượng thấp)
4	BR	Border Router (Bộ định tuyến biên)
5	CoAP	Constrained Application Protocol (Giao thức ứng dụng ràng buộc)
6	CPU	Central Processing Unit (Bộ xử lý trung tâm)
7	eCO_2	equivalent CO_2 (Nồng độ CO_2 tương đương)
8	ESP-IDF	Espressif IoT Development Framework
9	FTD	Full Thread Device (Thiết bị Thread đầy đủ)
10	GPIO	General-Purpose Input/Output (Chân vào/ra đa dụng)
11	I^2C	Inter-Integrated Circuit (Giao tiếp I^2C)
12	IoT	Internet of Things (Internet vạn vật)
13	IPv6	Internet Protocol version 6
14	JSON	JavaScript Object Notation
15	MCU	Microcontroller Unit (Vi điều khiển)
16	MLE	Mesh Link Establishment (Giao thức thiết lập liên kết mesh)
17	MQTT	Message Queuing Telemetry Transport
18	MTD	Minimal Thread Device (Thiết bị Thread tối thiểu)
19	PCB	Printed Circuit Board (Bo mạch in)
20	PCCC	Phòng cháy chữa cháy
21	PDR	Packet Delivery Ratio (Tỷ lệ truyền nhận gói tin)
22	QoS	Quality of Service (Chất lượng dịch vụ)
23	RCP	Radio Co-Processor (Bộ đồng xử lý vô tuyến)
24	SED	Sleepy End Device (Thiết bị đầu cuối dạng ngủ)
25	SoC	System on Chip (Hệ thống trên một chip)
26	TCP	Transmission Control Protocol
27	TVOC	Total Volatile Organic Compounds (Tổng hợp chất hữu cơ bay hơi)
28	UART	Universal Asynchronous Receiver/Transmitter
29	UDP	User Datagram Protocol
30	Wi-Fi	Wireless Fidelity (Mạng không dây)

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh đô thị hóa và công nghiệp hóa, các mô hình nhà ở, nhà thông minh và khu vực sản xuất luôn tiềm ẩn nguy cơ hỏa hoạn nghiêm trọng. Theo Bộ Công an, riêng năm 2025 cả nước xảy ra 3.060 vụ cháy, làm 90 người chết, gây thiệt hại khoảng 786,78 tỷ đồng, trong đó sự cố thiết bị điện chiếm gần 75% nguyên nhân. Thực trạng này đặt ra yêu cầu cấp thiết về một giải pháp phát hiện sớm và cảnh báo cháy chính xác.

Tuy nhiên, hệ thống báo cháy truyền thống hiện còn nhiều hạn chế: hệ thống có dây phức tạp, chi phí cao; hệ thống không dây dùng sóng thông thường lại dễ nhiễu và tiêu hao năng lượng lớn. Đặc biệt, các phương pháp cũ thường phụ thuộc vào cảm biến đơn lẻ (chỉ đo khói hoặc nhiệt), dễ dẫn đến báo động giả. Để giải quyết triệt để, xu hướng hiện nay là áp dụng giải pháp đa cảm biến, không chỉ giám sát nhiệt độ mà phải kết hợp đo nồng độ các khí độc sinh ra từ khói và sự suy giảm oxy như CO_2, CO, O_2 để nhận diện chính xác đám cháy. Dù vậy, việc triển khai này trong không gian nhà thông minh vấp phải rào cản về vật cản vật lý (tường bê tông, hàng hóa cản sóng) và thách thức tối ưu năng lượng cho các nút cảm biến dùng pin. Do đó, việc ứng dụng cấu trúc mạng lưới (Mesh) với thuật toán quản lý năng lượng thông minh—chỉ kích hoạt các module truyền thông công suất lớn khi phát hiện rủi ro—là hướng tiếp cận bắt buộc để kéo dài tuổi thọ thiết bị [1].

Xuất phát từ thực tiễn đó, đề tài "Thiết kế hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh" được lựa chọn thực hiện. Hệ thống tận dụng triệt để ưu điểm kết nối của mạng Thread, đồng thời áp dụng giải pháp đa cảm biến (nhiệt, khói, chất lượng không khí) nhằm mang lại một giải pháp cảnh báo hỏa hoạn sớm, tin cậy và tối ưu chi phí vận hành.

1.2 Mục tiêu của hệ thống

Dựa trên bối cảnh đã phân tích, hệ thống báo cháy dùng mạng Thread cần đạt các mục tiêu: phát hiện sớm nguy cơ cháy, truyền thông tin cậy, tiết kiệm năng lượng và cung cấp giao diện giám sát từ xa.

1.3 Đối tượng và phạm vi ứng dụng

Đối tượng nghiên cứu gồm các thành phần công nghệ và giải pháp kỹ thuật sau:

- Giao thức mạng Thread và nền tảng mã nguồn mở OpenThread trên vi điều khiển ESP32-H2.
- Các cảm biến chuyên dụng phát hiện cháy sớm: SHT31 (nhiệt độ, độ ẩm), CCS811 (eCO_2, TVOC) và RE46C194 (khói).
- Cơ chế truyền thông tiết kiệm năng lượng cho thiết bị Sleepy End Device (SED) trong mạng Thread, bao gồm light-sleep và đánh thức bằng ngắt ngoài.
- Phương pháp xử lý đa chỉ số (nhiệt độ, eCO_2, TVOC, khói) để ra quyết định cảnh báo cháy, hạn chế báo động giả.

Phạm vi ứng dụng gồm các không gian và mô hình triển khai điển hình:

- Kho bãi, nhà thông minh, nhà kho, nhà xưởng, trung tâm dữ liệu, bãi đỗ xe ngầm.
- Các tòa nhà thông minh (smart home / smart building) cần giải pháp báo cháy không dây, dễ mở rộng và tiết kiệm năng lượng.
- Các khu vực không có sẵn hạ tầng Wi-Fi hoặc cáp, nơi mạng lưới dạng lưới (mesh) của Thread phát huy ưu thế.
- Vị trí khó tiếp cận, yêu cầu thiết bị hoạt động lâu dài trên pin (1-2 năm) để giảm chi phí bảo trì.

Giới hạn của đồ án:

- Mạng Thread chỉ triển khai với một Border Router, một Router và tối đa 3 SED do điều kiện thiết bị thực tế.
- Cảm biến RE46C194 được tích hợp dạng module rời, chưa tối ưu PCB và chưa thử nghiệm trong buồng khói chuẩn.
- Chưa tích hợp cơ chế cảnh báo âm thanh (loa ngoài) hoặc đèn strobe; chỉ dừng ở mức gửi dữ liệu qua CoAP và hiển thị dashboard.
- Đánh giá thời lượng pin dựa trên tính toán lý thuyết, chưa có thí nghiệm đo dòng thực tế dài hạn.

Việc xác định rõ đối tượng, phạm vi và giới hạn là cơ sở định hướng toàn bộ quá trình thiết kế, triển khai và đánh giá, đồng thời khẳng định tính thực tiễn của giải pháp báo cháy dựa trên mạng Thread.

1.4 Nhiệm vụ thiết kế

Dựa trên mục tiêu và phạm vi đã xác định, đồ án chia nhiệm vụ thiết kế thành hai nhóm: đã hoàn thành và dự kiến nhưng chưa hoàn thiện.

Các nhiệm vụ đã thực hiện và đạt được:

- Thiết kế kiến trúc tổng thể: mô hình phân tầng gồm SED, Router, Border Router, MQTT Broker và Web Dashboard; xác định rõ luồng dữ liệu và chức năng từng khối.
- Thiết kế phần cứng: hoàn thiện sơ đồ nguyên lý cho Border Router, Router và SED, thiết kế mạch nguồn.
- Thiết kế phần mềm: lưu đồ thuật toán cho các thiết bị; driver I^2C và cấu hình ngưỡng cho SHT31, CCS811; đóng gói JSON và truyền CoAP trong mạng Thread; chuyển tiếp CoAP $\rightarrow$ MQTT trên Border Router kết nối HiveMQ Cloud.
- Triển khai mạng Thread: cấu hình OpenThread, đồng bộ datasheet, thiết lập Border Router với RCP qua UART; cấu hình SED ở chế độ Sleepy End Device với light-sleep, timer và GPIO wake-up.
- Tính toán và đánh giá sơ bộ: ước lượng công suất và thời lượng pin cho SED theo datasheet; đo thời gian gia nhập mạng, khả năng tự phục hồi, độ trễ và tỷ lệ truyền nhận gói tin.
- Tích hợp cảnh báo từ xa qua Telegram: xây dựng bot Telegram tự gửi tin nhắn cảnh báo khi nhiệt độ vượt ngưỡng, giúp người giám sát nhận thông tin kịp thời ngay cả khi không theo dõi Web Dashboard.

Các nhiệm vụ dự kiến nhưng chưa hoàn thiện

- Tích hợp và kiểm thử cảm biến khói RE46C194: đã hoàn thành sơ đồ nguyên lý và cơ chế đọc trạng thái qua GPIO, nhưng do thời gian và điều kiện chưa có buồng khói quang học hoàn chỉnh để đo đạc, hiệu chỉnh; hệ thống chưa đưa được tín hiệu khói thực tế vào luồng cảnh báo.
- Kiểm thử hiệu năng dài hạn và đo dòng thực tế: ước lượng pin mới ở mức lý thuyết, cần đo dòng tiêu thụ thực tế qua các chu kỳ để xác nhận thời lượng pin.

Việc phân chia nhiệm vụ đã và chưa hoàn thành giúp định hướng các bước phát triển tiếp theo và là cơ sở đánh giá kết quả, hạn chế cần khắc phục.

1.5 Phương pháp nghiên cứu và phát triển

Để đạt mục tiêu đề ra, đồ án triển khai theo quy trình kết hợp nghiên cứu lý thuyết, thực nghiệm và phát triển hệ thống, qua các phương pháp sau:

1.5.1 Phương pháp nghiên cứu lý thuyết

Phương pháp nghiên cứu lý thuyết nhằm xây dựng nền tảng kiến thức về công nghệ, giao thức và thiết bị, làm cơ sở cho thiết kế và triển khai:

- Thu thập và tổng hợp tài liệu: nghiên cứu datasheet, application note của ESP32-H2, ESP32 WROOM 32, các cảm biến SHT31, CCS811, RE46C194 và giao thức OpenThread.
- Nghiên cứu nền tảng phát triển: tìm hiểu ESP-IDF tích hợp trên VS Code, FreeRTOS, thư viện cảm biến và I²C, giao thức MQTT/CoAP cùng cơ chế quản lý năng lượng (SED, wake-up by GPIO).
- Phân tích giải pháp hiện có: đánh giá hệ thống báo cháy truyền thống và mạng cảm biến không dây (WSN) cho phát hiện cháy sớm để rút ưu nhược điểm.

Kiến thức tổng hợp từ bước này là tiền đề chọn giải pháp tối ưu, tránh sai sót khi phát triển hệ thống thực tế.

1.5.2 Phương pháp thiết kế hệ thống

Phương pháp thiết kế dựa trên phân tách chức năng và cấu trúc phân tầng, đảm bảo linh hoạt, mở rộng và tin cậy:

- Thiết kế phân tầng (layered design): Hệ thống được chia thành các tầng: tầng cảm biến (SED, Router), tầng mạng (Thread Mesh), tầng biên (Border Router), tầng đám mây (MQTT Broker) và tầng ứng dụng (Web Dashboard).
- Thiết kế phần cứng module: sơ đồ nguyên lý và bố trí linh kiện cho từng nút (SED, Router), dùng breadboard và module rời linh hoạt trong thử nghiệm.
- Thiết kế phần mềm hướng tác vụ: dùng FreeRTOS phân chia các tác vụ đọc cảm biến, xử lý ngắt, gửi CoAP, quản lý năng lượng, đảm bảo tính đồng thời và tin cậy.

Phương pháp này tạo hệ thống kiến trúc rõ ràng, các thành phần độc lập nhưng liên kết chặt, thuận lợi cho phát triển, kiểm thử và bảo trì.

1.5.3 Phương pháp thực nghiệm và phát triển

Phương pháp thực nghiệm và phát triển nhằm kiểm chứng thiết kế và đánh giá hiệu năng từng khối cũng như toàn hệ thống trong các điều kiện khác nhau:

- Phát triển lặp (iterative): mỗi thành phần (cảm biến, kết nối Thread, cơ chế ngủ/thức) được phát triển và kiểm thử độc lập trước khi tích hợp.
- Mô phỏng kiểm thử từng phần:
  - Kiểm tra giao tiếp I^2C với từng cảm biến bằng các chương trình riêng.
  - Thử nghiệm cơ chế wake-up bằng GPIO trong các kịch bản khác nhau.
  - Mô phỏng mạng Thread bằng OpenThread CLI trên các thiết bị phát triển trước khi triển khai thực tế.
- Đo đạc và đánh giá hiệu năng:
  - Sử dụng logic analyzer để kiểm tra tín hiệu I^2C, tín hiệu ngắt.
  - Đánh giá thời gian phản ứng từ lúc phát hiện ngưỡng đến khi gửi cảnh báo đến Border Router và Web dashboard.

Kết quả thực nghiệm khẳng định hệ thống hoạt động ổn định, đồng thời là cơ sở hiệu chỉnh, tối ưu các tham số về độ tin cậy, thời gian phản hồi và tiêu thụ năng lượng.

1.5.4 Phương pháp đánh giá kết quả

Phương pháp đánh giá nhằm lượng hóa mức đáp ứng của hệ thống so với mục tiêu, xác định ưu điểm, hạn chế và khả năng ứng dụng:

- So sánh và đối chiếu: Đối chiếu kết quả đọc được từ cảm biến với các thiết bị tham chiếu (nhiệt kế, máy đo eCO_2 chuyên dụng) để đánh giá độ chính xác.
- Kiểm thử kịch bản thực tế:
  - Tạo nguồn nhiệt (máy sấy tóc, đèn halogen) để kiểm tra phát hiện tăng nhiệt độ.
  - Sử dụng bút thử khói, que hương hoặc cồn để tạo nguồn CO_2 và TVOC, quan sát phản ứng của CCS811.
  - Thử nghiệm tình huống đồng thời (nhiệt độ cao và nồng độ CO_2 cao) để xác nhận thuật toán phát hiện cháy đa chỉ số.
- Phân tích thời lượng pin: Dựa trên dòng tiêu thụ đo được và thời gian hoạt động giả định (số lần thức dậy do sự kiện, số lần bảo trì định kỳ), ước tính thời gian hoạt động thực tế của SED.

Các chỉ số và nhận xét từ đánh giá là cơ sở khẳng định hiệu quả của giải pháp và định hướng cải tiến trong tương lai.

1.5.5 Công cụ và môi trường phát triển

Việc chọn công cụ và môi trường phát triển phù hợp giúp đẩy nhanh tiến độ, nâng cao hiệu quả lập trình và kiểm soát lỗi:

- Phần cứng: ESP32 WROOM 32, ESP32-H2 Super Mini, module cảm biến SHT31, CCS811, pin AA, các linh kiện tụ điện, diode, điện trở.
- Phần mềm: Frameworks ESP-IDF v5.5.4 + Visual Studio Code, OpenThread, MQTT Broker (HiveMQ Cloud), Github, Render, Railway.

Nhờ hệ công cụ và môi trường đồng bộ, việc lập trình, kiểm thử và gỡ lỗi được thực hiện bài bản, rút ngắn thời gian và đảm bảo độ tin cậy sản phẩm.

1.6 Cấu trúc của báo cáo

Báo cáo gồm sáu chương, mỗi chương phân tích, thiết kế và đánh giá một khía cạnh của hệ thống:

- Chương 1 – Giới thiệu: bối cảnh, lý do, mục tiêu, đối tượng, phạm vi, phương pháp và cấu trúc báo cáo.
- Chương 2 – Cơ sở lý thuyết: mạng Thread, giao thức CoAP/MQTT, các cảm biến SHT31, CCS811, RE46C194, vi điều khiển ESP32-H2/WROOM 32 và cơ chế quản lý năng lượng.
- Chương 3 – Phân tích và thiết kế: kiến trúc tổng thể, yêu cầu chức năng/phi chức năng, thiết kế phần cứng và phần mềm, thiết kế giao diện giám sát.
- Chương 4 – Triển khai: môi trường phát triển, biên dịch, nạp firmware, thiết lập mạng Thread, cấu hình MQTT broker và web dashboard.
- Chương 5 – Kết quả thực nghiệm: đo đạc kết nối mạng, độ chính xác cảm biến, hiệu quả năng lượng, độ trễ và đánh giá hệ thống qua các kịch bản.
- Chương 6 – Kết luận và hướng phát triển: tổng kết kết quả, hạn chế và đề xuất hướng phát triển.

Cấu trúc này giúp báo cáo trình bày hệ thống, rõ ràng từ nghiên cứu lý thuyết, thiết kế, triển khai đến đánh giá.

1.7 Kết luận chương

Chương 1 đã trình bày bối cảnh và lý do chọn đề tài, xuất phát từ nguy cơ cháy nổ cao và hạn chế của các hệ thống báo cháy truyền thống. Trên cơ sở đó, đồ án đặt mục tiêu xây dựng hệ thống báo cháy thông minh, tiết kiệm năng lượng, tận dụng khả năng tự phục hồi và tính mở rộng của giao thức Thread kết hợp cảm biến nhiệt độ và chất lượng không khí để phát hiện và cảnh báo cháy sớm.

Đối tượng, phạm vi ứng dụng và các giới hạn của đề tài cũng được xác định rõ, với yêu cầu thiết bị hoạt động lâu dài trên pin.

Các nhiệm vụ thiết kế được chia thành hai nhóm: nhóm đã hoàn thành gồm thiết kế kiến trúc tổng thể, phần cứng và phần mềm nhúng, triển khai mạng Thread, tích hợp cảnh báo từ xa qua bot Telegram khi nhiệt độ vượt ngưỡng và đánh giá sơ bộ hiệu năng; nhóm chưa hoàn thiện gồm tích hợp, kiểm thử cảm biến khói RE46C194, kiểm thử hiệu năng dài hạn và thử nghiệm kịch bản thực tế.

Chương cũng giới thiệu phương pháp nghiên cứu (kết hợp lý thuyết, thiết kế, thực nghiệm và đánh giá), các công cụ phát triển và cấu trúc báo cáo gồm sáu chương, làm cơ sở cho các chương tiếp theo.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Giới thiệu về giao thức Thread và mạng lưới mesh cho hệ thống IoT

2.1.1 Lý do lựa chọn giao thức Thread

Thread là giao thức mạng mesh công suất thấp trên nền IPv6 do Thread Group (liên minh công nghiệp do Google/Nest dẫn dắt) chuẩn hóa và được giới nghiên cứu đánh giá là một lựa chọn khả thi cho IoT tương lai [2]. Giao thức này được lựa chọn làm giải pháp truyền thông cốt lõi cho hệ thống báo cháy không dây dựa trên bốn ưu thế kỹ thuật sau:

- Khả năng tự chữa lành cao: Khi một nút Router gặp sự cố hoặc bị hỏa hoạn phá hủy, mạng tự động định tuyến lại qua các nút lân cận để thông tin cảnh báo không bị gián đoạn.
- Hỗ trợ IPv6 toàn diện: Định danh bằng địa chỉ IP trực tiếp giúp giảm độ trễ biên dịch giao thức, tối ưu truyền tin thời gian thực và tăng bảo mật mã hóa đầu–cuối.
- Tối ưu năng lượng cho nút ngủ (SED): Nút cảm biến đầu cuối có thể ngủ sâu với dòng tiêu thụ cỡ micro-ampere và thức dậy ngay khi có ngắt báo cháy, giúp kéo dài tuổi thọ pin.
- Phân tách vai trò linh hoạt: Mạng tự bầu Leader và thăng cấp vai trò các nút theo trạng thái thực tế, giúp mở rộng vùng phủ sóng và xóa các điểm chết tín hiệu.

Bảng dưới đây so sánh Thread với các công nghệ không dây phổ biến (Wi-Fi, Bluetooth, LoRa) trong hệ sinh thái IoT:

Bảng 2.1 So sánh Thread với các công nghệ không dây phổ biến

Tiêu chí so sánh	Thread (Mesh)	Wi-Fi	Bluetooth Low Energy (BLE)
Băng tần & Tiêu chuẩn	2.4 GHz (IEEE 802.15.4)	2.4 GHz / 5 GHz / 6 GHz	2.4 GHz
Hiệu quả năng lượng	Rất cao (Hỗ trợ chế độ ngủ SED, chạy nhiều năm với pin nhỏ)	Thấp (Băng thông lớn, tiêu thụ năng lượng cao, khó dùng pin)	Cao (Tối ưu cho thiết bị đeo cá nhân)
Kiến trúc & Độ tin cậy	Mạng lưới (Mesh); tự cấu hình, tự phục hồi, không có điểm lỗi duy nhất	Hình sao (Star); phụ thuộc hoàn toàn vào Router trung tâm	Điểm - Điểm (P2P); giới hạn số kết nối và khoảng cách
Khả năng mở rộng diện rộng	Rất tốt (Liên kết đồng thời hàng trăm nút, tự định tuyến đa chặng)	Trung bình (Bị giới hạn bởi năng lực chịu tải của Router)	Kém (Phạm vi hẹp, không tối ưu cho mạng diện rộng tòa nhà)
Tối ưu chi phí & Phần cứng	Cao (Tích hợp sẵn trên SoC như ESP32, tối giản mạch PCB)	Trung bình (Mạch phần cứng phức tạp hơn)	Cao (Chip rẻ, tích hợp phổ biến)
Khả năng ứng dụng nhà kho / PCCC	Tối ưu nhất (Độ trễ thấp, tin nhắn báo cháy truyền liên tục qua nhiều chặng)	Kém phù hợp (Dễ mất kết nối khi Router trung tâm gặp sự cố hỏa hoạn)	Kém phù hợp (Không phủ hết được diện tích kho bãi lớn)

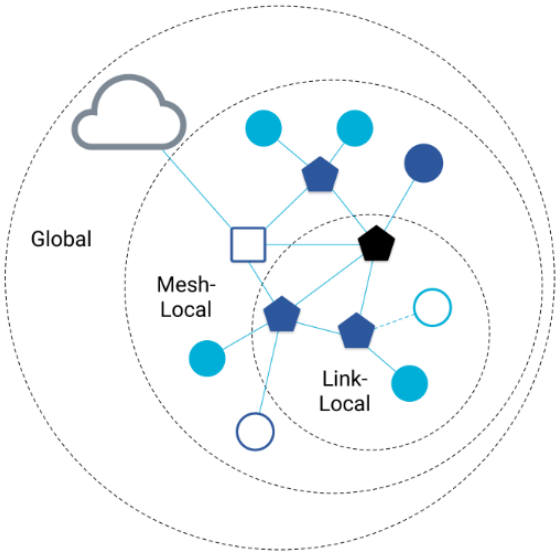
- Hiệu quả năng lượng vượt trội Wi-Fi: Wi-Fi băng thông lớn nhưng tiêu thụ năng lượng cao, khó duy trì cảm biến bằng pin lâu dài; Thread là giao thức siêu nhẹ, cho phép nút SED chạy nhiều năm chỉ với một viên pin nhỏ.
- Mở rộng và tin cậy hơn Bluetooth (BLE): BLE giới hạn số kết nối trực tiếp và khoảng cách, trong khi kiến trúc Mesh của Thread liên kết đồng thời hàng trăm thiết bị, tự định tuyến qua nút trung gian để phủ sóng toàn bộ tòa nhà.
- Tối ưu chi phí so với LoRa: LoRa truyền xa hàng km nhưng cần mô-đun phần cứng chuyên dụng đắt và cổng kênh; với nhà thông minh, băng tần 2.4 GHz (IEEE 802.15.4) tích hợp sẵn trên SoC như ESP32 giúp tối giản PCB và tiết kiệm chi phí.
- Không có điểm lỗi duy nhất: Khác Wi-Fi/Bluetooth phụ thuộc một Router/Hub cố định, mạng Thread Mesh tự bầu cấu trúc quản lý; nếu một

Router hỏng, mạng vẫn hoạt động và tín hiệu báo cháy vẫn được truyền liên tục.

Từ những phân tích trên, Thread là giải pháp tối ưu đảm bảo tính thời gian thực, độ tin cậy của tuyến truyền trong môi trường sự cố phức tạp, đồng thời giải quyết bài toán tiết kiệm năng lượng cho các nút cảm biến chạy pin.

2.1.2 Kiến trúc mạng Thread

Kiến trúc mạng Thread dựa trên nền IPv6, hoạt động trên lớp MAC của chuẩn IEEE 802.15.4. Không phụ thuộc ZigBee hay Bluetooth LE, Thread cung cấp mạng lưới dạng mesh hoàn chỉnh với các đặc tính: đơn giản, bảo mật (xác thực và mã hóa toàn bộ thiết bị và lưu lượng), tin cậy (tự khắc phục, không có điểm lỗi duy nhất, chống nhiễu), hiệu quả năng lượng (chạy pin nhiều năm) và mở rộng tới hàng trăm thiết bị.



Hình 2.1 Sơ đồ kiến trúc phân cấp trong mạng Thread

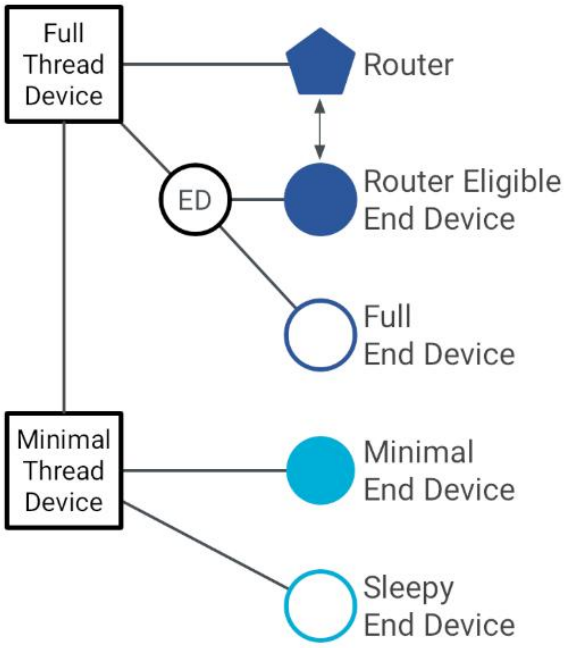
Hình 2.1 thể hiện sự tinh giản của ngăn xếp giao thức Thread qua các đặc điểm cốt lõi:

- Tầng Vật lý (PHY) và Liên dữ liệu (MAC): Theo chuẩn IEEE 802.15.4 băng tần 2.4 GHz, đảm nhiệm mã hóa dữ liệu vô tuyến, truyền tín hiệu và quản lý truy cập kênh tránh xung đột.
- Tầng Mạng: Tích hợp 6LoWPAN nén tiêu đề gói IPv6 và phân đoạn dữ liệu vừa khung truyền giới hạn, cho phép định vị nút trực tiếp bằng địa chỉ IPv6.
- Tầng Giao vận và Ứng dụng: Dùng UDP truyền không hướng kết nối để giảm độ trễ và dòng tiêu thụ cho SED; cơ chế bảo mật và giao thức MLE vận hành song song làm nền cho tầng ứng dụng (CoAP, Matter).

Về kiến trúc, Thread chia thiết bị theo hai vai trò chính: bộ định tuyến (Router) chuyển tiếp gói tin, cấp quyền cho thiết bị mới và luôn bật thu phát; thiết bị đầu cuối (End Device) chỉ giao tiếp với một Router, không chuyển tiếp và có thể tắt thu phát để tiết kiệm điện. Theo đặc tả, mỗi phân vùng mạng Thread có tối đa 32 Router luôn bật, còn các nút lá luôn cách một Router đúng một bước nhảy và được phép ngủ luân phiên; ràng buộc kiến trúc này tách việc định tuyến khỏi

hoạt động tiết kiệm năng lượng, giúp mạng vừa tin cậy vừa kéo dài tuổi thọ pin cho thiết bị đầu cuối [2]. Thread định nghĩa hai loại thiết bị chính:

- Thiết bị Thread đầy đủ (FTD): Luôn bật thu phát, đăng ký địa chỉ, phát đa hướng và duy trì liên kết IPv6. Gồm ba loại:
  - Bộ định tuyến (Router): nút lõi tham gia mạng mesh, chuyển tiếp gói tin, cấp địa chỉ cho thiết bị con và duy trì liên kết mạng; luôn bật thu phát (~15 mA), thường cấp nguồn liên tục nên không phù hợp chạy pin. Trong đồ án, Router chạy trên ESP32-H2, trung chuyển dữ liệu giữa Border Router và các SED, đồng thời đọc cảm biến SHT31 và CCS811.
  - Thiết bị đầu cuối đủ điều kiện làm bộ định tuyến (REED): có thể được nâng cấp thành Router khi cần.
  - Thiết bị đầu cuối đầy đủ (FED): không thể nâng cấp lên Router.
- Thiết bị Thread tối thiểu (MTD): Không đăng ký địa chỉ đa hướng, chỉ chuyển mọi thông báo tới thiết bị mẹ. Gồm:
  - Thiết bị đầu cuối tối thiểu (MED): bộ thu phát luôn bật, không cần thăm dò.
  - Thiết bị đầu cuối dạng ngủ (SED): tắt thu phát hầu hết thời gian, chỉ thức dậy định kỳ hoặc khi có sự kiện cảm biến; khi ngủ sâu chỉ tiêu thụ vài µA. SED tối ưu cho nút cảm biến chạy pin, tuổi thọ 1–2 năm. Đồ án dùng 3 SED gửi cảnh báo lên Router qua CoAP để phát hiện cháy sớm tại các vị trí khác nhau.



Hình 2.2 Sơ đồ phân loại các thiết bị trong mạng Thread

Hình 2.2 thể hiện sự phân cấp thiết bị của mạng Thread thành hai nhóm FTD và MTD.

Ngoài ra trong mạng Thread còn có các vai trò đặc biệt sau:

- Leader: một Router được tự động bầu để quản lý nhóm Router, đảm bảo chịu lỗi và phân phối thông tin cấu hình toàn mạng.
- Bộ định tuyến biên (Border Router): vừa là Router trong mạng Thread vừa kết nối mạng Thread với mạng IP ngoài (Wi-Fi, Ethernet), chuyển tiếp gói tin giữa hai miền (hỗ trợ DHCPv6 và NAT64 khi cần) và thường là điểm ra Internet duy nhất. Trong đồ án, Border Router dùng ESP32 WROOM 32 kết hợp RCP (ESP32-H2), kết nối Wi-Fi để đẩy dữ liệu lên MQTT broker và web dashboard.

Về địa chỉ, Thread dùng IPv6 với ba phạm vi unicast chính: Link-Local (fe80::/16), Mesh-Local (fd00::/8) và Global; mỗi thiết bị có thể có nhiều địa chỉ cho các mục đích khác nhau:

- Bộ định vị định tuyến (RLOC): xác định vị trí thiết bị trong cấu trúc mạng, tạo từ ID router và ID con; thay đổi khi cấu trúc mạng thay đổi.
- Định danh điểm cuối (ML-EID): địa chỉ duy nhất định danh một giao diện Thread trong toàn phân vùng, độc lập với cấu trúc mạng nên được ưu tiên dùng để liên lạc bền vững.
- Địa chỉ Global Unicast (GUA): có phạm vi toàn cầu, cho phép thiết bị Thread giao tiếp trực tiếp với máy chủ bên ngoài.

Quá trình hình thành mạng bắt đầu bằng cơ chế Beacon: thiết bị quét kênh 802.15.4, gửi yêu cầu và nhận beacon chứa PAN ID, XPAN ID, tên mạng, rồi tham gia hoặc tạo mạng mới. Cốt lõi là giao thức MLE đảm nhiệm khám phá, đánh giá chất lượng, thiết lập liên kết và thương lượng tham số, đồng thời phổ biến dữ liệu mạng và tuyến đường.

Để duy trì ổn định, Thread dùng Tập con chi phối kết nối (CDS), yêu cầu:

- Có đường dẫn chỉ dành cho bộ định tuyến giữa hai bộ định tuyến bất kỳ.
- Mọi bộ định tuyến đều có thể kết nối với nhau mà không cần qua thiết bị đầu cuối.
- Mọi thiết bị đầu cuối kết nối trực tiếp với một bộ định tuyến.

Thuật toán phân tán tự thêm/bớt router khi cấu trúc mạng thay đổi để đảm bảo dư thừa tối thiểu và khả năng mở rộng. Thiết bị mới tham gia với vai trò đầu cuối; khi cần, REED tự nâng cấp thành router qua MLE.

Tóm lại, kiến trúc Thread với vai trò thiết bị linh hoạt, địa chỉ IPv6 đa tầng, giao thức MLE và khả năng tự phục hồi qua CDS tạo nền tảng vững chắc cho các ứng dụng IoT quy mô lớn, đặc biệt là hệ thống báo cháy kho bãi yêu cầu độ tin cậy và mở rộng cao.

2.1.3 Giao thức 6LoWPAN và định tuyến

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) là lớp thích ứng do IETF chuẩn hóa (RFC 4944), cho phép truyền gói IPv6 hiệu quả trên các liên kết IEEE 802.15.4 công suất thấp -nền tảng vô tuyến của Thread [3]. Kết hợp với cơ chế định tuyến lưới tự phục hồi, 6LoWPAN tạo nên xương sống vừa tin cậy vừa tiết kiệm năng lượng cho hệ thống báo cháy kho bãi.

a, Tổng quan về 6LoWPAN – cầu nối giữa IPv6 và mạng không dây công suất thấp

IEEE 802.15.4 bị giới hạn nghiêm ngặt: khung tin chỉ 127 byte, tốc độ 250 kbps ở băng 2,4 GHz, tầm phủ ngắn và truyền đa chặng, chạy trên vi điều khiển chỉ vài KB RAM. Trong khi đó gói IPv6 yêu cầu MTU tối thiểu 1280 byte, nên không thể đưa IPv6 trực tiếp lên 802.15.4 nếu thiếu một lớp trung gian [3]. 6LoWPAN chèn lớp thích ứng giữa tầng liên kết và tầng mạng, giảm mạnh chi phí phụ trợ của IP thông qua ba chức năng chính:

- Nén header: nén header IPv6 (40 byte) và UDP (8 byte) xuống còn vài byte bằng cách lược bỏ các trường suy ra được (địa chỉ Link-Local lấy từ MAC, các trường mặc định), nhờ đó tiết kiệm băng thông và giảm thời gian radio hoạt động .
- Phân mảnh và tái hợp: khi gói IPv6 vẫn lớn hơn khung 127 byte sau khi nén, 6LoWPAN chia gói thành nhiều mảnh kèm Datagram Tag và Offset để đầu nhận ghép lại .
- Chuyển tiếp lớp hai (mesh forwarding): 6LoWPAN thêm Mesh Header chứa địa chỉ nguồn/đích của từng bước nhảy, cho phép một gói IPv6 đi qua nhiều chặng vô tuyến để tới đích.

b, Định tuyến trong mạng Thread – cơ chế tự phục hồi

Trên nền 6LoWPAN, Thread tự tổ chức và tự phục hồi nhờ giao thức Mesh Link Establishment (MLE). MLE thiết lập, cấu hình và duy trì các liên kết an toàn, hoạt động như một giao thức định tuyến distance-vector: khám phá mạng qua Beacon, thiết lập quan hệ Cha–Con (Parent Request/Response, cấp RLOC16) và phổ biến cấu trúc liên kết, danh sách Router cùng tuyến ra ngoài tới mọi thiết bị.

Các Router liên tục trao đổi MLE để cập nhật bảng định tuyến; mỗi tuyến có chi phí tính theo chất lượng liên kết, số bước nhảy và mức tắc nghẽn, thiết bị luôn chọn tuyến chi phí thấp nhất. Nhờ cơ chế động này, khi một Router mất nguồn, các Router lân cận phát hiện, lan truyền sự kiện qua MLE, loại bỏ tuyến cũ và nhanh chóng định tuyến lại để giữ kết nối không gián đoạn.

Từ đó ta có thể thấy 6LoWPAN là lớp keo đưa IPv6 chạy hiệu quả trên phần cứng hạn chế của mạng cảm biến; kết hợp định tuyến và tự phục hồi của MLE, nó tạo nền tảng vừa tin cậy vừa tiết kiệm năng lượng cho hệ thống báo cháy kho bãi.

2.2 Giao thức CoAP và MQTT trong truyền thông IoT

2.2.1 Lý do lựa chọn giao thức CoAP và MQTT

Trong hệ thống IoT, lựa chọn giao thức truyền thông quyết định hiệu quả, khả năng mở rộng và độ tin cậy. Hai giao thức phổ biến nhất là MQTT (Message Queuing Telemetry Transport) và CoAP (Constrained Application Protocol); cả hai đều phù hợp môi trường băng thông hạn chế, tài nguyên eo hẹp nhưng có thế mạnh riêng cho các mô hình truyền thông khác nhau.

Sự kết hợp lai (hybrid) giữa CoAP và MQTT tận dụng ưu điểm của từng giao thức tại các phân đoạn mạng khác nhau, khắc phục hạn chế của HTTP truyền thống:

- Tối ưu tài nguyên mạng không dây (CoAP): khác HTTP tốn năng lượng duy trì bắt tay TCP, CoAP dùng gói nhị phân nhỏ gọn trên UDP, giúp nút cảm biến chạy pin truyền nhanh, giảm thời gian bật anten và tiết kiệm pin.
- Đảm bảo thời gian thực (MQTT): ở phân đoạn từ Border Router lên Cloud, cơ chế Publish/Subscribe của MQTT giúp Dashboard nhận dữ liệu ngay khi có thay đổi mà không cần polling, giảm tải máy chủ.
- Phân tách phân đoạn hợp lý: CoAP lo "băng thông hẹp, năng lượng thấp" ở mạng Mesh nội bộ, còn MQTT lo "kết nối diện rộng, dữ liệu lớn" khi ra Internet.

Để làm rõ lý do lựa chọn hai công nghệ này, bảng dưới đây so sánh định lượng các thông số kỹ thuật cốt lõi giữa CoAP, MQTT và HTTP:

Bảng 2.2 So sánh các giao thức truyền tin phổ biến

Đặc tính kỹ thuật	**Giao thức CoAP**	**Giao thức MQTT**	**Giao thức HTTP**
Giao thức tầng vận chuyển	UDP	TCP	TCP
Kiến trúc truyền thông	Request / Response	Publish / Subscribe	Request / Response
Kích thước Header tối thiểu	4 Bytes	2 Bytes	Hàng trăm Bytes (Văn bản)
Mức tiêu thụ năng lượng	Rất thấp (Tối ưu cho pin)	Thấp đến trung bình	Rất cao
Cơ chế chất lượng dịch vụ	Hỗ trợ QoS cơ bản (Confirmable)	Hỗ trợ 3 mức QoS (0, 1, 2)	Không hỗ trợ trực tiếp
Môi trường phù hợp	Mạng Mesh nội bộ, nút nguồn yếu	Kết nối Internet, Cloud Dashboard	Ứng dụng Web truyền thống

Việc kết hợp đồng bộ giao thức CoAP tại mạng lưới cảm biến biên và MQTT tại phân đoạn kết nối đám mây là giải pháp kiến trúc tối ưu, vừa đáp ứng nghiêm ngặt tiêu chí tiết kiệm năng lượng cho các nút mạng dùng pin, vừa đảm bảo khả năng giám sát trực quan thời gian thực với độ trễ thấp và độ tin cậy cao khi xảy hỏa hoạn.

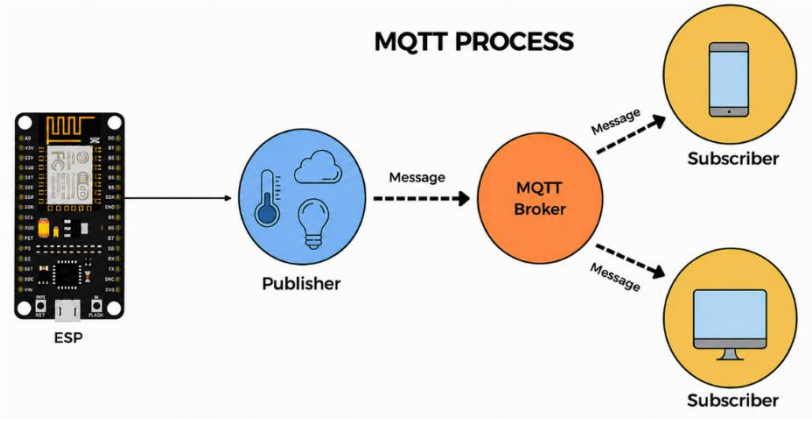
2.2.2 Giao thức MQTT

a, Giới thiệu chung

MQTT là giao thức truyền thông theo mô hình publish/subscribe, do Andy Stanford-Clark (IBM) và Arlen Nipper (Arcom) phát triển năm 1999. Chạy trên TCP/IP, tối ưu cho thiết bị tài nguyên hạn chế trong mạng không ổn định, băng thông thấp; nhờ đó trở thành chuẩn thực tế cho IoT từ nhà thông minh, giám sát công nghiệp đến cảnh báo sớm [4].

b, Kiến trúc Publish/Subscribe và thành phần chính

Để minh họa phương thức vận chuyển thông tin trong tầng ứng dụng của hệ thống, sơ đồ kiến trúc truyền thông của giao thức MQTT được thể hiện trực quan tại hình 2.3.



Hình 2.3 Mô hình cấu trúc mạng truyền thông MQTT

Khác mô hình client-server (yêu cầu–phản hồi), MQTT dùng publish/subscribe: publisher và subscriber tách biệt, không biết nhau, mọi truyền thông qua một MQTT Broker trung tâm; nhờ sự tách biệt này, publisher và subscriber không cần hoạt động đồng thời, cho phép truyền thông bất đồng bộ, dễ mở rộng và tiết kiệm năng lượng [4]. Kiến trúc gồm ba thành phần chính:

- Publisher (bên xuất bản): tạo và gửi dữ liệu lên Broker kèm một topic, không cần biết ai nhận. Trong đồ án, Border Router (ESP32 WROOM 32) là Publisher, đẩy dữ liệu cảm biến lên HiveMQ Cloud qua các topic như sensor/Router_H2, sensor/SED_1.
- Subscriber (bên đăng ký): đăng ký các topic quan tâm và được Broker chuyển message khi có dữ liệu mới. Trong đồ án, web dashboard đăng ký sensor/# để nhận dữ liệu từ mọi nút; bot Telegram trên Railway cũng là Subscriber, đăng ký các topic cảm biến để tự gửi cảnh báo khi nhiệt độ vượt ngưỡng.
- Broker (trung gian): máy chủ trung tâm tiếp nhận, lọc và phân phối message theo topic, đồng thời quản lý phiên, xác thực, bảo mật và lưu message khi subscriber offline. Trong đồ án, HiveMQ Cloud là Broker, cho phép Border Router và dashboard kết nối từ các mạng khác nhau.
- Topic (chủ đề): chuỗi UTF-8 phân cấp để định tuyến message (vd: sensor/Router_H2/temperature); ký tự + (một cấp) và # (nhiều cấp) cho phép đăng ký linh hoạt. Dashboard dùng sensor/# để nhận dữ liệu mọi nút.

c, Chất lượng dịch vụ (QoS)

MQTT có ba mức chất lượng dịch vụ (QoS) cân bằng giữa độ tin cậy và hiệu quả [4]:

- QoS 0 – At most once: gửi tối đa một lần, không xác nhận; nhanh nhất nhưng có thể mất message. Phù hợp dữ liệu không quan trọng, gửi định kỳ (vd nhiệt độ bình thường).
- QoS 1 – At least once: gửi ít nhất một lần, có xác nhận (PUBACK), có thể trùng lặp. Phù hợp dữ liệu quan trọng chấp nhận trùng.

- QoS 2 – Exactly once: gửi đúng một lần qua bắt tay hai bước (PUBREC, PUBREL, PUBCOMP), tin cậy cao nhất nhưng tốn tài nguyên. Phù hợp cảnh báo khẩn cấp như phát hiện cháy.

Chọn mức QoS phù hợp giúp cân bằng độ tin cậy và hiệu quả: dữ liệu cảnh báo cháy nên dùng QoS 2 để không mất, còn nhiệt độ/độ ẩm thông thường dùng QoS 0 hoặc 1 để giảm tải và tiết kiệm năng lượng.

d, Các phiên bản và MQTT-SN

Phiên bản phổ biến nhất là MQTT 3.1.1 (OASIS chuẩn hóa 2014). Năm 2019, MQTT 5.0 ra đời với nhiều cải tiến (reason codes, shared subscription, thuộc tính mở rộng). Với thiết bị không dùng TCP (Zigbee, LoRaWAN), biến thể nhẹ MQTT-SN cho phép MQTT chạy trên mạng không hỗ trợ TCP/IP trực tiếp: MQTT-SN dùng UDP và một gateway để chuyển đổi message sang broker MQTT truyền thống, đồng thời quản lý thời gian thức/ngủ của radio nên tập trung tiêu thụ năng lượng tại gateway -rất phù hợp mạng cảm biến không dây.

e, Ứng dụng trong hệ thống

Trong đồ án, MQTT dùng ở tầng đám mây kết nối Border Router với HiveMQ Cloud và web dashboard. Border Router là publisher, đẩy dữ liệu cảm biến (nhiệt độ, độ ẩm, eCO_2, TVOC, trạng thái khói) lên topic sensor/<node_id>. Web dashboard (JavaScript, MQTT over WebSocket) là subscriber, đăng ký sensor/# để nhận dữ liệu thời gian thực; ngoài ra bot Telegram trên Railway cũng là subscriber, đăng ký các topic cảm biến và tự gửi cảnh báo khi nhiệt độ vượt ngưỡng.

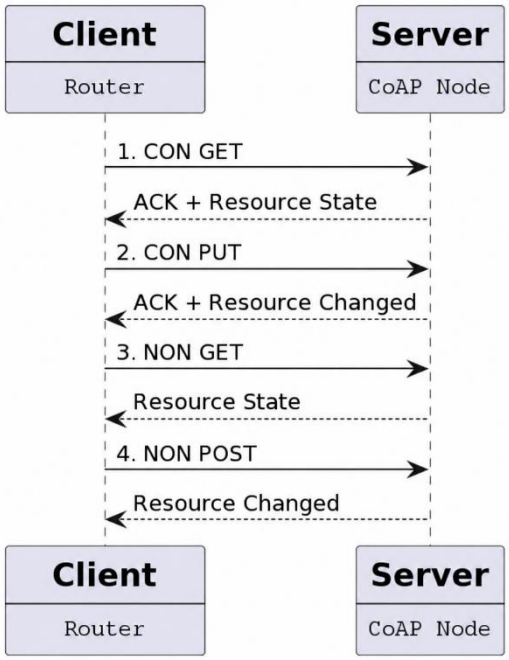
2.2.3 Giao thức CoAP

a, Giới thiệu chung

CoAP (Constrained Application Protocol) là giao thức của IETF (RFC 7252) cho thiết bị IoT tài nguyên hạn chế hoạt động trong môi trường mạng dễ mất gói, được xây dựng trên kiến trúc REST [5]. Chạy trên UDP thay vì TCP để giảm overhead, theo mô hình request/response như HTTP nhưng message rất nhỏ (header 4 byte), rất phù hợp mạng Thread với gói tin nhỏ, tần suất thấp và yêu cầu tiết kiệm năng lượng.

b, Kiến trúc và đặc điểm

CoAP có kiến trúc tối giản nhưng đủ tính năng cho truyền thông trong môi trường tài nguyên hạn chế.



Hình 2.4 Mô hình tương tác giữa Client và Server sử dụng CoAP

Hình 2.4 minh họa cấu trúc gói tin CoAP tinh giản trên nền UDP/IPv6, với các trường cốt lõi: mã lệnh (Code: GET/POST), Message ID và Token, giúp hai vi điều khiển đồng bộ và truyền nhận thông tin chính xác mà không quá tải băng thông.

Các đặc điểm kiến trúc cốt lõi của CoAP:

- Hoạt động trên UDP: giảm overhead bắt tay và kiểm soát tắc nghẽn, phù hợp mạng không dây giới hạn băng thông và năng lượng. Do UDP vốn không đảm bảo tin cậy và dễ tắc nghẽn, CoAP bổ sung cơ chế kiểm soát tắc nghẽn mặc định dựa trên thuật toán lùi nhị phân theo hàm mũ (binary exponential backoff) [5].
- Mô hình request/response: dùng GET, POST, PUT, DELETE như HTTP; mỗi tài nguyên định danh bằng URI (vd coap://[addr]/sensor).
- Header nhẹ (4 byte): gồm Version, Type, Token, Code, Message ID -tiết kiệm hơn nhiều so với header TCP (20 byte) hay IPv6 (40 byte).
- Các loại message: CON (có xác nhận), NON (không xác nhận), ACK (xác nhận), RST (báo lỗi); CON được gửi lại nếu không nhận ACK, đảm bảo độ tin cậy khi cần.
- Cơ chế quan sát (Observe, RFC 7641): client đăng ký một tài nguyên và nhận cập nhật tự động khi thay đổi -tương tự publish/subscribe của MQTT; ngoài ra CoAP còn hỗ trợ khám phá tài nguyên (resource discovery) qua /.well-known/core [5].

Nhờ các đặc điểm này, CoAP là giải pháp lý tưởng cho mạng cảm biến không dây và mạng Thread, nơi mỗi byte và mỗi mili giây radio đều ảnh hưởng tới hiệu suất và tuổi thọ pin.

c, Ứng dụng trong hệ thống

Trong đồ án, CoAP dùng cho truyền thông nội bộ mạng Thread (giữa SED, Router, Border Router): SED và Router định kỳ gửi dữ liệu qua CoAP POST tới URI /sensor với payload JSON; Border Router đóng vai trò CoAP server nhận, xử lý và chuyển tiếp lên MQTT broker. CoAP giúp tiết kiệm năng lượng cho SED và hỗ trợ multicast khi cần mở rộng.

2.2.4 So sánh MQTT và CoAP

Cả MQTT và CoAP đều phổ biến trong IoT, mỗi giao thức có thế mạnh riêng. Bảng dưới đây so sánh các khác biệt chính:

Bảng 2.3 So sánh MQTT và CoAP

Tiêu chí	MQTT	CoAP
Mô hình truyền thông	Publish/Subscribe (qua Broker)	Request/Response (client-server), có thể mở rộng Observe
Giao thức vận chuyển	TCP	UDP
Kích thước header	2 byte (fixed header) + tùy chọn	4 byte
Hỗ trợ multicast	Không (phải dùng broker trung gian)	Có (tận dụng UDP multicast)
Độ tin cậy	3 mức QoS (0,1,2)	CON (có xác nhận) / NON (không xác nhận)
Bảo mật	TLS (MQTT over TLS)	DTLS (CoAP over DTLS)
Tương tự với Web	Không	RESTful, dễ tích hợp với HTTP qua proxy
Phù hợp cho	Ứng dụng cần trung gian, nhiều subscriber, cloud	Mạng lưới mesh, máy chủ nhúng, thiết bị cực kỳ hạn chế về tài nguyên
Ví dụ ứng dụng	Giám sát từ xa, IoT cloud platform, thông báo đẩy	Mạng cảm biến không dây, giao tiếp giữa các nút trong mạng Thread, Zigbee, 6LoWPAN

Lựa chọn cho hệ thống: đồ án dùng kết hợp cả hai: CoAP cho truyền thông nội bộ mạng Thread (nhẹ, UDP, phù hợp mesh) và MQTT cho truyền từ Border Router lên đám mây, web dashboard (publish/subscribe linh hoạt, dễ mở rộng và tích hợp).

2.3 Vi điều khiển và nền tảng phát triển

2.3.1 ESP32-H2

a, Lý do lựa chọn ESP32-H2

ESP32-H2 là vi điều khiển thế hệ mới của Espressif, một SoC kết hợp Bluetooth LE 5 và IEEE 802.15.4 hỗ trợ Thread, Matter và Zigbee, chuyên cho nhà thông minh và mạng cảm biến công suất thấp [6]. Chip được chọn làm lõi xử lý và truyền thông cho các nút (SED, Router) nhờ các ưu thế sau:

- Tích hợp IEEE 802.15.4 và BLE 5.3: có sẵn bộ thu phát chuẩn IEEE 802.15.4 để chạy trực tiếp OpenThread mà không cần chip ngoài; BLE hỗ trợ provisioning qua điện thoại.
- Kiến trúc RISC-V 32-bit: lõi RISC-V một nhân tới 96 MHz, đủ mạnh cho xử lý dữ liệu cảm biến mà vẫn gọn nhẹ, tiêu thụ ít hơn kiến trúc Xtensa cũ.
- Quản lý năng lượng Light-sleep: hỗ trợ ngủ sâu dòng rất thấp nhưng vẫn giữ kết nối Thread và thức dậy ngay qua ngắt GPIO, phù hợp nút SED chạy pin AA.
- Hệ sinh thái ESP-IDF và OpenThread chính thức: Espressif hỗ trợ đầy đủ qua ESP-IDF với thư viện OpenThread cập nhật liên tục, rút ngắn thời gian triển khai.

Để làm rõ tính tối ưu khi lựa chọn, bảng dưới đây so sánh hiệu năng giữa ESP32-H2 và một số dòng vi điều khiển phổ biến có hỗ trợ phần cứng mạng Thread hiện nay (như nRF52840 của Nordic Semiconductor và STM32WB55 của STMicroelectronics):

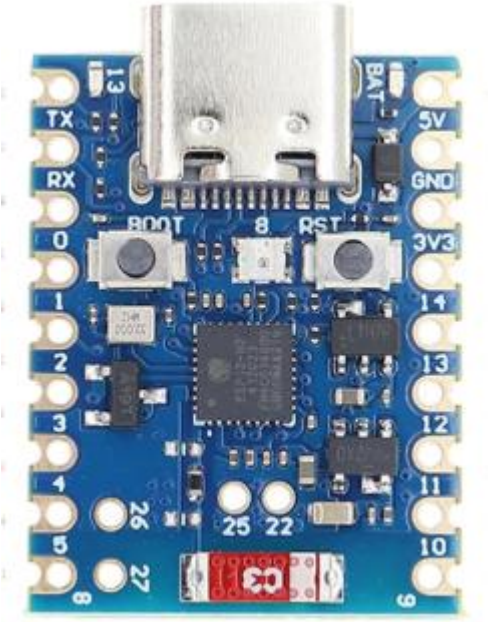
Bảng 2.4 So sánh các vi điều khiển hỗ trợ mạng Thread

Thông số kỹ thuật	**ESP32-H2 (Espressif)**	**nRF52840 (Nordic)**	**STM32WB55 (ST)**
Kiến trúc Lõi (Core)	RISC-V 32-bit (96 MHz)	ARM Cortex-M4 (64 MHz)	song song ARM Cortex-M4 & M0+
Kết nối vô tuyến	IEEE 802.15.4 (Thread/Zigbee), BLE 5.3	IEEE 802.15.4, BLE 5.4, ANT	IEEE 802.15.4, BLE 5.4
Bộ nhớ Flash / RAM	Up to 4 MB (External) / 320 KB	1 MB / 256 KB	Up to 1 MB / 256 KB
Hệ sinh thái phần mềm	ESP-IDF / OpenThread gốc	nRF Connect SDK	STM32CubeWB
Hiệu quả kinh tế	Rất cao (Giá thành tối ưu)	Trung bình (Giá chip cao)	Trung bình

Việc lựa chọn vi điều khiển ESP32-H2 mang lại sự cân bằng hoàn hảo giữa hiệu suất xử lý phần cứng RISC-V mạnh mẽ, khả năng quản lý năng lượng tối ưu cho nguồn pin và đặc biệt là sự hỗ trợ đầy đủ các thư viện OpenThread chính thức từ nhà sản xuất, giúp đảm bảo tính ổn định cao và giảm thiểu tối đa thời gian phát triển hệ thống.

b, Sơ đồ chân module ESP32-H2 Super mini

Module ESP32-H2 Super Mini có các chân được bố trí như trong hình 2.5.



Hình 2.5 Module ESP32-H2 Supermini

Bảng dưới đây liệt kê 22 chân chính của module kèm theo chức năng tương ứng.

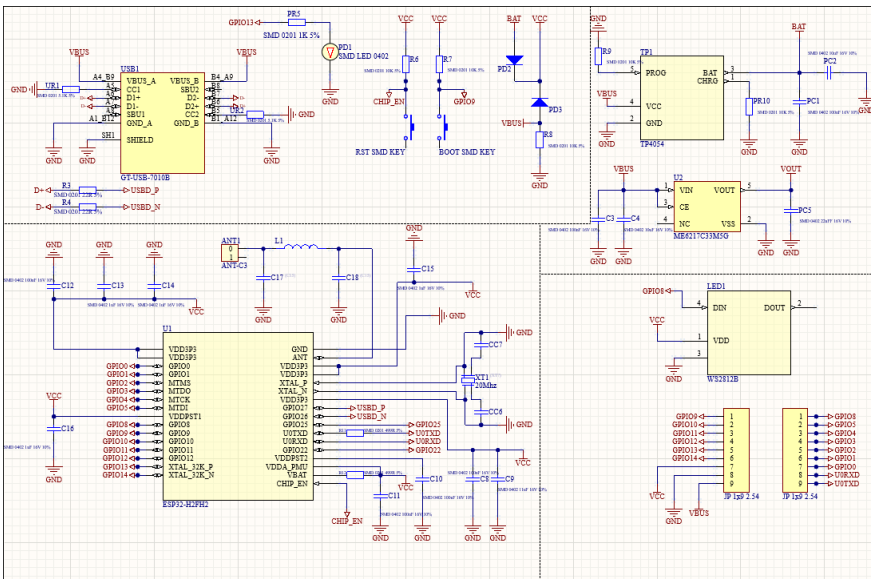
Bảng 2.5 Các chân module ESP32-H2 Supermini

Tên chân	Chức năng
TX	U0TXD (UART0 transmit, GPIO23)
RX	U0RXD (UART0 receive, GPIO22)
0	GPIO0 (có thể dùng để chọn chế độ boot)
1	GPIO1 (chức năng đa năng)
2	GPIO2 (chức năng đa năng)
3	GPIO3 (chức năng đa năng)
4	GPIO4 (chức năng đa năng)
5	GPIO5 (chức năng đa năng)
8	GPIO8 (strapping pin, có thể ảnh hưởng boot mode)
5V	Đầu vào nguồn 5V (từ USB hoặc nguồn ngoài)
GND	Đất
3V3	Đầu ra nguồn 3,3V (từ onboard LDO)
14	GPIO14 (chức năng đa năng)
13	GPIO13 (chức năng đa năng)
12	GPIO12 (chức năng đa năng)
11	GPIO11 (chức năng đa năng)
10	GPIO10 (chức năng đa năng)
9	GPIO9 (strapping pin, có pull-up nội)
22	GPIO22 (cũng là U0RXD)
25	GPIO25 (strapping pin liên quan JTAG)
26	GPIO26 (USB_D-)
27	GPIO27 (USB_D+)

Cách bố trí chân của ESP32-H2 Super Mini tối ưu cho ứng dụng nhỏ gọn, đủ tín hiệu nguồn, đất, UART, GPIO và USB; các GPIO đa năng cho phép kết nối linh hoạt nhiều cảm biến. Cấu trúc này phù hợp cả khi ESP32-H2 làm nút SED lẫn RCP trong mạng Thread.

c, Các giao tiếp ngoại vi chính

Sơ đồ nguyên lý kết nối ESP32-H2 với các khối ngoại vi, mạch nguồn, LED và nút nhấn được thể hiện ở hình 2.6.



Hình 2.6 Sơ đồ nguyên lý vi điều khiển ESP32-H2

Sơ đồ nguyên lý tập trung quanh ESP32-H2 với các kết nối và ngoại vi cốt lõi:

- Mạch Anten và RF: tín hiệu từ chân ANT qua mạng phối hợp trở kháng (cuộn cảm L1, tụ C17, C18) ra anten chip, tối ưu công suất phát 2.4 GHz cho mạng Mesh Thread.
- Giao tiếp I²C và GPIO ngắt ngoài: cấu hình bus I²C dùng GPIO5 (SCL) và GPIO4 (SDA) để đọc SHT31 và CCS811.
- UART và USB: cổng USB Type-C nối trực tiếp vào chân USB của chip qua điện trở bảo vệ 22 Ω để nạp code và debug không cần IC chuyển đổi; UART0 cũng đưa ra chân cắm để xuất log khi cần.

Các giao tiếp ngoại vi được dùng trong đồ án:

- I²C (bus 0): kết nối SHT31 và CCS811 (100 kHz); RE46C194 không dùng I²C nên chỉ hai thiết bị trên bus.
- GPIO ngắt ngoài: GPIO11 và GPIO14 làm nguồn đánh thức từ light-sleep (gpio_wakeup_enable, esp_light_sleep_start).
- UART0: ghi log debug và nạp code (qua USB-to-UART); chân U0TXD (GPIO23) và U0RXD (GPIO22) đưa ra header.
- USB Serial/JTAG: tích hợp sẵn, nạp code và debug qua USB mà không cần UART ngoài.
- SPI / I2S / RMT: Không sử dụng trong đồ án này.

Các giao tiếp cần thiết được khai thác đầy đủ, trong đó I²C và GPIO wake-up là hai yếu tố then chốt để vận hành hệ thống báo cháy tiết kiệm năng lượng.

d, Thông số kỹ thuật

Theo datasheet, ESP32-H2 gồm CPU RISC-V 32-bit, SRAM 320 KB, bộ nhớ LP 4 KB (giữ dữ liệu khi ngủ sâu), flash SiP 2/4 MB, các ngoại vi số, bộ thu phát 2.4 GHz (BLE và IEEE 802.15.4), các bộ tăng tốc mật mã và bộ quản lý nguồn nhiều chế độ ngủ. Bảng dưới tóm tắt các thông số quan trọng.

Bảng 2.6 Thông số kỹ thuật ESP32-H2 Supermini

Thông số	Giá trị	Ghi chú
CPU	32-bit RISC-V, lên đến 96 MHz	Nhân đơn, pipeline 4 tầng
SRAM	320 KB	Chia sẻ dữ liệu và lệnh
LP memory	4 KB	Giữ dữ liệu trong deep-sleep
Flash	2 MB hoặc 4 MB (SiP)	Tích hợp trong gói
Giao thức không dây	Bluetooth 5 (LE, mesh), IEEE 802.15.4 (Thread, Zigbee, Matter)	Hỗ trợ cả 1 Mbps, 2 Mbps, Coded PHY (125/500 Kbps)
GPIO	19 chân	Có thể cấu hình chức năng, có điện trở kéo lên/kéo xuống nội
Chế độ nguồn	Active, Modem-sleep, Light-sleep, Deep-sleep	Dòng tiêu thụ ở light-sleep ~0.2 mA
Nhiệt độ hoạt động	–40°C … +105°C	

Với hiệu năng vừa đủ (96 MHz, 320 KB SRAM) và tiêu thụ cực thấp ở chế độ ngủ, ESP32-H2 đáp ứng tốt cả vai trò Router luôn hoạt động lẫn SED cần kéo dài thời lượng pin.

e, Các chế độ nguồn và quản lý năng lượng

ESP32-H2 hỗ trợ nhiều chế độ nguồn để tối ưu pin cho SED:

- Active: CPU và radio hoạt động, dòng cao (vài chục mA); dùng cho Router hoặc khi SED xử lý sự cố.
- Light-sleep: CPU tạm dừng, ngoại vi (I^2C, GPIO) giữ trạng thái, radio có thể tắt nhưng vẫn giữ kết nối Thread (poll định kỳ); dòng ~0,2 mA -chế độ chính của SED khi chờ.
- Deep-sleep: CPU và hầu hết ngoại vi tắt, chỉ giữ LP memory và RTC; dòng ~5 µA. Tuy nhiên để duy trì kết nối Thread, SED phải thức mỗi vài chục giây nên ưu tiên light-sleep.

Trong đồ án, Router luôn ở active mode; SED dùng light-sleep kết hợp timer wake-up (25 giây) và GPIO wake-up từ ALERT (GPIO11) và nINT (GPIO14), tiết kiệm năng lượng tối đa mà vẫn phản ứng nhanh với sự cố.

f, Hỗ trợ giao thức Thread

ESP32-H2 có radio IEEE 802.15.4 (2.4 GHz, 250 Kbps) và ngăn xếp OpenThread tích hợp sẵn trong ESP-IDF, cho phép:

- Tạo mạng Thread mesh tự phục hồi, mở rộng tới hàng trăm thiết bị.

- Dùng CoAP (trên UDP) truyền dữ liệu cảm biến giữa SED, Router và Border Router.
- Hỗ trợ chế độ SED với polling để tiết kiệm năng lượng.

Nhờ Espressif hỗ trợ chính thức OpenThread, việc phát triển ứng dụng mạng Thread trên ESP32-H2 dễ dàng, ổn định và rút ngắn thời gian triển khai.

g, Ứng dụng trong hệ thống báo cháy

- Router: ESP32-H2 làm bộ định tuyến trong mạng Thread, luôn được cấp nguồn, chạy đầy đủ OpenThread, chuyển tiếp gói tin, cấp địa chỉ cho SED và duy trì mesh; đồng thời đọc cảm biến và gửi lên Border Router qua CoAP.
- SED (Sleepy End Device): ESP32-H2 ở chế độ ngủ để tiết kiệm pin, chỉ kết nối Router cha, thường ở light-sleep và được đánh thức bởi ngắt ngoại vi; khi thức dậy nó đọc dữ liệu, gửi cảnh báo qua CoAP rồi ngủ lại, phù hợp các nút phân tán chạy pin AA.
- RCP (Radio Co-Processor): trong cấu hình Border Router, ESP32-H2 nạp firmware RCP, chỉ đảm nhiệm tầng PHY/MAC của IEEE 802.15.4 và giao tiếp với ESP32 WROOM 32 qua UART; toàn bộ OpenThread, CoAP, MQTT, web server chạy trên WROOM 32.

Nhờ ba vai trò Router, SED và RCP, ESP32-H2 trở thành nền tảng thống nhất cho toàn hệ thống báo cháy trên mạng Thread, đáp ứng yêu cầu về độ tin cậy, thời lượng pin và khả năng mở rộng.

2.3.2 ESP32 WROOM 32

a, Lý do lựa chọn ESP32 WROOM 32

ESP32 WROOM 32 là SoC mạnh, đa năng thuộc dòng ESP32 của Espressif -tích hợp Wi-Fi 2,4 GHz, Bluetooth và Bluetooth LE -được chọn làm lõi xử lý trung tâm cho Border Router nhờ các đặc tính sau [7]:

- Hiệu suất lõi kép: Xtensa dual-core 32-bit LX6 tới 240 MHz, đủ sức xử lý đồng thời giải mã JSON, điều phối luồng dữ liệu liên tục và chạy Web Server giám sát.
- Tích hợp Wi-Fi và Bluetooth: Wi-Fi 802.11 b/g/n kết nối Internet để publish dữ liệu lên Cloud Broker, kèm Bluetooth để cấu hình mạng ban đầu.
- Giao tiếp ngoại vi phong phú làm chip Host: nhiều UART tốc độ cao, SPI và quản lý ngắt, cho phép kết nối ổn định qua UART với ESP32-H2 đóng vai trò RCP.
- Hệ sinh thái ESP-IDF: hỗ trợ đầy đủ các thư viện mạng, bảo mật TLS/DTLS và tác vụ định thời thời gian thực một cách tin cậy.

Vì bộ định tuyến biên (Border Router) bắt buộc phải sở hữu hai chuẩn kết nối song song (Wi-Fi để lên mạng Internet và IEEE 802.15.4 để quản lý mạng Thread), bảng dưới đây thực hiện so sánh định lượng giữa ESP32 WROOM 32 khi kết hợp với chip RCP so với các dòng vi điều khiển kết nối mạng cao cấp khác:

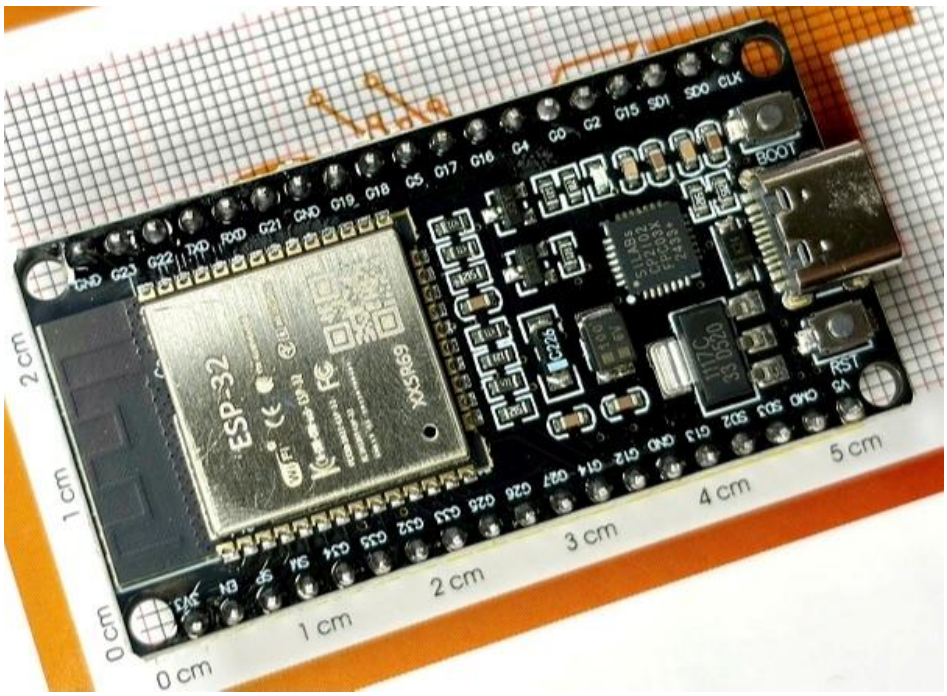
Bảng 2.7 So sánh các vi điều khiển chạy Border Router

Thông số kỹ thuật	ESP32 WROOM 32	STM32WB55 (STMicroelectronics)	nRF5340 (Nordic Semiconductor)
Kiến trúc lõi	Dual-core Xtensa (240 MHz) + Lõi phụ RISC-V	song song ARM Cortex-M4 & M0+	Dual-core ARM Cortex-M33 (128 MHz)
Kết nối Internet	Wi-Fi 2.4GHz tích hợp sẵn	Yêu cầu kết nối thêm module Wi-Fi	Không tích hợp Wi-Fi (Thuần BLE/Thread)
Vai trò tối ưu	Khối xử lý trung tâm Border Router	Bộ điều khiển Gateway cơ bản	Nút mạng Router / End Device nâng cao
Xử lý dữ liệu Cloud	Rất mạnh (Độ trễ thấp, bộ nhớ lớn)	Trung bình (Bị nghẽn ở bus giao tiếp ngoài)	Hạn chế (Không chuyên dụng cho Web/Cloud)
Hiệu quả triển khai	Chi phí thấp, hệ sinh thái phần mềm đồng bộ	Thiết kế mạch phức tạp, giá thành cao	Chi phí chip cao, thiếu kết nối Wi-Fi gốc

Việc lựa chọn module ESP32 WROOM 32 làm chip Host điều phối cho bộ định tuyến biên là giải pháp kiến trúc tối ưu, tận dụng triệt để sức mạnh xử lý lõi kép và kết nối Wi-Fi tích hợp sẵn để xử lý dữ liệu đám mây thời gian thực, đồng thời kết hợp hoàn hảo với hệ sinh thái OpenThread của nhà sản xuất để quản trị toàn vẹn mạng lưới báo cháy không dây.

b, Sơ đồ chân

Module ESP32 WROOM 32 có 38 chân như hình 2.7.



Hình 2.7 Module ESP32 WROOM 32 38 pins out

Bảng dưới liệt kê tên và chức năng từng chân theo tài liệu Espressif, giúp kết nối đúng với ngoại vi.

Bảng 2.8 Các chân module ESP32 WROOM 32

Tên chân	Chức năng
3.3V	Nguồn 3.3V
EN	Reset (Enable)
SP (GPIO23)	GPIO, SPI
SM (GPIO22)	GPIO, I²C
G34 (GPIO34)	GPIO (input-only)
G35 (GPIO35)	GPIO (input-only)
G32 (GPIO32)	GPIO
G33 (GPIO33)	GPIO
G25 (GPIO25)	GPIO, DAC
G26 (GPIO26)	GPIO, DAC
G27 (GPIO27)	GPIO
G14 (GPIO14)	GPIO
G12 (GPIO12)	GPIO
GND	Đất
G13 (GPIO13)	GPIO
SD2 (GPIO9)	Flash (không dùng)
SD3 (GPIO10)	Flash (không dùng)
CMD (GPIO11)	Flash (không dùng)
5V	Nguồn 5V đầu vào
GND	Đất
G23 (GPIO23)	GPIO, SPI
G22 (GPIO22)	GPIO, I²C
TXD (GPIO1)	UART0 TX
RXD (GPIO3)	UART0 RX
G21 (GPIO21)	GPIO, I²C
GND	Đất
G19 (GPIO19)	GPIO, SPI
G18 (GPIO18)	GPIO, SPI
G5 (GPIO5)	GPIO, SPI
G17 (GPIO17)	UART2 TX
G16 (GPIO16)	UART2 RX
G4 (GPIO4)	GPIO
G0 (GPIO0)	GPIO (BOOT)
G2 (GPIO2)	GPIO
G15 (GPIO15)	GPIO
SD1 (GPIO8)	Flash (không dùng)
SDO (GPIO19)	GPIO, SPI
CLK (GPIO6)	Flash (không dùng)

Nắm rõ 38 chân của ESP32 WROOM 32 giúp thiết kế mạch ngoại vi chính xác cho Border Router; lưu ý các chân flash nội (SD1, SD2, SD3, CMD, CLK) không nên dùng, G34/G35 chỉ input, nguồn chính cấp vào chân 5V.

c, Thông số kỹ thuật

Các thông số kỹ thuật nổi bật của ESP32 WROOM 32 được tóm tắt ở bảng dưới.

Bảng 2.9 Thông số kỹ thuật ESP32 WROOM 32

Thông số	Giá trị	Ghi chú
CPU	Xtensa 32-bit LX6, lên đến 240 MHz	Hai nhân, có thể lập trình riêng biệt
Bộ nhớ SRAM	520 KB	Dung lượng lớn cho các ứng dụng phức tạp
Bộ nhớ Flash	4 MB (tích hợp trong module)	Lưu trữ firmware, web server và dữ liệu
Kết nối không dây	Wi-Fi 802.11 b/g/n + Bluetooth v4.2 (BR/EDR + BLE)	Đa dạng lựa chọn kết nối internet
Điện áp hoạt động	2,2V – 3,6V	Thường được cấp nguồn 3,3V hoặc qua chân 5V
Dòng tiêu thụ	Active: ~80 mA *Deep-sleep*: ~5 µA	Linh hoạt trong quản lý năng lượng, dù không phải là ưu tiên hàng đầu
Nhiệt độ hoạt động	–40°C … +85°C	Bền bỉ trong nhiều môi trường
Số chân GPIO	38 chân, trong đó nhiều chân có chức năng đa dụng	Cung cấp khả năng kết nối linh hoạt

Với cấu hình mạnh, ESP32 WROOM 32 đảm nhận đồng thời quản lý mạng, web server, xử lý MQTT và điều khiển toàn bộ luồng dữ liệu hệ thống báo cháy.

d, Ứng dụng trong hệ thống – Vai trò Border Router

Trong đồ án, ESP32 WROOM 32 đảm nhận vai trò Border Router cho mạng Thread, kết nối với một ESP32-H2 nạp firmware RCP qua UART, tạo cầu nối giữa mạng Thread và Wi-Fi.

Các chức năng chính:

- Khởi tạo và quản lý mạng Thread: qua RCP thiết lập mạng với kênh, PAN ID, network key… và trở thành Leader.
- Nhận và chuyển tiếp dữ liệu: nhận gói CoAP từ Router và SED, giải nén, đóng gói lại thành JSON và publish lên HiveMQ Cloud; có thể nhận lệnh MQTT để điều khiển ngược thiết bị.
- Vận hành web server: web server nhúng hiển thị trạng thái các nút dạng card, tự làm mới và cảnh báo bằng âm thanh khi vượt ngưỡng.

- Đồng bộ thời gian: kết nối NTP để giữ thời gian thực, đảm bảo timestamp dữ liệu chính xác.

Với hiệu năng mạnh, Wi-Fi tích hợp và hệ sinh thái phần mềm phong phú, ESP32 WROOM 32 là nền tảng lý tưởng cho Border Router -trung tâm điều phối và kết nối mạng báo cháy với bên ngoài.

2.4 Cảm biến và cơ chế hoạt động

2.4.1 Cảm biến nhiệt độ độ ẩm SHT31

a, Lý do lựa chọn cảm biến SHT31

SHT31 là cảm biến nhiệt độ và độ ẩm số độ chính xác cao của Sensirion, được chọn làm linh kiện thu thập thông số môi trường nền nhờ các đặc tính sau:

- Giao tiếp số I²C tin cậy: hỗ trợ I²C tới 1 MHz với hai địa chỉ tùy chọn và dải nguồn rộng 2,15–5,5 V; dữ liệu được số hóa trong chip nên ít nhiễu hơn cảm biến analog, dễ đọc qua bus chung và giảm số chân GPIO [8].
- Đáp ứng nhanh, chính xác cao: sai số ±0,2 °C và ±2% RH, thời gian phản hồi vài giây -điều kiện để phát hiện sớm tăng nhiệt bất thường khi mới nhen cháy.
- Tiêu thụ thấp cho thiết bị pin: dòng đo chỉ vài mA và có chế độ Sleep, phù hợp chu trình ngủ–thức định kỳ của nút chạy pin AA.
- Tích hợp bộ sưởi nội bộ: bật/tắt bằng phần mềm để loại bỏ ngưng tụ hơi nước trên màng cảm biến trong môi trường ẩm, tránh báo ảo và sai số theo thời gian.
- Để làm rõ ưu thế của giải pháp, bảng dưới đây so sánh các thông số kỹ thuật cốt lõi giữa SHT31 và các dòng cảm biến nhiệt độ - độ ẩm phổ biến khác trên thị trường hiện nay:

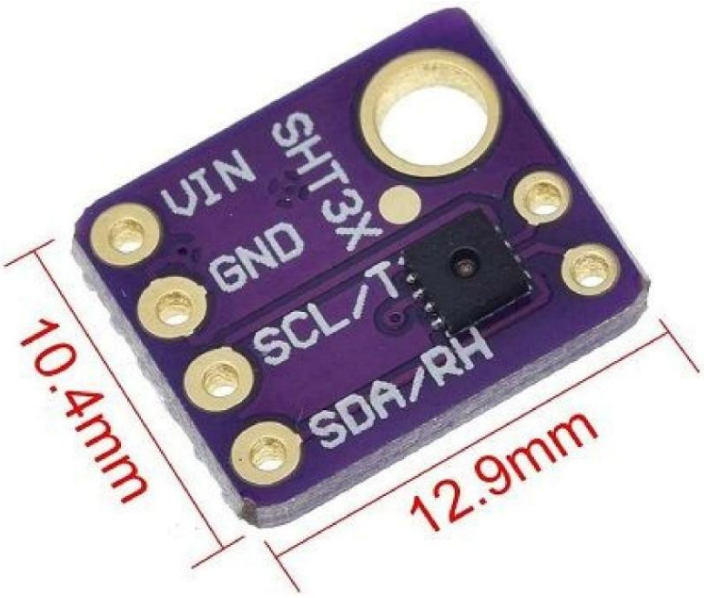
Bảng 2.10 So sánh các cảm biến nhiệt độ

Thông số kỹ thuật	Cảm biến SHT31 (Sensirion)	Cảm biến DHT22 / AM2302	Cảm biến AHT20 (ASAIR)
Giao thức truyền thông	I²C (Hỗ trợ thay đổi địa chỉ cứng)	Single-wire (Chuẩn riêng, chậm)	I²C
Sai số đo nhiệt độ	±0.2°C	± 0.5°C	± 0.3°C
Sai số đo độ ẩm	± 2%RH	± 2% đến 5% RH	± 2% RH
Thời gian đáp ứng (T63%)	8 giây (Nhiệt độ) / 8 giây (Độ ẩm)	15 giây	10 giây
Dòng tiêu thụ ở chế độ chờ	0.2 mA (Cực thấp)	10 đến 15 mA	0.25 mA
Kích thước phần cứng	Siêu nhỏ gọn (DFN-8, SMD)	Lớn (Dạng module có vỏ nhựa)	Nhỏ gọn (SMD)

Việc lựa chọn cảm biến SHT31 là giải pháp tối ưu cho nút mạng biên, đáp ứng hoàn hảo yêu cầu khắt khe về độ chính xác, tốc độ phản hồi thời gian thực khi xảy ra sự cố nhiệt, đồng thời cấu trúc nguồn dòng siêu thấp của chip giúp kéo dài tối đa tuổi thọ vận hành của hệ thống pin.

b, Sơ đồ chân module và kết nối I²C

Cảm biến SHT31 được triển khai thành một module độc lập để đảm bảo tính cơ động và chống nhiễu tốt cho tín hiệu đầu vào.



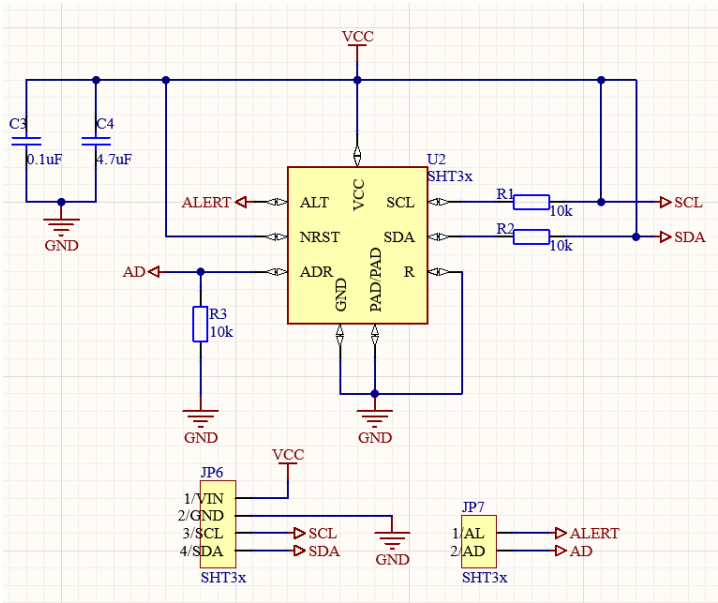
Hình 2.8 Cảm biến SHT31

Module SHT31 có 6 chân kết nối; chức năng từng chân được liệt kê ở bảng dưới.

Bảng 2.11 Các chân cảm biến SHT31

Chân	Chức năng
VCC	Nguồn cấp (2,15V – 5,5V)
GND	Đất
SCL	Xung clock I²C
SDA	Dữ liệu I²C
ALERT	Chân báo động (active HIGH)

Sơ đồ nguyên lý khối SHT31 kết nối với vi điều khiển trung tâm được thể hiện ở hình dưới.



Hình 2.9 Sơ đồ nguyên lý cảm biến SHT31

Dựa trên sơ đồ nguyên lý, khối SHT31 được cấu hình qua các phần sau:

- Mạch giao tiếp I²C (SCL, SDA): hai đường tín hiệu có cặp điện trở kéo lên R1, R2 (10k) lên VCC để ổn định mức logic, kèm điện trở nối tiếp bảo vệ, đưa ra connector JP6 về vi điều khiển.
- Cấu hình địa chỉ (ADR): chân ADR kéo xuống GND qua R3 (10k) để cố định địa chỉ I²C mặc định 0x44, đồng thời đưa ra connector JP7 để kiểm thử/đổi trạng thái khi cần.
- Đường cảnh báo ngắt (ALERT): chân ALT nối ra connector JP7, xuất ngắt phần cứng mức cao (active HIGH) về vi điều khiển ngay khi nhiệt độ/độ ẩm vượt ngưỡng.

Việc tối ưu sơ đồ nguyên lý khối SHT3x -từ lọc nhiễu, mạch kéo lên đến ngắt ngưỡng phần cứng -là cơ sở để nút cảm biến đo chính xác, phản hồi tức thời và tiết kiệm năng lượng trong mạng Mesh Thread.

c, Thông số kỹ thuật

Cảm biến SHT31 có các thông số kỹ thuật nổi bật như sau:

- Dải đo nhiệt độ: từ -40°C đến +125°C, đáp ứng rộng rãi các điều kiện môi trường
- Độ chính xác nhiệt độ: ±0,3°C trong dải 20°C – 80°C và ±0,5°C trong toàn dải đo.
- Dải đo độ ẩm tương đối (RH): 0 – 100% RH.
- Độ chính xác độ ẩm: ±2% RH trong dải 20% – 80% RH, đảm bảo đo lường chính xác trong môi trường sinh hoạt.
- Giao tiếp: I²C, hỗ trợ hai địa chỉ (0x44 và 0x45) nhờ chân ADDR.
- Điện áp hoạt động: 2,15V – 5,5V, tương thích trực tiếp với nguồn 3V của pin AA.
- Tiêu thụ dòng điện: 600 - 1500 µA khi đo, 0,3 µA ở chế độ Sleep, rất phù hợp cho thiết bị chạy pin.
- Độ phân giải: 16 bit cho cả nhiệt độ và độ ẩm, cho phép đọc giá trị với độ chi tiết cao.

Nhìn chung, SHT31 là cảm biến nhiệt độ–độ ẩm lý tưởng cho IoT và hệ thống báo cháy: dải đo rộng, độ chính xác cao, giao tiếp I²C dễ tích hợp, nguồn linh hoạt và đặc biệt tiêu thụ cực thấp (0,3 µA khi ngủ), cho phép nút SED chạy pin lâu dài.

d, Nguyên lý hoạt động

SHT31 dùng cảm biến độ ẩm điện dung và cảm biến nhiệt độ band-gap kết hợp ADC 16 bit; tín hiệu được số hóa và truyền qua I²C. Cảm biến có hai chế độ: Single-Shot (đo một lần) và Periodic (đo định kỳ).

- Chế độ đo Single-Shot (Đo một lần)

Ở chế độ này, MCU gửi lệnh đo (2 byte) rồi đọc kết quả:

- MCU gửi lệnh đo (vd 0x2C 0x06 cho high repeatability kèm clock stretching).

- Cảm biến đo, thời gian tùy độ lặp lại (cao 12,5 ms; trung bình 4,5 ms; thấp 2,5 ms).
- MCU đọc nhiệt độ (2 byte) + CRC và độ ẩm (2 byte) + CRC.
- Nếu lỗi CRC, đọc lại hoặc bỏ qua dữ liệu.

– Chế độ đo Periodic (Đo định kỳ)

Chế độ này cho SHT31 tự đo liên tục với tần số cố định, giúp MCU ngủ và chỉ thức để đọc định kỳ, tiết kiệm năng lượng -dùng cho các nút SED trong đồ án.

Để khởi tạo Periodic, MCU gửi một lệnh duy nhất (vd 0x21 0x30 cho 1 mps high repeatability); sau đó cảm biến tự đo với tần số đã chọn (0,5/1/2/4/10 mps).

Bảng dưới tổng hợp các lệnh chế độ Periodic:

Bảng 2.12 Tập lệnh cảm biến SHT31

Tần số (mps)	Độ lặp lại	Lệnh (Hex)	Ghi chú
0,5	Cao (High)	0x20 0x32	Tiêu thụ dòng thấp nhất
0,5	Trung bình (Medium)	0x20 0x24	
0,5	Thấp (Low)	0x20 0x2F	
1	Cao (High)	0x21 0x30	Lựa chọn trong đồ án
1	Trung bình (Medium)	0x21 0x26	
1	Thấp (Low)	0x21 0x2D	
2	Cao (High)	0x22 0x36	
2	Trung bình (Medium)	0x22 0x20	
2	Thấp (Low)	0x22 0x2B	
4	Cao (High)	0x23 0x34	
4	Trung bình (Medium)	0x23 0x22	
4	Thấp (Low)	0x23 0x29	
10	Cao (High)	0x27 0x37	Tiêu thụ dòng cao nhất
10	Trung bình (Medium)	0x27 0x21	
10	Thấp (Low)	0x27 0x2A	

Bảng trên cung cấp nhiều tùy chọn tần suất đo (0,5–10 mps) và độ lặp lại, cho phép cân bằng giữa tiêu thụ năng lượng, độ chính xác và tốc độ cập nhật -quan trọng khi tối ưu cho nút SED.

e, Chân ALERT và chế độ ngắt ngưỡng

SHT31 có chân ALERT (chân 3, active HIGH) cấu hình kích hoạt khi nhiệt độ/độ ẩm vượt giới hạn trên hoặc dưới: bình thường ALERT ở LOW, khi vượt

ngưỡng SET lên HIGH. Ngưỡng SET và CLEAR lập trình qua lệnh I²C (0x61 0x1D giới hạn trên, 0x61 0x16 giới hạn dưới), mỗi lệnh kèm một word 16 bit.

f, Ứng dụng trong hệ thống báo cháy

Trong đồ án, SHT31 dùng trên cả Router và SED. Router luôn thức, đọc SHT31 mỗi 10 giây để giám sát trung tâm. SED (chạy pin) cấu hình periodic (1 mps) với ngưỡng nhiệt độ cài đặt; khi nhiệt độ vượt ngưỡng, chân ALERT kéo HIGH, đánh thức ESP32-H2 khỏi light-sleep để gửi thông số qua CoAP đến Border Router.

2.4.2 Cảm biến chất lượng không khí CCS811

a, Lý do lựa chọn cảm biến CCS811

CCS811 là cảm biến khí số siêu tiết kiệm năng lượng của ams, tích hợp cảm biến khí ô-xít kim loại (MOX) trên nền công nghệ micro-hotplate để phát hiện các hợp chất hữu cơ bay hơi (TVOC), giám sát chất lượng không khí trong nhà qua nồng độ TVOC và eCO_2, được chọn nhờ các đặc tính sau [9]:

- Tích hợp vi xử lý và thuật toán nội bộ: chip có sẵn MCU 8-bit tự đọc tín hiệu thô, bù nhiệt độ/độ ẩm và xuất trực tiếp eCO_2 (ppm), TVOC (ppb), nên vi điều khiển trung tâm không phải hiệu chuẩn phức tạp.
- Giao tiếp số I²C: truyền dữ liệu qua I²C chuẩn, tiết kiệm số chân kết nối.
- Quản lý năng lượng linh hoạt: nhiều chế độ đo (1/10/60 giây); ở chế độ giãn cách dòng trung bình dưới 3 mA, phù hợp chu trình tiết kiệm năng lượng của nút mạng.
- Kích thước siêu nhỏ (LGA 2,7 × 4,0 × 1,1 mm): thu nhỏ diện tích PCB, dễ lắp đặt ở nhiều vị trí.

Để làm rõ ưu thế của giải pháp, bảng dưới đây so sánh các thông số kỹ thuật cốt lõi giữa CCS811 và một số dòng cảm biến đo chất lượng không khí phổ biến khác trên thị trường hiện nay:

Bảng 2.13 So sánh các cảm biến chất lượng không khí

Thông số kỹ thuật	Cảm biến CCS811 (AMS)	Cảm biến MQ-135	Cảm biến SGP30 (Sensirion)
Giao thức truyền thông	I²C (Xuất dữ liệu số trực tiếp)	Analog (Xuất điện áp thô)	I²C (Xuất dữ liệu số trực tiếp)
Đại lượng đo lường	eCO_2 (400 đến 8192 ppm), TVOC (0 đến 1187 ppb)	CO_2, Khói, Benzen, NH3, Alcohol	eCO_2 (400 đến 60000 ppm), TVOC (0 đến 60000 ppb)
Thuật toán hiệu chuẩn	Tích hợp sẵn bộ xử lý hiệu chuẩn nội bộ	Phải hiệu chuẩn thủ công bằng mạch ngoài và code	Tích hợp thuật toán bù ẩm nâng cao
Dòng điện tiêu thụ	26 mA (khi đo liên tục) / dưới 3 mA (khi đo giãn cách)	Khoảng 150 mA (Rất cao do phải đốt nóng dây mayso)	Khoảng 48 mA
Phù hợp nguồn pin	Có (Rất phù hợp nhờ dòng tiêu thụ thấp và có chế độ nghỉ)	Không (Dòng nuôi dây mayso quá lớn, nhanh cạn pin)	Trung bình

b, Sơ đồ chân (module) và giao tiếp I²C

Hình ảnh thực tế của module CCS811 được thể hiện ở hình 2.10.



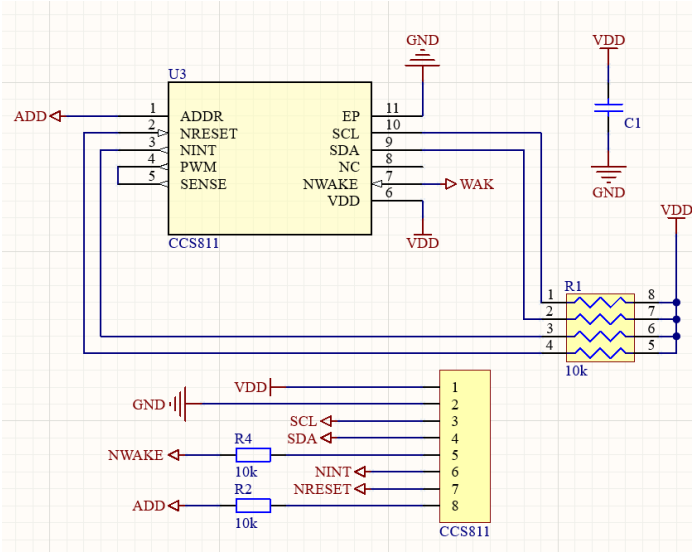
Hình 2.10 Cảm biến CCS811

Module CCS811 có 8 chân kết nối; chức năng từng chân được liệt kê ở bảng dưới.

Bảng 2.14 Sơ đồ chân cảm biến CCS811

Chân	Chức năng
VDD	Nguồn cấp (1,8V – 3,6V)
GND	Đất
SDA	Dữ liệu I²C
SCL	Xung clock I²C
nWAKE	Wake (active LOW)
nINT	Ngắt (active LOW)
nRESET	Reset (active LOW)
ADDR	Chọn địa chỉ I²C

Sơ đồ nguyên lý khối CCS811 kết nối với ESP32-H2 được thể hiện ở hình dưới.



Hình 2.11 Sơ đồ nguyên lý cảm biến CCS811

Dựa trên sơ đồ nguyên lý hình 2.11, khối CCS811 được cấu hình theo các đặc điểm sau:

- Chân I²C (SCL, SDA): kéo lên VDD qua điện trở để giữ mức logic cao mặc định, đảm bảo truyền nhận ổn định.
- Chân chọn địa chỉ (ADDR): qua điện trở kéo -mức thấp cho địa chỉ 0x5A, mức cao cho 0x5B, tránh xung đột địa chỉ trên bus I²C.
  Chân đánh thức (nWAKE): mức thấp kích hoạt chip để giao tiếp I²C, mức cao đưa chip vào ngủ sâu để tiết kiệm năng lượng.
- Chân báo ngắt (NINT): Được kéo lên VDD thông qua mạng điện trở băng.
- Chân reset (nRESET): kéo lên VDD để chip hoạt động bình thường; kéo xuống thấp sẽ buộc CCS811 khởi động lại về mặc định.

Sơ đồ nguyên lý khối CCS811 được thiết kế đầy đủ đường nguồn, linh kiện phân cực và tín hiệu điều khiển số, tạo tiền đề quản lý năng lượng hiệu quả và thu thập dữ liệu chính xác.

c, Thông số kỹ thuật chính

Bảng 2.15 Thông số kỹ thuật cảm biến CCS811

Thông số	Giá trị	Ghi chú
Điện áp hoạt động	1,8V – 3,6V	Khuyến nghị 3,3V
Dòng tiêu thụ (Mode 1)	26 mA (đo), 0,7 mA (trung bình)	1 s/lần
Dòng tiêu thụ (Mode 2)	7 mW (≈2,1 mA/3,3V)	10 s/lần
Dòng tiêu thụ (Mode 3)	1,2 mW (≈0,36 mA/3,3V)	60 s/lần
Dòng tiêu thụ (Sleep)	19 µA	Khi nWAKE ở mức HIGH
Dải đo eCO_2	400 – 8192 ppm	Giá trị ngoài dải bị cắt
Dải đo TVOC	0 – 1187 ppb	
Giao tiếp	I^2C, tốc độ ≤ 400 kHz	Địa chỉ 0x5A / 0x5B
Chân ngắt	nINT (active LOW)	Có thể cấu hình ngắt dữ liệu hoặc ngắt ngưỡng
Nhiệt độ hoạt động	-5°C đến 50°C	
Kích thước	2,7 × 4,0 × 1,1 mm	Gói LGA-10

Nhìn chung, CCS811 tiêu thụ rất thấp ở chế độ ngủ (19 µA) và có chế độ đo linh hoạt, đáp ứng tốt yêu cầu của SED: vừa phát hiện sớm eCO_2/TVOC khi cháy, vừa kéo dài thời gian chạy pin.

d, Nguyên lý hoạt động và cấu trúc bên trong

CCS811 dùng công nghệ micro-hotplate của ams, kết hợp cảm biến khí oxit kim loại (MOX) với MCU tích hợp. Nguyên lý chính:

- Cảm biến MOX đổi điện trở khi gặp VOC; bộ gia nhiệt vi cơ giữ nhiệt độ ổn định để tăng độ nhạy.
- MCU tích hợp quản lý chế độ đo, đọc tín hiệu thô, bù nhiệt độ/độ ẩm (qua ENV_DATA) rồi tính eCO_2 và TVOC theo thuật toán nội bộ.
- eCO_2 không phải eCO_2 thực mà là giá trị tương đương suy từ nồng độ VOC (vì eCO_2 và TVOC thường tỷ lệ thuận khi có người).
- Giao tiếp I^2C được sử dụng để cấu hình cảm biến, ghi dữ liệu bù (nhiệt độ, độ ẩm) và đọc kết quả.

Nhờ tích hợp MCU, CCS811 giảm tải cho vi điều khiển chủ, chỉ cần gửi lệnh đọc và nhận kết quả.

e, Các chế độ đo (Modes of Operation)

CCS811 hỗ trợ 5 chế độ hoạt động được điều khiển qua thanh ghi *MEAS_MODE (0x01):*

Bảng 2.16 Các chế độ đo trên cảm biến CCS811

Chế độ	Tên	Tần suất đo	Công suất (1,8V)	Ứng dụng
0	Idle	Không đo	0,034 mW	Chờ, không thực hiện phép đo
1	Constant power	1 s/lần	46 mW	Phản ứng nhanh nhất, dùng khi có sự cố
2	Pulse heating	10 s/lần	7 mW	Cân bằng – chọn cho SED
3	Low power pulse	60 s/lần	1,2 mW	Tiết kiệm năng lượng nhất

Lưu ý quan trọng:

- Khi hạ chế độ (vd Mode 1 → Mode 3), phải để cảm biến ở Mode 0 (Idle) ít nhất 10 phút trước khi chuyển.
- Khi chuyển từ thấp lên cao hơn (Mode 3 → Mode 1), không cần thời gian chờ.

Trong đồ án, CCS811 trên các SED chạy Mode 2 (10 s/lần) để cân bằng tiết kiệm pin và phát hiện sớm; khi ESP thức dậy có thể tạm chuyển lên Mode 1, lưu ý thời gian chờ khi quay lại.

f, Các thanh ghi quan trọng và giao tiếp I²C

Dưới đây là các thanh ghi I²C cơ bản của CCS811:

Bảng 2.17 Các thanh ghi của CCS811

Địa chỉ	Tên thanh ghi	Kích thước	Mô tả
0x00	STATUS	1 byte	Trạng thái: FW_MODE, APP_VALID, DATA_READY, ERROR
0x01	MEAS_MODE	1 byte	Cấu hình chế độ đo (DRIVE_MODE) và chế độ ngắt (INT_DATARDY, INT_THRESH)
0x02	ALG_RESULT_DATA	8 byte	eCO_2 (2 byte), TVOC (2 byte), STATUS, ERROR_ID, RAW_DATA (2 byte)
0x05	ENV_DATA	4 byte	Ghi nhiệt độ và độ ẩm để bù (mỗi giá trị 2 byte)
0x10	THRESHOLDS	5 byte	Cài ngưỡng cho INT_THRESH: Low-to-Medium (2 byte), Medium-to-High (2 byte), Hysteresis (1 byte)
0x11	BASELINE	2 byte	Đọc/ghi baseline (giá trị nền) để tăng tốc ổn định
0x20	HW_ID	1 byte	Hardware ID, luôn là 0x81
0xE0	ERROR_ID	1 byte	Mã lỗi chi tiết (khi STATUS báo ERROR)

Quy trình đọc dữ liệu cơ bản:

- Kéo chân nWAKE xuống LOW, chờ tAWAKE ≥ 50 µs.
- Ghi địa chỉ thanh ghi ALG_RESULT_DATA (0x02) rồi đọc 8 byte qua I^2C.
- Kiểm tra bit DATA_READY (byte STATUS); nếu =1, hai cặp byte đầu là eCO_2 và TVOC (MSB first).
- Nếu bit ERROR bật, đọc thanh ghi ERROR_ID (0xE0) để biết lỗi cụ thể.
- Kéo nWAKE lên HIGH để tiết kiệm điện.

Nhờ các thanh ghi rõ ràng và giao tiếp I^2C có nWAKE, CCS811 là giải pháp tối ưu để phát hiện sớm nguy cơ cháy qua eCO_2 và TVOC, đặc biệt khi kết hợp với SHT31.

2.4.3 Cảm biến khói RE46C194 + IR LED + Photodiode

a, Lý do lựa chọn cảm biến RE46C194

RE46C194 là vi mạch chuyên dụng (ASIC) của Microchip để quản lý buồng khói quang học với mức tiêu thụ cực thấp [10]. Thay vì dùng module khói cồng kềnh, hệ thống dùng RE46C194 kết hợp LED hồng ngoại (IR LED) và photodiode trong buồng tối, nhờ các ưu điểm sau:

- Nhận diện khói quang học tin cậy: chip tích hợp khuếch đại nhạy, chuyển đổi tín hiệu và điều khiển xung IR LED; khi khói tán xạ ánh sáng vào photodiode, chip khuếch đại rồi kiểm tra ba chu kỳ liên tiếp để xác nhận, loại bỏ báo giả do nhiễu sáng.
- Tối ưu năng lượng: dòng chờ dưới 4 µA; chip cấp nguồn IR LED theo xung rất ngắn (~100 µs) mỗi 10 giây, giảm tối đa tiêu hao -phù hợp nút chạy pin .
- Tự chẩn đoán và kiểm tra buồng khói: có chế độ tự kiểm tra độ nhạy và cảnh báo pin yếu trên phần cứng, tự phát hiện buồng bám bụi hoặc LED suy giảm và báo về vi điều khiển.
- Giao tiếp tầng vật lý tin cậy: chip xuất logic trực tiếp tới chân ngắt của ESP32-H2, kích báo động ngay khi khói vượt ngưỡng mà không cần ADC ngoại vi.

Để làm rõ tính ưu việt của việc tự thiết kế buồng khói dùng RE46C194, bảng dưới đây so sánh giải pháp này với các module cảm biến khói phổ biến khác trên thị trường:

Bảng 2.18 So sánh các cảm biến khói

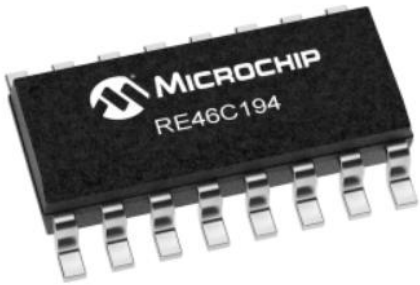
Thông số kỹ thuật	**Giải pháp RE46C194 + Buồng quang học**	**Module cảm biến khói MQ-2**	**Cảm biến khói quang học độc lập (Vỏ bảo vệ sẵn)**
Nguyên lý phát hiện	Quang học (Tán xạ ánh sáng hồng ngoại)	Điện hóa (Thay đổi điện trở bề mặt chất bán dẫn)	Quang học (Tán xạ ánh sáng)
Dạng tín hiệu ngõ ra	Kỹ thuật số (Digital) trực tiếp từ chip	Analog (Điện áp thô) hoặc Digital qua Op-amp	Tiếp điểm khô (Relay) hoặc mức điện áp
Dòng điện tiêu thụ	Dưới 4 mA (Chế độ chờ)	Khoảng 150 mA (Do cần nung nóng đầu đo liên tục)	Từ 15 đến 30 mA
Khả năng điều khiển xung phát	Tích hợp sẵn trong chip (Tự động bật/tắt LED để tiết kiệm pin)	Không hỗ trợ (Luôn luôn đốt nóng dây mayso)	Không thể can thiệp sâu vào cấu trúc điều khiển
Sử dụng nguồn pin	Rất tốt (Kéo dài tuổi thọ pin lên tới hàng năm)	Không khả thi (Dòng quá lớn, làm cạn pin trong vài giờ)	Trung bình (Yêu cầu nguồn nuôi lớn)

Việc sử dụng chip chuyên dụng RE46C194 kết hợp buồng khói quang học tự thiết kế là giải pháp tối ưu hàng đầu cho nút mạng báo cháy, giúp phát hiện khói một cách nhanh chóng, chính xác theo tiêu chuẩn công nghiệp, đồng thời giải quyết

triệt để bài toán khắt khe về dòng tiêu thụ mA để duy trì hệ thống pin dự phòng dài hạn.

b, Sơ đồ chân và giao tiếp với vi điều khiển

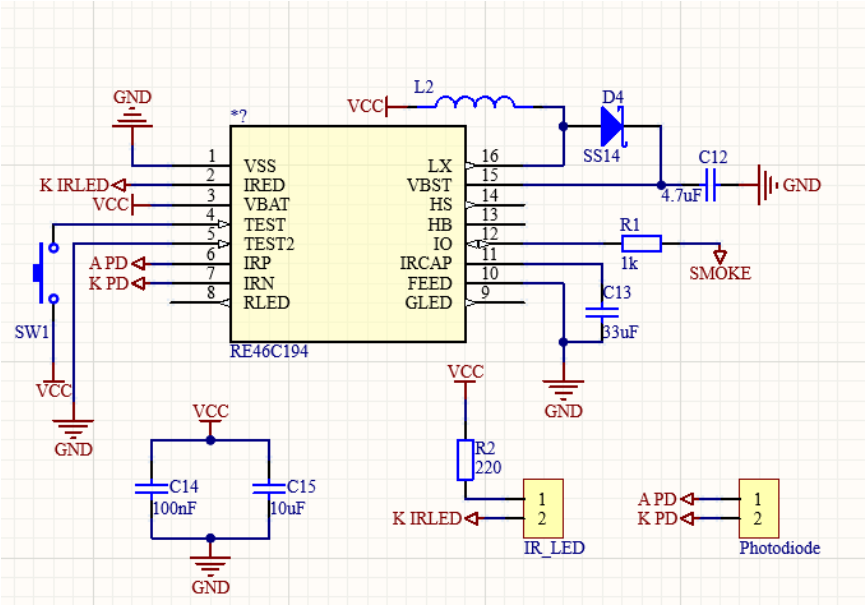
Hình ảnh chân IC RE46C194 (gói SOIC-16) được thể hiện ở hình 2.12.



Hình 2.12 IC cảm biến RE46C194

RE46C194 dùng ngõ ra open-drain qua chân IO, kéo xuống mức thấp (LOW) khi phát hiện khói; kết nối với ESP32-H2 chỉ cần một điện trở kéo lên và một GPIO đầu vào, đơn giản hóa phần cứng.

Sơ đồ nguyên lý khối cảm biến khói (IC, IR LED, photodiode, còi) được thể hiện ở hình dưới.



Hình 2.13 Sơ đồ nguyên lý cảm biến RE46C194

Dựa trên sơ đồ nguyên lý, các khối chức năng và cách giao tiếp với vi điều khiển như sau:

– Đường tín hiệu điều khiển (SMOKE): GPIO số truyền ngắt cứng về vi điều khiển; khi khói vượt ngưỡng, chip kích trạng thái logic để đánh thức vi điều khiển xử lý báo cháy.

- Khối phát hồng ngoại (IRED, IR_DRV): IR_DRV đóng cắt transistor công suất để kích xung cho IR LED chiếu vào buồng quang học, đủ mạnh nhưng tiết kiệm điện.
  Khối thu hồng ngoại (DETECTOR, DET_FEEDBACK): DETECTOR nhận dòng rò nhỏ từ photodiode khi khói tán xạ ánh sáng, kết hợp hồi tiếp DET_FEEDBACK khuếch đại tín hiệu trước bộ so sánh.
- Cấp nguồn và lọc nhiễu (VDD, VSS): tụ gốm lọc nguồn song song triệt nhiễu tần số cao, giữ điện áp tham chiếu ổn định cho các bộ khuếch đại nhạy.
- Mạch dao động và kiểm tra (TEST, OSC): TEST cấu hình kiểm tra trạng thái, kết hợp dao động RC ở OSC để định chu kỳ quét và kiểm tra buồng quang học tự động.

Như vậy, sơ đồ nguyên lý khối RE46C194 được thiết kế đồng bộ từ xử lý tín hiệu quang học, lọc nhiễu nguồn đến ngắt cứng GPIO, đảm bảo phản ứng nhanh với hỏa hoạn và tương thích chế độ ngủ nguồn thấp của vi điều khiển.

c, Thông số kỹ thuật chính

Bảng dưới tóm tắt các thông số kỹ thuật quan trọng của IC RE46C194.

Bảng 2.19 Thông số kỹ thuật cảm biến RE46C194

Thông số	Giá trị	Ghi chú
Điện áp hoạt động	2,0V – 5,0V	Hoạt động với 2 pin AA mắc nối tiếp
Dòng tiêu thụ chờ (Standby)	< 2 µA (điển hình)	Rất thấp, phù hợp cho SED chạy pin
Dòng kích IR LED	50 – 200 mA (lập trình)	Điều chỉnh thông qua điện trở hạn dòng
Điện áp Boost (chế độ chờ)	3,6V	Đủ để duy trì hoạt động nền
Điện áp Boost (chế độ báo động)	8,5V hoặc 10V (tùy cấu hình)	Có thể sử dụng để cấp nguồn cho còi nếu cần; trong ứng dụng này chỉ sử dụng tín hiệu ngắt IO
Tần số kiểm tra khói (Smoke Check)	Mỗi 10s	Định kỳ kích IRED và tích phân tín hiệu
Tần số kiểm tra pin (Low Battery Test)	Mỗi 86s	Có tải (bật RLED) để đo chính xác
Nhiệt độ hoạt động	0°C đến 60°C	Dải nhiệt cho môi trường trong nhà
Giao diện với vi điều khiển	IO (Open-Drain, active LOW)	Chỉ cần 1 GPIO + điện trở kéo lên
Đóng gói	SOIC-16 (16 chân)	Dễ dàng hàn và bố trí trên PCB

Nhìn chung, RE46C194 cung cấp đủ chức năng cho một máy dò khói quang điện hoàn chỉnh: tự điều khiển IR LED và photodiode, xác nhận báo động nhiều tầng, quản lý nguồn boost và xuất tín hiệu ngắt qua chân IO -tất cả trong gói SOIC-16 tiêu thụ cực thấp, chỉ cần một GPIO để giao tiếp, phù hợp nút SED chạy pin.

d, Nguyên lý hoạt động và cấu trúc bên trong

RE46C194 là vi mạch CMOS công suất thấp, tích hợp đủ các khối cho máy dò khói quang điện. Nguyên lý chính:

- Phát hiện khói quang điện: IR LED và photodiode đặt trong buồng tối sao cho ánh sáng không chiếu thẳng vào photodiode; khi có khói, ánh sáng tán xạ vào photodiode với cường độ tỷ lệ nồng độ khói.
- Xử lý tín hiệu tích phân và số hóa: IC kích IRED định kỳ (10 giây), mỗi chu kỳ thực hiện hai tích phân (IRED tắt và bật) để loại dòng rò, ánh sáng nền; kết quả số hóa bằng ADC 6-bit và so với ngưỡng trong EEPROM.
- Xác nhận báo động: yêu cầu ba lần phát hiện liên tiếp trước khi vào trạng thái báo động cục bộ; khi đó tăng tần suất kiểm tra và chuyển sang High Boost để đánh thức vi điều khiển.

Nhờ tích hợp và cơ chế xác nhận nhiều tầng, RE46C194 đảm bảo độ tin cậy cao trong phát hiện khói và giảm tối đa báo giả do nhiễu hoặc bụi tạm thời.

e, Các cơ chế hoạt động và quản lý năng lượng

RE46C194 tối ưu tiêu thụ điện và vận hành qua các chế độ sau:

- Chế độ chờ (Standby): tiết kiệm nhất, dòng dưới 2 μA, boost giữ 3.6V; kiểm tra khói mỗi 10 giây, pin mỗi 86 giây, buồng khói mỗi 43 giây; chân IO ở HIGH nên ESP32-H2 duy trì light-sleep.
- Chế độ báo động (Local Alarm): sau 3 lần phát hiện liên tiếp, chân IO (open-drain) kéo xuống LOW tạo ngắt mức thấp đưa thẳng tới GPIO của ESP32-H2, đánh thức ngay khỏi light-sleep; đèn RLED nhấp nháy báo động.
- Chế độ Hush (tạm dừng cảnh báo): nhấn nút TEST kích bộ đếm 9 phút vô hiệu hóa báo động khi báo giả, nhưng chân IO vẫn LOW để ESP32 vẫn nhận ngắt và xử lý theo phần mềm.
- Chế độ kiểm tra pin (Low Battery Test): mỗi 86 giây có một lần kiểm tra có tải; nếu pin yếu, IC báo bằng tín hiệu khác, không ảnh hưởng chân IO.

Nhờ cơ chế phát hiện, xác nhận ba lần và tạo ngắt phần cứng qua chân IO, RE46C194 cho phép ESP32-H2 ngủ lâu, chỉ thức khi thực sự có khói đủ điều kiện báo động -vừa tiết kiệm năng lượng vừa tin cậy.

f, Cơ chế ngắt và đánh thức vi điều khiển

Khả năng đánh thức vi điều khiển là đặc điểm quan trọng nhất của RE46C194 với nút SED. Cơ chế vận hành:

- Tín hiệu ngắt chủ động: khi xác nhận báo động (sau 3 lần phát hiện), chân IO (open-drain) kéo xuống LOW và duy trì suốt thời gian báo động, tạo ngắt mức thấp kéo dài.

- Đánh thức ESP32-H2: chân IO nối thẳng một GPIO cấu hình wake-up (GPIO_INTR_LOW_LEVEL); khi IO xuống LOW, ESP32-H2 thoát light-sleep ngay, đọc cảm biến và gửi gói CoAP cảnh báo tới Router, không bỏ lỡ sự kiện.
- Kết thúc sự cố: khi khói giảm dưới ngưỡng, RE46C194 thoát báo động và kéo IO về HIGH (qua điện trở kéo lên), ESP32-H2 quay lại giám sát nền và light-sleep.
- Hoạt động độc lập: toàn bộ phát hiện và xác nhận khói do RE46C194 tự thực hiện, không cần vi điều khiển; ngay cả khi ESP32-H2 ngủ, IC vẫn kiểm tra và kéo IO xuống khi có khói, đảm bảo ngắt luôn được gửi tin cậy.

Nhờ ngắt open-drain và khả năng hoạt động độc lập, RE46C194 là giải pháp phát hiện khói an toàn, tin cậy và tiết kiệm năng lượng, phù hợp kiến trúc nút SED trong mạng Thread.

2.5 Kết luận chương

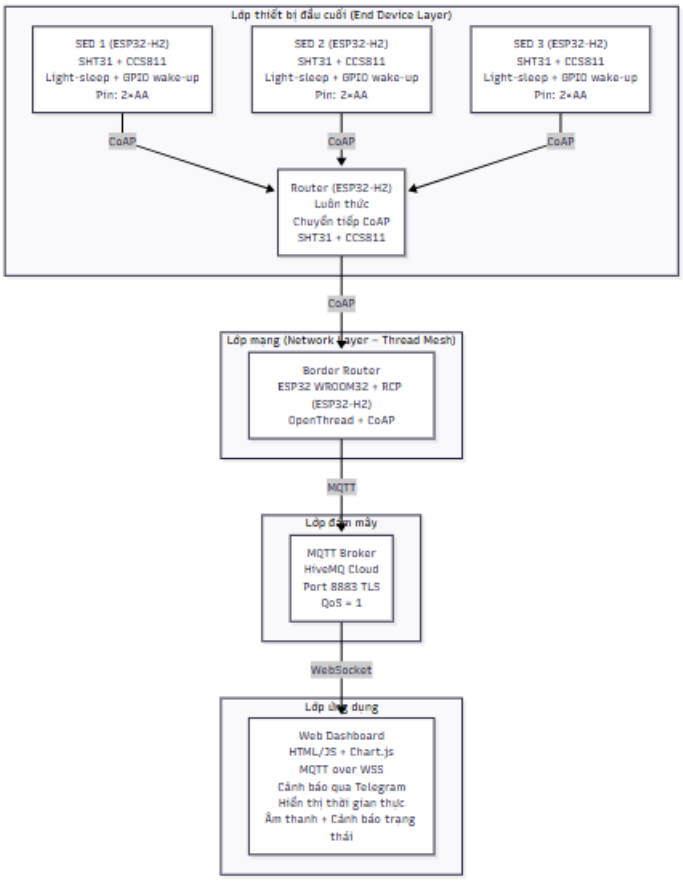
Chương 2 đã trình bày cơ sở lý thuyết và công nghệ nền tảng. Về mạng, Thread được chọn nhờ tự phục hồi, hỗ trợ IPv6 và tối ưu năng lượng; về truyền thông, kết hợp CoAP trong mạng Thread và MQTT từ Border Router lên đám mây là giải pháp tối ưu cho kiến trúc phân tầng.

Về vi điều khiển, ESP32-H2 đảm nhận tốt vai trò Router và SED nhờ tích hợp IEEE 802.15.4 và OpenThread, còn ESP32 WROOM 32 làm Border Router nhờ hiệu năng mạnh và Wi-Fi. Ba cảm biến SHT31, CCS811, RE46C194 đã được phân tích về nguyên lý, thông số và giao tiếp, làm cơ sở cho thiết kế và triển khai ở các chương sau.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Sơ đồ khối tổng thể

Hệ thống báo cháy được xây dựng theo mô hình phân tầng, trong đó các thiết bị cảm biến (SED) kết nối với nhau qua mạng lưới dạng mesh (Thread) và truyền dữ liệu về bộ định tuyến biên (Border Router) để xử lý và đẩy lên đám mây. Sơ đồ khối tổng thể thể hiện rõ các thành phần chính, luồng dữ liệu và vai trò của từng khối trong hệ thống.



Hình 3.1 Sơ đồ khối tổng thể hệ thống

Hình 3.1 minh họa cấu trúc phân tầng hệ thống và sự dịch chuyển của các gói tin qua 3 khối chức năng cốt lõi bám sát mô hình phần cứng của đồ án:

- Lớp thiết bị đầu cuối (End Device Layer): tầng dưới cùng gồm 3 nút ngủ SED chạy pin AA (ESP32-H2); xử lý tại chỗ việc thu thập tín hiệu từ SHT31, CCS811 và chỉ đóng gói gửi đi qua CoAP/UDP khi thức dậy.
- Lớp mạng trung chuyển (Thread Mesh Layer): trục xương sống truyền dữ liệu, gồm nút Router (ESP32-H2) giữ lưới, đóng gói dữ liệu cảm biến của chính nó và thu nhận, đóng gói lại gói tin từ các SED để chuyển tiếp lên Border Router (ESP32 + RCP) -nơi chuyển đổi giao thức sang MQTT.
- Lớp ứng dụng và giám sát (Application Layer): tầng trên cùng lưu trữ và trực quan hóa dữ liệu, gồm MQTT Broker (HiveMQ Cloud) lọc/phân phối gói tin bảo mật qua cổng 8883 và Web Dashboard nhận dữ liệu qua WebSocket để hiển thị biểu đồ thời gian thực và kích hoạt cảnh báo. Ngoài

ra, một bot Telegram trên Railway cũng kết nối MQTT Broker để gửi cảnh báo từ xa khi nhiệt độ vượt ngưỡng.

Sơ đồ khối tổng thể cho thấy rõ tính phân tán, khả năng mở rộng (có thể thêm nhiều SED), độc lập giữa các lớp và khả năng hoạt động bền bỉ trên pin nhờ cơ chế ngủ/thức thông minh.

3.2 Yêu cầu chức năng và phi chức năng

Để đảm bảo hệ thống hoạt động ổn định, đáp ứng mục tiêu phát hiện sớm nguy cơ cháy và có thể triển khai trong thực tế, các yêu cầu về chức năng (what the system does) và phi chức năng (how the system performs) được xác định và phân tích dưới đây.

3.2.1 Yêu cầu chức năng

a, Thu thập dữ liệu cảm biến

- Hệ thống phải đọc được nhiệt độ, độ ẩm từ SHT31, nồng độ eCO_2 và TVOC từ CCS811.
- Dữ liệu phải được cập nhật định kỳ (ví dụ 10 s/lần) hoặc ngay khi có sự kiện vượt ngưỡng.

b, Phát hiện ngưỡng và cảnh báo

- SHT31: Khi nhiệt độ vượt ngưỡng, chân ALERT kích hoạt (HIGH) để đánh thức ESP32-H2.
- CCS811: Khi ESP32-H2 được đánh thức, nó tiến hành lấy dữ liệu từ SHT31 và CCS811 để gửi dữ liệu đi.
- Hệ thống phải tổng hợp cả hai chỉ số để đưa ra quyết định cảnh báo cháy (ví dụ: nhiệt độ cao + eCO_2 cao).

c, Truyền thông trong mạng Thread

- SED phải gửi dữ liệu CoAP (định dạng JSON) lên Router qua unicast (để tránh lỗi checksum).
- Router phải chuyển tiếp dữ liệu từ SED và dữ liệu của chính nó lên Border Router.
- Border Router phải nhận CoAP, xử lý và chuyển tiếp lên MQTT Broker.

d, Kết nối Internet và MQTT

- Border Router (ESP32 WROOM 32) phải kết nối Wi-Fi và đẩy dữ liệu lên MQTT Broker (HiveMQ Cloud) qua TLS (port 8883).
- Web dashboard phải kết nối đến cùng MQTT Broker qua WebSocket để hiển thị dữ liệu thời gian thực.

e, Giao diện giám sát

- Cung cấp giao diện web (HTML/CSS/JS) hiển thị các card thông số của từng nút cảm biến (nhiệt độ, độ ẩm, eCO_2, TVOC, trạng thái khói, thời gian cập nhật).
- Hỗ trợ biểu đồ đường (Chart.js) cho nhiệt độ, độ ẩm, eCO_2, TVOC theo thời gian.
- Phát ra âm thanh cảnh báo (còi) khi phát hiện cháy.

f, Cảnh báo từ xa qua Telegram

- Hệ thống phải tự động gửi tin nhắn cảnh báo qua bot Telegram đến người dùng khi nhiệt độ vượt ngưỡng cài đặt.
- Bot Telegram được triển khai trên nền tảng Railway, đăng ký nhận dữ liệu từ MQTT Broker (HiveMQ Cloud) và kiểm tra ngưỡng theo thời gian thực.

g, Quản lý năng lượng cho SED

- SED phải sử dụng light-sleep với timer 25 giây (duy trì kết nối Thread) và có thể được đánh thức bởi GPIO.
- Khi không có sự kiện, dòng tiêu thụ trung bình phải ≤ 2 mA.
- Hoạt động được trên hai pin AA với thời gian tối thiểu 6 tháng (mục tiêu 1-2 năm).

h, Tự động phục hồi kết nối

- Nếu mất kết nối với Router, SED phải tự động quét lại và tham gia mạng.
- Border Router phải tự động kết nối lại Wi-Fi và MQTT khi mất kết nối.

Các yêu cầu chức năng tập trung vào: thu thập chính xác dữ liệu cảm biến, phát hiện kịp thời ngưỡng nguy hiểm, truyền thông ổn định trong mạng Thread và lên đám mây, cùng giao diện giám sát trực quan -làm nền tảng cho thiết kế phần cứng, logic xử lý sự kiện và quản lý năng lượng SED.

3.2.2 Yêu cầu phi chức năng

a, Độ tin cậy (Reliability)

- Hệ thống phải hoạt động liên tục 24/7, không bị treo hoặc reset ngoài ý muốn.
- Thời gian gián đoạn do lỗi phần mềm hoặc mạng không quá 1% thời gian hoạt động.

b, Độ trễ (Latency)

- Thời gian từ khi cảm biến phát hiện ngưỡng đến khi web dashboard hiển thị cảnh báo không quá 5 giây (trong điều kiện mạng bình thường).
- Thời gian từ khi phát hiện ngưỡng đến khi người dùng nhận được tin nhắn cảnh báo qua bot Telegram không quá 10 giây (phụ thuộc độ trễ của Internet và dịch vụ Telegram).
- Độ trễ trong nội bộ mạng Thread (SED → Border Router) ≤ 2 giây.

c, Bảo mật (Security)

- Mạng Thread sử dụng mã hóa AES-128 (network key) và xác thực thiết bị qua EUI-64.
- Kết nối MQTT sử dụng TLS (HiveMQ Cloud).
- Web dashboard giao tiếp qua WebSocket Secure (WSS) nếu triển khai HTTPS.

d, Khả năng mở rộng (Scalability)

- Hệ thống phải hỗ trợ thêm tối thiểu 10 SED mà không cần thay đổi cấu trúc mạng.
- Border Router có thể xử lý được lưu lượng từ tất cả SED.

e, Tiêu thụ năng lượng (Power consumption)

- Router và Border Router được cấp nguồn liên tục (5V/3.3V), không yêu cầu tiết kiệm năng lượng đặc biệt.
- SED: Dòng tiêu thụ trung bình ≤ 2 mA, tuổi thọ pin ≥ 6 tháng (với pin AA 2000 mAh).

f, Tính tương thích (Compatibility)

- Hệ thống phải hoạt động với các thiết bị Thread tuân thủ chuẩn 1.1 (OpenThread).
- Border Router có thể kết nối với bất kỳ mạng Wi-Fi nào có SSID/password được cấu hình.

g, Khả năng bảo trì (Maintainability)

- Firmware của SED, Router, Border Router có thể được cập nhật qua USB (hoặc OTA nếu có kết nối mạng).
- Code được tổ chức theo module (cảm biến, mạng, ngủ/thức) dễ dàng sửa đổi và kiểm thử.

Các yêu cầu phi chức năng đặt ra chỉ tiêu về độ tin cậy, độ trễ, bảo mật, khả năng mở rộng, tiêu thụ năng lượng và bảo trì, đảm bảo hệ thống vận hành ổn định, bền bỉ và dễ nâng cấp.

3.3 Tính toán công suất tiêu thụ và ước lượng thời lượng pin cho SED

3.3.1 Cơ chế tiết kiệm năng lượng cho SED

Để đạt thời lượng hoạt động lâu dài (mục tiêu 9–12 tháng), SED được thiết kế với cơ chế quản lý năng lượng hai tầng, tận dụng chế độ light-sleep của ESP32-H2 và khả năng đánh thức linh hoạt từ ngoại vi.

SED dùng bộ quản lý nguồn (PMU) với light-sleep: CPU tạm dừng, ngoại vi không cần thiết tắt, chỉ giữ RTC và trạng thái GPIO để đánh thức. Việc chuyển sang light-sleep do tầng quản lý nguồn tự điều phối sau khi các tác vụ FreeRTOS hoàn tất và không còn ngắt chờ.

Ở trạng thái giám sát nền, SED gần như luôn ở light-sleep, đánh thức định kỳ mỗi 25 giây để poll parent (duy trì liên kết với Router cha) và đọc nhanh cảm biến; nếu các chỉ số an toàn, thiết bị lập tức ngủ lại, giữ dòng trung bình rất thấp.

Ngoài ra, SED có nguồn đánh thức phần cứng ưu tiên cao: chân ALERT của SHT31 (GPIO 11) lên mức HIGH khi nhiệt độ vượt ngưỡng SET -dấu hiệu điển hình của đám cháy.

Chân này được cấu hình làm nguồn wake-up, cho phép vi điều khiển thoát ngủ ngay khi có bất thường mà không chờ chu kỳ timer; trong điều kiện bình thường chân ở trạng thái không kích hoạt nên vẫn tiết kiệm năng lượng.

Khi có sự cố (ALERT kích hoạt hoặc tới chu kỳ poll), SED đọc tất cả cảm biến (nhiệt độ, độ ẩm, eCO_2, TVOC), đóng gói JSON và gửi CoAP tới Router cha; đồng thời tăng tốc độ đo SHT31 từ 0,5 lên 2 mps. Nếu bất thường còn tiếp diễn, SED lặp đo và gửi mỗi 3 giây; khi mọi chỉ số về vùng an toàn, tốc độ đo giảm lại và SED quay về light-sleep.

Để tránh báo giả do nhiễu hoặc dao động quanh ngưỡng, phần mềm bổ sung cơ chế latch hai tầng kèm bộ đếm xác nhận (debounce) yêu cầu 3 lần đọc liên tiếp dưới ngưỡng trước khi xác nhận trạng thái an toàn.

Như vậy, kết hợp timer wake-up định kỳ, ngắt ngoài phát hiện sự cố tức thời và cơ chế xác nhận nhiều tầng tạo nên giải pháp quản lý năng lượng toàn diện cho SED -vừa phản ứng nhanh với dấu hiệu cháy, vừa kéo dài thời lượng pin cho các ứng dụng giám sát phân tán.

3.3.2 Tính toán công suất tiêu thụ và ước lượng thời lượng pin

Ước lượng thời lượng pin cho nút SED là yếu tố then chốt đánh giá tính khả thi thực tế. SED chạy trên hai pin AA nối tiếp, nên cần tính chính xác dòng tiêu thụ trung bình dựa trên thông số từng thành phần và cơ chế hoạt động ở mục 3.3.1.

Dòng tiêu thụ của SED gồm ba nhóm: vi điều khiển ESP32-H2, các cảm biến SHT31, CCS811 và các điện trở kéo/nguồn phụ trợ. Số liệu lấy từ datasheet, áp cho chu kỳ điển hình: 25 giây light-sleep, thức ~20 ms để đọc cảm biến và poll parent.

Dòng trung bình từng thành phần được tính như sau:

– ESP32-H2: Theo datasheet ESP32-H2 tại mục 3.10 Low-power Management [6], các chế độ nguồn bao gồm:
  - Active mode: CPU và radio hoạt động, tiêu thụ dòng cao (vài chục mA).
  - Light-sleep mode: CPU tạm dừng, ngoại vi có thể giữ trạng thái, dòng tiêu thụ ~0.2 mA.

Trong chu kỳ, ESP32-H2 ở light-sleep phần lớn thời gian, chỉ thức ~20 ms mỗi 25 giây. Dòng trung bình của ESP32-H2:

$$I_{ESP_{avg}} = 0.2\,\text{mA} \times \frac{24.98\,\text{s}}{25\,\text{s}} + 30\,\text{mA} \times \frac{0.02\,\text{s}}{25\,\text{s}} \approx 0.224\,\text{mA}$$

– SHT31: Theo datasheet SHT31 tại mục 2.1 Electrical Specifications [8], dòng tiêu thụ ở chế độ periodic:
  - Idle state (periodic data acquisition mode): 45 µA (điển hình)
  - Measuring: 600 – 1500 µA
  - Average current (operation with one measurement per second): 1.7 µA

Ở giám sát nền, SHT31 chạy 0.5 mps với dòng trung bình 1.7 µA; khi có sự cố tăng lên 2 mps nhưng thời gian không đáng kể. Giá trị dùng:

$$I_{SHT31} = 0.0017\,\text{mA}$$

– CCS811: Theo datasheet CCS811 tại mục Electrical Characteristics [9]:
  - Dòng khi đo (Mode 1): 26 mA
  - Dòng trung bình Mode 1: 0.7 mA
  - Công suất Mode 2: 7 mW → tại 3.3V, dòng ~2.1 mA
  - Công suất Mode 3: 1.2 mW → tại 3.3V, dòng ~0.36 mA
  - Chế độ Sleep (nWAKE = HIGH): 19 µA

Ở giám sát nền, CCS811 đặt ở Idle (Mode 0), dòng ~10 µA; khi có sự cố chuyển Mode 1 (0.7 mA), nhưng do thời gian sự cố rất nhỏ nên dòng trung bình gần bằng Idle:

$$I_{CCS811_{\text{Mode2}}} = 0.01\,\text{mA}$$

Tổn hao phụ trợ: Bao gồm dòng qua các điện trở kéo lên trên bus I²C và GPIO, ước tính khoảng 0,1 mA: $I_{phụ} = 0.1\,\text{mA}$

Tổng dòng tiêu thụ trung bình của SED ở điều kiện bình thường (giám sát nền):

$$\begin{aligned} I_{avg} &= I_{ESP_{avg}} + I_{SHT31} + I_{CCS811} + I_{phụ} \\ &= 0.224 + 0.0017 + 0.01 \\ &\quad + 0.1 = 0.3357\,\text{mA} \end{aligned}$$

Với pin AA dung lượng 2000 mAh, thời lượng lý thuyết:

$$\begin{aligned} T_{pin} &= \frac{C_{pin}}{I_{avg}} = \frac{2000\,\text{mAh}}{0.3357\,\text{mA}} \approx 5958\,\text{giờ} \\ &\approx 248\,\text{ngày} \approx 8{,}3\,\text{tháng} \end{aligned}$$

Như vậy, với quản lý năng lượng hai tầng (light-sleep của ESP32-H2 và điều khiển cảm biến theo trạng thái), dòng trung bình của SED ~0,34 mA, tương ứng ~8,3 tháng với hai pin AA 2000 mAh -cân bằng hợp lý giữa khả năng phát hiện sự cố và tiết kiệm điện, tiệm cận mục tiêu 9–12 tháng.

3.4 Thiết kế phần cứng

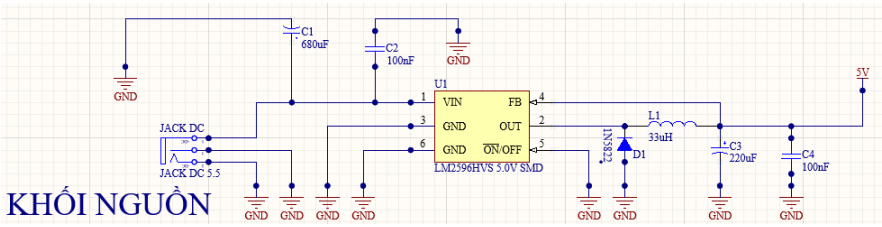
Mạng Thread trong hệ thống được tổ chức theo cấu trúc hình sao – mesh, trong đó Border Router đóng vai trò là cửa ngõ kết nối với mạng Wi-Fi và Internet. Border Router (ESP32 WROOM 32) giao tiếp với RCP (ESP32-H2) qua giao tiếp UART sử dụng giao thức Spinel để điều khiển tầng radio IEEE 802.15.4. Router (ESP32-H2) tham gia mạng Thread với vai trò bộ định tuyến, chịu trách nhiệm chuyển tiếp dữ liệu giữa Border Router và các SED, đồng thời tự đọc cảm biến và gửi dữ liệu lên Border Router qua CoAP. Ba SED (ESP32-H2) hoạt động ở chế độ Sleepy End Device, chỉ kết nối với Router cha, thức dậy định kỳ hoặc khi có ngắt từ cảm biến để gửi dữ liệu qua CoAP.

3.4.1 Sơ đồ mạch Border Router

Sơ đồ nguyên lý Border Router gồm các khối chính: vi điều khiển ESP32 WROOM 32, mạch nguồn 5V/3.3V, kết nối UART với RCP, mạch anten Wi-Fi và các header mở rộng.

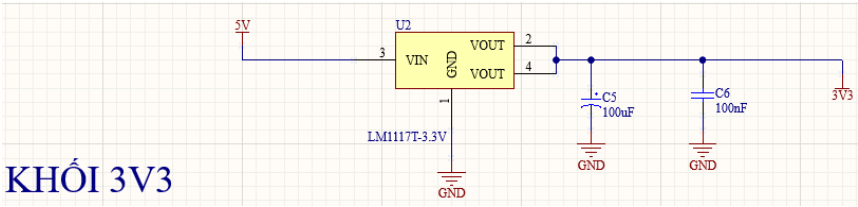
Các khối nguồn, giao tiếp và điều khiển được cấu trúc như sau:

- Hạ áp xung 12V→5V: IC Buck cùng cuộn cảm và diode xung biến nguồn vào thành 5V ổn định, hiệu suất cao, cấp dòng lớn cho còi báo động mà ít tỏa nhiệt.



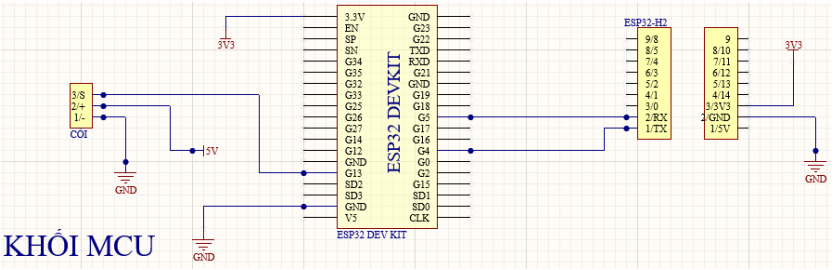
Hình 3.2 Khối nguồn Border Router

– Ổn áp tuyến tính 5V→3V3: IC LDO kèm tụ gốm lọc nhiễu ngõ vào/ra san phẳng điện áp, cấp nguồn sạch cho vi điều khiển.



Hình 3.3 Khối hạ áp 3V3 Border Router

– Giao tiếp UART chéo (TX/RX): ESP32 WROOM 32 và RCP trao đổi dữ liệu qua đường chéo (TX chip này nối RX chip kia), kèm điện trở đệm nối tiếp hạn dòng và bảo vệ GPIO.



Hình 3.4 Khối vi điều khiển Border Router

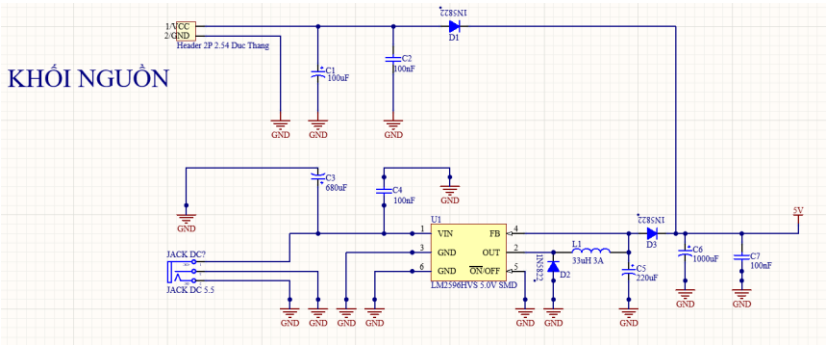
– Mạch đệm transistor điều khiển còi: GPIO của ESP32 qua điện trở kích cực nền transistor (công tắc điện tử) đóng ngắt nguồn 5V cấp cho còi, kèm diode ngược dập xung bảo vệ mạch.

Thiết kế đáp ứng phân phối nguồn ổn định qua hạ áp hai tầng, đường UART chéo an toàn và mạch đệm kích dòng cho còi, tạo nền tảng vận hành tin cậy, chống nhiễu cao.

3.4.2 Sơ đồ mạch cho Router

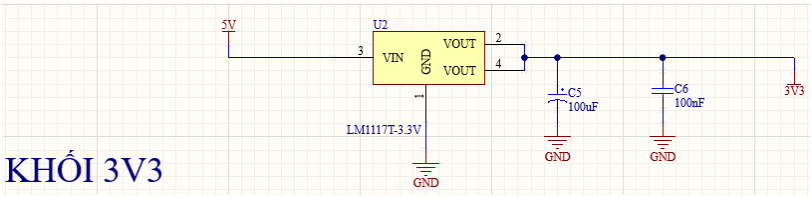
Dựa trên sơ đồ nguyên lý, cấu trúc phân phối nguồn dự phòng thông minh và giải pháp liên kết các khối chức năng được phân tích cụ thể như sau:

– Cấp nguồn linh hoạt hai tầng và nguồn dự phòng: vì Router phải luôn thức để duy trì mesh, mạch ưu tiên nguồn chính từ adapter 12V hạ xuống 5V qua IC Buck; kèm khối pin dự phòng (3 pin AA nối tiếp). Hai diode hướng về đường nguồn chung tự chuyển nguồn theo so lệch áp: có điện thì adapter cấp và khóa diode pin, mất điện thì diode pin dẫn ngay, không gây sụt nguồn.



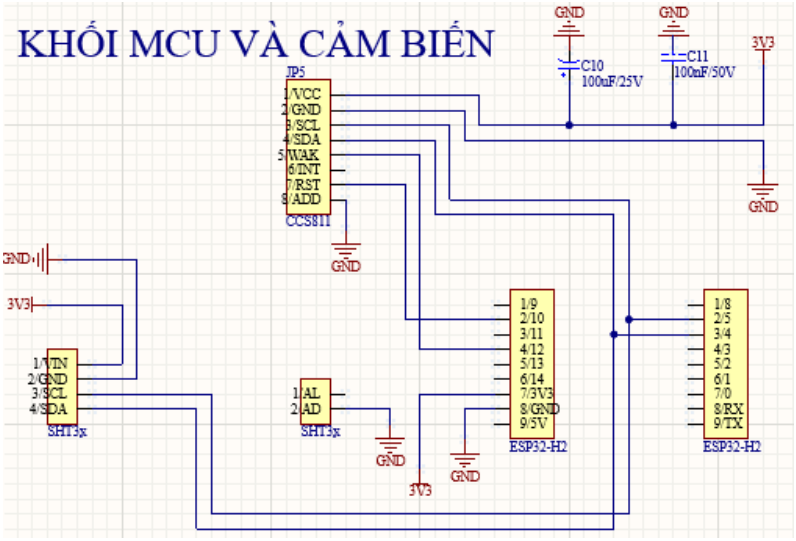
Hình 3.5 Khối nguồn Router

– Ổn áp tuyến tính 5V xuống 3V3: điện áp 5V qua IC LDO hạ xuống 3V3 ổn định, kèm tụ gốm lọc nhiễu ngõ vào/ra để cấp nguồn sạch cho vi điều khiển và cảm biến.



Hình 3.6 Khối hạ áp 3V3 Router

– Giao tiếp khối cảm biến với vi điều khiển: ESP32-H2 kết nối SHT31 và CCS811 qua bus I²C chung (SCL/SDA) có điện trở kéo lên đảm bảo toàn vẹn dữ liệu, đồng thời nối IC quản lý buồng khói qua đường tín hiệu số độc lập để cập nhật trạng thái khói liên tục.



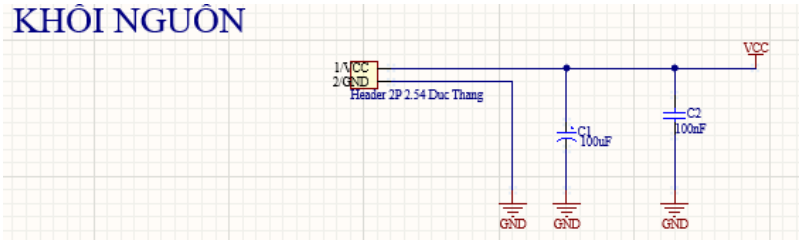
Hình 3.7 Khối MCU và cảm biến cho Router

Thiết kế nguồn của Router giải quyết tốt bài toán cấp điện liên tục nhờ mạch chọn nguồn tự động qua hai diode, đồng thời tối ưu bus I²C và đường tín hiệu khói, tạo nền tảng cho nút Router vận hành ổn định, bền bỉ.

3.4.3 Sơ đồ mạch cho SED

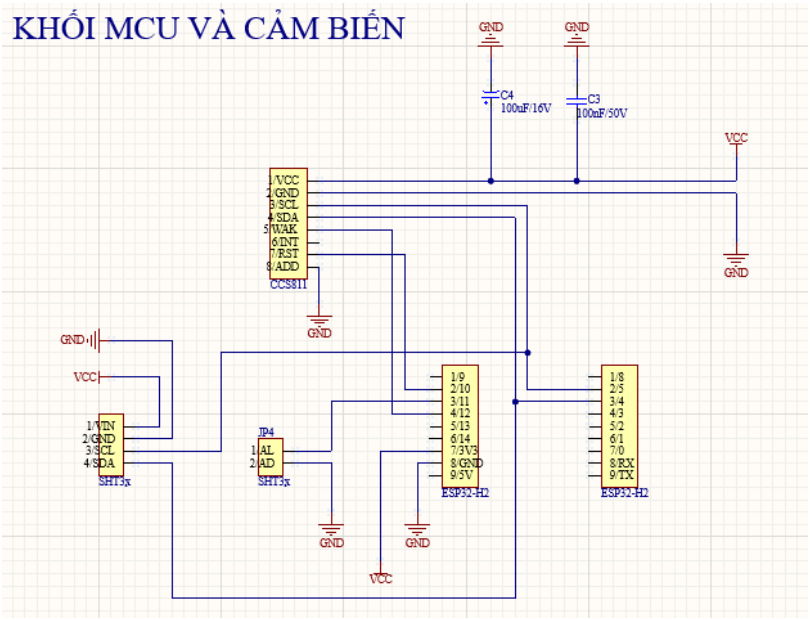
Sơ đồ nguyên lý toàn mạch của nút đầu cuối (SED) thu thập dữ liệu và cảnh báo cháy được thể hiện dưới đây:

Cấp nguồn từ pin: mạch dùng hai pin AA nối tiếp tạo điện áp nền ổn định, tối ưu không gian, đảm bảo cơ động cho nút cảm biến không dây và vận hành lâu dài ở chế độ tiết kiệm năng lượng.



Hình 3.8 Khối nguồn SED

– Giao tiếp cảm biến với vi điều khiển: ESP32-H2 nối cảm biến nhiệt độ, độ ẩm qua bus I²C chung (SCL/SDA) có điện trở kéo lên đảm bảo toàn vẹn dữ liệu.



Hình 3.9 Khối MCU và cảm biến SED

Thiết kế toàn mạch cụ thể hóa giải pháp phần cứng tối ưu cho nút đầu cuối: nguồn dòng thấp bền bỉ, kết hợp giao tiếp số I²C và ngắt cứng GPIO để đảm bảo tính thời gian thực khi có sự cố cháy.

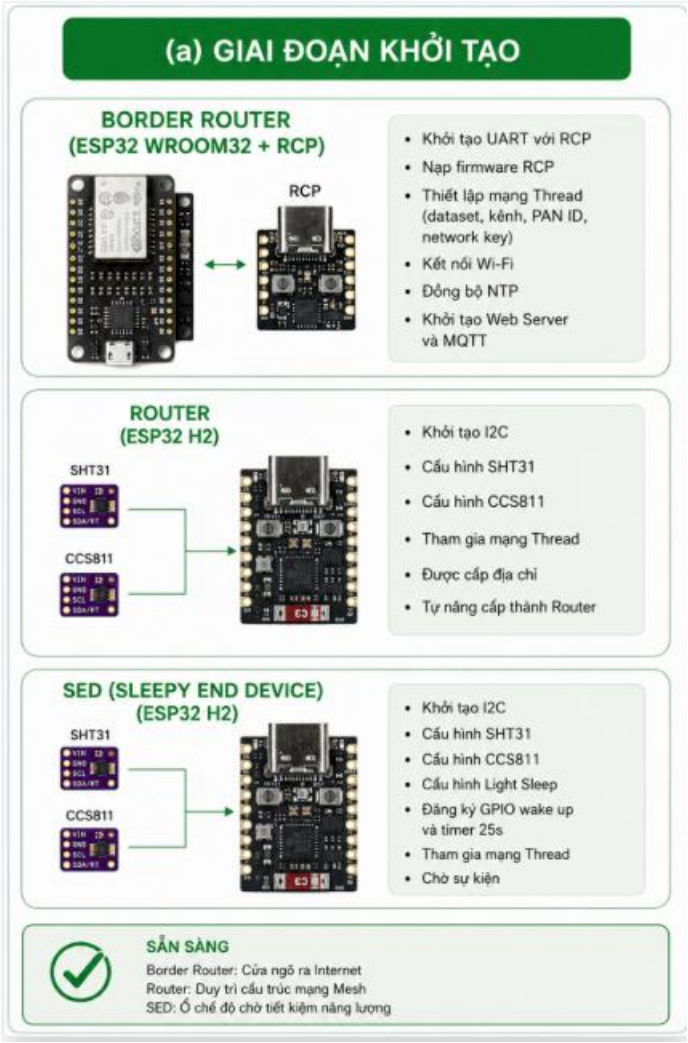
3.5 Thiết kế phần mềm

3.5.1 Lưu đồ hoạt động tổng quát

Hoạt động của toàn bộ hệ thống báo cháy được chia thành ba giai đoạn chính: khởi tạo, giám sát nền (không sự cố) và xử lý sự cố (khi có cháy). Mỗi giai đoạn được mô tả bằng một lưu đồ riêng, thể hiện rõ trách nhiệm của từng thiết bị (Border Router, Router, SED) cũng như luồng dữ liệu giữa chúng.

a, Giai đoạn khởi tạo

Quy trình thiết lập phần cứng và khởi tạo mạng ban đầu được mô tả ở hình 3.10.



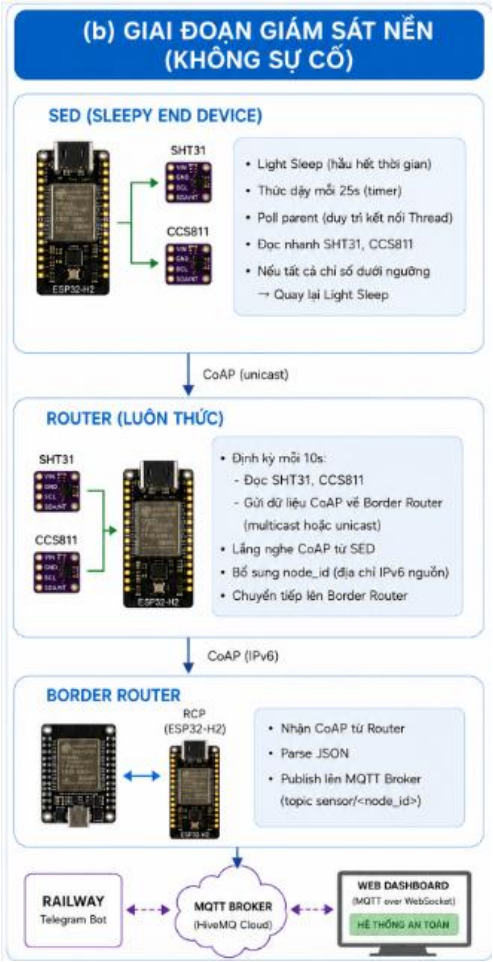
Hình 3.10 Lưu đồ hệ thống giai đoạn khởi tạo

- Border Router (ESP32 WROOM 32 + RCP): khởi tạo UART với RCP, nạp firmware RCP, thiết lập mạng Thread (datasheet, kênh, PAN ID, network key), kết nối Wi-Fi, đồng bộ NTP, khởi tạo web server và MQTT.
- Router (ESP32-H2): khởi tạo I²C, cấu hình SHT31 và CCS811, tham gia mạng Thread, được cấp địa chỉ rồi tự nâng cấp thành Router.
- SED (ESP32-H2): khởi tạo I²C, cấu hình SHT31 và CCS811, cấu hình light-sleep, đăng ký GPIO wake-up (ALERT) và timer 25s, tham gia mạng Thread, chờ sự kiện.

Sau khởi tạo, các thiết bị sẵn sàng: Border Router làm cửa ngõ Internet, Router duy trì mesh, SED ở chế độ chờ tiết kiệm năng lượng.

b, Giai đoạn giám sát nền

Quy trình truyền nhận dữ liệu định kỳ ở điều kiện bình thường được thể hiện ở hình 3.11.



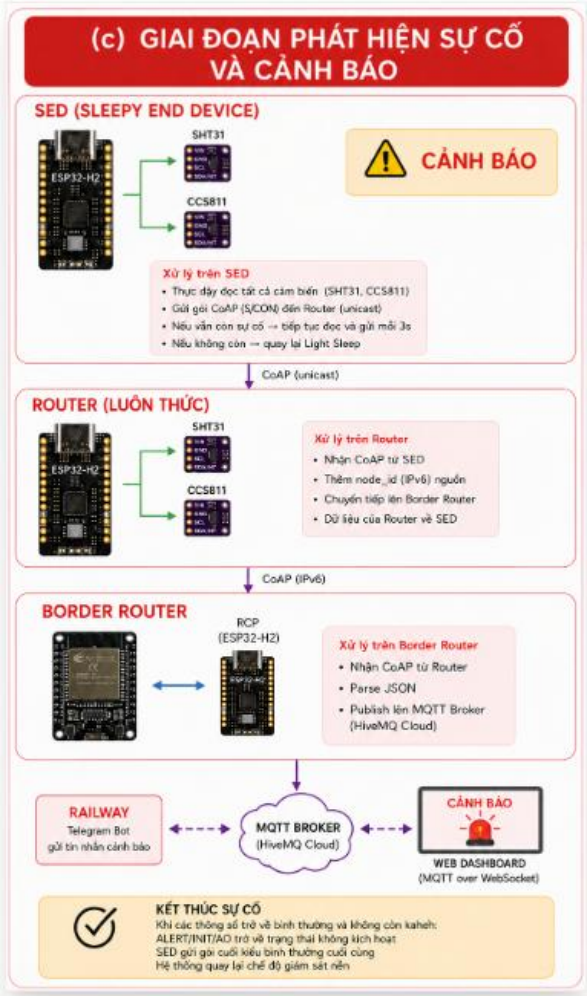
Hình 3.11 Lưu đồ hệ thống giai đoạn giám sát nền

- SED hầu hết thời gian ở light-sleep, chỉ thức theo timer 25s để poll parent và đọc nhanh cảm biến; nếu mọi chỉ số dưới ngưỡng thì ngủ lại.
- Router luôn thức, định kỳ 10s đọc SHT31 và CCS811, gửi CoAP về Border Router; đồng thời nhận CoAP từ SED, bổ sung node_id và chuyển tiếp.
- Border Router nhận CoAP, parse JSON và publish lên MQTT (topic sensor/<node_id>); web dashboard hiển thị trực tuyến ở trạng thái "HỆ THỐNG AN TOÀN"; Railway kết nối HiveMQ sẵn sàng cho cảnh báo.

Hệ thống vận hành ổn định với mức tiêu thụ tối thiểu, luôn sẵn sàng phát hiện bất thường.

c, Giai đoạn phát hiện sự cố và cảnh báo

Tiến trình phản ứng nhanh, chuyển tiếp gói báo động và kịch bản kết thúc sự cố được mô tả ở hình 3.12.



Hình 3.12 Lưu đồ hệ thống giai đoạn phát hiện sự cố

- Kích hoạt: SHT31 (nhiệt độ đạt ngưỡng) kéo ALERT lên HIGH → đánh thức SED khỏi light-sleep.
- Xử lý trên SED:
  - Đọc tất cả cảm biến (SHT31, CCS811).
  - Gửi gói CoAP (JSON: nhiệt độ, độ ẩm, eCO_2, TVOC) tới Router (unicast).
  - Nếu còn sự cố, tiếp tục đọc và gửi mỗi 3 giây; nếu hết, quay lại light-sleep.
- Xử lý trên Router: nhận CoAP từ SED, thêm node_id (từ địa chỉ IPv6 nguồn), chuyển tiếp lên Border Router.
- Xử lý trên Border Router:
  - Nhận CoAP, parse JSON, publish lên MQTT Broker (HiveMQ Cloud).
  - Web dashboard subscribe topic, hiển thị cảnh báo đỏ và phát âm thanh báo động.
  - Dịch vụ bot Telegram (trên Railway) cũng subscribe cùng topic; khi phát hiện vượt ngưỡng, bot lập tức gửi tin nhắn cảnh báo tới người dùng qua Telegram.

- Kết thúc sự cố: khi nhiệt độ về dưới ngưỡng, $eCO_2 < 950$ ppm và hết khói, các chân ALERT/nINT/IO trở về không kích hoạt; ESP gửi gói dữ liệu bình thường cuối cùng và hệ thống quay lại giám sát nền.

Nhờ wake-up bằng GPIO và xử lý ưu tiên, hệ thống phản ứng kịp thời với mọi dấu hiệu cháy, gửi cảnh báo trong thời gian ngắn nhất (dưới 5 giây).

Lưu đồ tổng quát đã làm rõ ba pha: khởi tạo, giám sát nền, phát hiện và xử lý sự cố -giúp dễ kiểm thử từng module, tối ưu năng lượng cho SED và đảm bảo phát hiện cháy sớm tin cậy.

3.5.2 Giao tiếp, cấu hình cho SHT31 và xử lý ngưỡng nhiệt độ

SHT31 trên SED đo nhiệt độ, độ ẩm và đóng vai trò nguồn đánh thức phần cứng nhờ cơ chế ngưỡng nhiệt độ sẵn có.

- Giao tiếp với vi điều khiển qua bus I^2C tiêu chuẩn, dùng chung bus với cảm biến khác.
- Cấu hình hai ngưỡng (Set Point và Clear Point) trong thanh ghi cảnh báo, tạo vùng trễ phần cứng tránh dao động khi nhiệt độ quanh ngưỡng.
- Chân ALERT nối thẳng GPIO đánh thức vi điều khiển khỏi ngủ sâu, chỉ kích hoạt khi nhiệt độ vượt ngưỡng.
- Khi vượt ngưỡng, hệ thống thức dậy, đọc nhiệt độ/độ ẩm và tăng tần suất lấy mẫu để theo dõi sát diễn biến nhiệt.
- Phần mềm bổ sung cơ chế chốt trạng thái và đếm xác nhận để lọc nhiễu, chỉ xác nhận cảnh báo khi nhiều lần đọc liên tiếp vượt ngưỡng.
- Tín hiệu ngưỡng nhiệt độ còn làm điều kiện phụ điều khiển cảm biến khác (vd tạm dừng đo eCO_2 khi nhiệt độ thấp), tối ưu năng lượng.

Nhờ ngưỡng phần cứng kết hợp logic xác nhận và thay đổi tốc độ đo linh hoạt, hệ thống vừa phát hiện sớm sự cố nhiệt vừa tiết kiệm năng lượng cho thiết bị đầu cuối.

3.5.3 Giao tiếp, cấu hình với CCS811

CCS811 giao tiếp với vi điều khiển qua bus I^2C, quản lý qua hai chân điều khiển chính nWAKE và nRESET, kết hợp các chế độ đo linh hoạt để tối ưu năng lượng.

- Chân nWAKE (active LOW): điều khiển thức/ngủ -mức LOW cho phép giao tiếp I^2C, mức HIGH đưa cảm biến vào ngủ dòng thấp (19 µA); được điều khiển bởi GPIO của ESP32-H2 để tối ưu năng lượng.
- Chân nRESET (active LOW): khởi động lại phần cứng khi gặp lỗi (MAX_RESISTANCE, HEATER_FAULT) -kéo LOW ≥ 20 µs rồi thả HIGH, không cần ngắt nguồn tổng.

Nhờ phối hợp các chân điều khiển và chế độ đo linh hoạt, CCS811 vừa phát hiện sớm nguy cơ cháy qua eCO_2 và TVOC, vừa tối ưu năng lượng cho nút SED.

3.5.4 Đóng gói dữ liệu JSON và gửi CoAP

Sau khi thu thập dữ liệu (nhiệt độ, độ ẩm, eCO_2, TVOC), hệ thống đóng gói thành JSON thống nhất và truyền qua mạng Thread bằng CoAP. Dùng unicast từ

SED đến Router cha và JSON có thêm trường node_id giúp tránh lỗi UDP checksum và nhận diện duy nhất mỗi thiết bị tại Border Router.

a, Định dạng JSON cho dữ liệu cảm biến

Dữ liệu cảm biến được đóng gói thành JSON; tùy loại thiết bị, payload có hoặc không có trường node_id:

- SED: gửi CoAP unicast tới Router cha, payload chỉ chứa giá trị đo, không có *node_id: {"temp":29.5,"hum":65.2,"co2":450,"tvoc":12}*
- Router: khi gửi dữ liệu của chính nó, payload: *{"node_id":"Router_H2","temp":28.4,"hum":61.2,"co2":410,"tvoc":5 }*
- Router khi chuyển tiếp từ SED: thêm node_id (4 ký tự cuối địa chỉ IPv6 nguồn) rồi gửi lên Border Router, ví dụ SED1: *{"temp":29.5,"hum":65.2,"co2":450,"tvoc":12,"node_id":"SED_xxxx" }*

b, Cơ chế gửi CoAP unicast từ SED

Dữ liệu SED được gửi unicast trực tiếp tới địa chỉ RLOC của Router cha (từ tiền tố Mesh-Local và RLOC16), không dùng multicast để tránh kích hoạt lớp MPL. Kết hợp MPL với nén 6LoWPAN có thể gây sai biên dịch địa chỉ nguồn, dẫn đến lỗi UDP checksum và hủy gói; unicast trực tiếp loại bỏ nguy cơ này và cho phép chỉ định ML-EID của SED để checksum luôn chính xác.

c, Xử lý và chuyển tiếp gói tin lên Router

Router đóng vai trò CoAP server:

- Parse payload JSON, kiểm tra nếu đã có trường node_id (tránh loopback).
- Lấy địa chỉ IPv6 nguồn từ info->mPeerAddr, trích 4 ký tự hex cuối để tạo node_id dạng SED_xxxx.
- Thêm node_id vào JSON rồi gửi gói CoAP (NON-CONFIRMABLE) đến Border Router qua multicast ff03::1 (hoặc unicast).
- Đồng thời router tự định kỳ gửi dữ liệu của mình qua coap_send() với node_id là "Router_H2".

3.5.5 Xử lý nhận CoAP trên Border Router và chuyển tiếp MQTT

Border Router (BR) là cầu nối giữa mạng Thread (qua RCP) và Internet. Nó nhận các gói CoAP từ router trung gian (đã được bổ sung node_id), phân tích dữ liệu JSON, cập nhật trạng thái các nút cảm biến, sau đó đóng gói lại thành message MQTT và publish lên broker HiveMQ Cloud. Quá trình này đảm bảo dữ liệu từ các thiết bị Thread có thể được giám sát từ xa qua web dashboard.

a, Kiến trúc xử lý CoAP trên Border Router

Border Router dùng ESP32 WROOM 32 kết hợp RCP (ESP32-H2) qua UART: RCP đảm nhận tầng radio IEEE 802.15.4, còn ESP32 WROOM 32 chạy OpenThread và xử lý logic ứng dụng.

- Khởi tạo CoAP server: Border Router cấu hình mạng và đăng ký tài nguyên nhận dữ liệu tại /sensor; khi có gói CoAP đến (từ Router chuyển tiếp hoặc trực tiếp từ SED), tiến trình xử lý tương ứng tự động được kích hoạt.
- Nhận dữ liệu: hệ thống bóc tách chuỗi JSON từ payload và trích địa chỉ IPv6 nguồn; do payload đã có sẵn trường node_id do Router tuyến dưới gắn, Border Router nhận diện ngay nguồn tin mà không cần tự sinh định danh.

Nhờ cơ chế đăng ký resource, Border Router lắng nghe và nhận dữ liệu từ mọi nút trong mạng Thread một cách thụ động, không cần gửi yêu cầu định kỳ.

b, Phân tích và lưu dữ liệu nút

Border Router truyền payload và địa chỉ IPv6 nguồn của thiết bị gửi vào bộ xử lý dữ liệu trung tâm để cập nhật thông tin nút:

- Parse JSON nhận được bằng thư viện cJSON.
- Trích node_id từ JSON; nếu không có thì tự sinh từ địa chỉ IPv6 (dự phòng).
- Tìm hoặc thêm nút vào mảng s_nodes (tối đa 5 nút), bảo vệ bằng mutex.
- Cập nhật nhiệt độ, độ ẩm, eCO_2, TVOC và thời gian last_seen.
- Tạo JSON mới đầy đủ thông tin nút (kèm last_seen) để chuẩn bị publish MQTT.

Lưu trữ dữ liệu nút trong bộ nhớ với mutex giúp Border Router quản lý an toàn nhiều thiết bị và cập nhật thời gian thực, theo dõi thời điểm nhận dữ liệu cuối.

c, Chuyển tiếp dữ liệu lên MQTT Broker

Sau khi cập nhật cấu trúc nút, Border Router thực hiện publish MQTT:

- Kết nối MQTT: Border Router kết nối bảo mật tới HiveMQ Cloud qua TLS (cổng 8883) kèm chứng chỉ xác thực; các sự kiện trong vòng đời kết nối được quản lý để cập nhật cờ trạng thái liên kết cho các module khác đồng bộ.
- Publish dữ liệu: với mỗi gói CoAP nhận được, Border Router trích định danh nguồn để tạo topic dạng sensor/<node_id>, rồi publish với QoS=1 (có bắt tay xác nhận) để đảm bảo dữ liệu không thất lạc.
- Định dạng MQTT payload: JSON chứa node_id, temperature, humidity, co2, tvoc, last_seen.

Publish MQTT với QoS 1 đảm bảo dữ liệu được chuyển lên cloud tin cậy ngay cả khi mạng gián đoạn tạm thời, nhờ khả năng lưu và gửi lại của broker.

d, Đồng bộ thời gian và Web Server

Đồng bộ thời gian (SNTP): sau khi có IP, Border Router tự đồng bộ thời gian qua mạng; khi hoàn tất, nó bật cờ xác thực và gắn chuỗi thời gian thực vào các gói tin, nếu chưa xong sẽ xuất thông báo chưa cập nhật giờ.

Web Server: Border Router chạy máy chủ nhúng với hai cổng -một cổng giao diện đồ họa tự cập nhật mỗi 5 giây và một cổng dịch vụ cung cấp dữ liệu dạng chuỗi cấu trúc; giao diện lưu trong Flash, tách dữ liệu từng nút thành các khối riêng và dùng màu sắc để hiển thị rõ từng chỉ số.

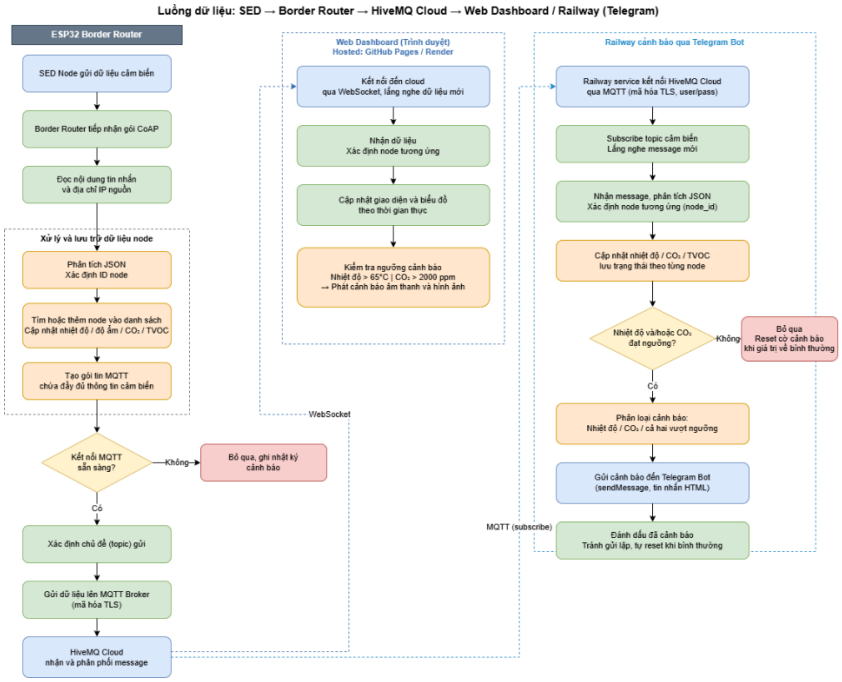
e, Dịch vụ cảnh báo từ xa qua Telegram trên Railway

Hệ thống tích hợp thêm dịch vụ cảnh báo từ xa dạng bot Telegram triển khai thường trực trên Railway, hoạt động như một MQTT client độc lập đăng ký các topic cảm biến từ HiveMQ Cloud và giám sát ngưỡng để chủ động đẩy cảnh báo tới người dùng kể cả khi không mở web.

- Đăng ký topic sensor/# trên HiveMQ Cloud qua MQTT over TLS để nhận dữ liệu thời gian thực từ tất cả các nút cảm biến.
- Khi nhiệt độ hoặc các chỉ số khác vượt ngưỡng cài đặt, bot tự động gửi tin nhắn cảnh báo kèm mã nút và giá trị đo đến người dùng hoặc nhóm Telegram đã đăng ký.

f, Luồng dữ liệu từ Border Router lên web và cảnh báo qua Telegram

Hình 3.13 minh họa luồng dữ liệu từ khi BR nhận CoAP đến khi publish MQTT và hiển thị lên web dashboard và cảnh báo qua Telegram.



Hình 3.13 Lưu đồ luồng dữ liệu từ Border Router

Lưu đồ trên thể hiện rõ ràng từng bước xử lý, từ khâu nhận gói CoAP, phân tích, lưu trữ, đến chuyển tiếp lên cloud và hiển thị trên giao diện người dùng, đảm bảo tính minh bạch và dễ dàng kiểm tra, gỡ lỗi khi cần.

Border Router hoạt động như một cổng giao tiếp hai chiều, vừa đảm nhận vai trò CoAP server để nhận dữ liệu từ mạng Thread, vừa chuyển tiếp lên MQTT cloud và cung cấp web giám sát trực quan. Sự phối hợp nhịp nhàng giữa các thành phần này đảm bảo dữ liệu cảnh báo cháy được truyền tải nhanh chóng, tin cậy và dễ dàng truy cập từ xa.

3.6 Thiết kế giao diện giám sát và cảnh báo

3.6.1 Mục tiêu và các yêu cầu kỹ thuật đối với giao diện hệ thống

Giao diện giám sát là lớp trình diễn (Presentation Layer) của kiến trúc IoT, chuyển các gói tin thô từ hạ tầng mạng thành thông tin trực quan cho người vận

hành. Mục tiêu là một trung tâm điều khiển tập trung, cập nhật thời gian thực với độ trễ tối thiểu các thông số an toàn cháy nổ (nhiệt độ, độ ẩm, eCO_2, TVOC) từ các nút cảm biến Thread.

Về kỹ thuật, giao diện phải đáp ứng chuẩn Responsive Web Design: lưới hiển thị và thành phần đồ họa tự tối ưu tỷ lệ, layout linh hoạt để hiển thị sắc nét trên cả màn hình trung tâm lẫn thiết bị di động.

Đặc biệt, giao diện tích hợp cơ chế phân tích biên để cảnh báo tường minh: ngay khi dữ liệu vượt ngưỡng an toàn, giao diện lập tức đổi trạng thái hiển thị bằng hiệu ứng thị giác mạnh, cô lập vùng nguy hiểm để người quản lý nhận biết tức thời.

3.6.2 Mô hình luồng dữ liệu thời gian thực tại giao diện (Data Flow)

Luồng dữ liệu của giao diện theo kiến trúc hướng sự kiện, liên kết liên tục từ tầng đám mây đến trình duyệt. Trình duyệt khởi tạo kênh hai chiều bất đồng bộ qua WebSocket bảo mật (wss://) tới MQTT Broker (HiveMQ Cloud); mã hóa này đảm bảo an toàn đường truyền và vượt qua tường lửa mạng cục bộ.

Sau khi kết nối, Front-end đăng ký (subscribe) chủ đề sensor/#; khi mạng Thread đẩy gói tin lên, Broker lập tức chuyển chuỗi dữ liệu cấu trúc xuống trình duyệt.

Bộ lọc logic tại giao diện tách gói tin theo node_id (vd Router_H2, Node_ef7:c0c9…) để định tuyến dữ liệu về đúng khối hiển thị; các giá trị được ánh xạ động vào DOM để cập nhật ô chỉ số và nạp vào biểu đồ xu hướng (Chart.js), hoàn tất chu trình với độ trễ dưới một giây.

3.6.3 Giải pháp bố cục và tổ chức không gian hiển thị (Wireframe/Mockup)

Bố cục giao diện tổ chức theo phân cấp thông tin từ vĩ mô đến vi mô, giúp người vận hành nắm toàn cục trong vài giây. Không gian chia thành ba khu vực chính:

- Thanh tiêu đề (Header): trên cùng, định danh hệ thống và tích hợp thẻ trạng thái (Badge) tự cập nhật theo thời gian thực (đang kết nối, thành công, mất liên kết) với Cloud MQTT Broker.
- Khối trạng thái an ninh tổng thể (System Status Block): bảng lớn dưới tiêu đề, tự chuyển giữa "HỆ THỐNG AN TOÀN" (xanh) và "CẢNH BÁO NGUY HIỂM" (đỏ chớp nháy) dựa trên quét ngưỡng liên tục từ các nút.
- Bảng chi tiết các nút (Nodes Grid): lưới thẻ (Cards) cho 4 nút cảm biến, mỗi thẻ gồm ba tầng: trạng thái hoạt động (Online/Offline theo Last Seen), đồ thị đường thu nhỏ thể hiện xu hướng, và bốn ô số liệu Nhiệt độ, Độ ẩm, CO_2, TVOC.

Thiết kế giao diện theo phân cấp thông tin khoa học kết hợp luồng dữ liệu thời gian thực qua WebSocket bảo mật giúp tối ưu khả năng quản lý và phản ứng nhanh -nền tảng để hiện thực hóa mã nguồn và triển khai ở các chương sau.

3.6.4 Tích hợp cảnh báo từ xa qua Telegram

Bên cạnh web dashboard, hệ thống bổ sung kênh cảnh báo từ xa qua Telegram để người dùng luôn nhận thông báo kịp thời kể cả khi không theo dõi web. Một bot Telegram triển khai thường trực trên Railway, đóng vai trò MQTT client độc lập kết nối HiveMQ Cloud. Cơ chế gồm các bước:

- Kết nối và đăng ký: bot duy trì kết nối bảo mật (MQTT over TLS) tới HiveMQ Cloud và đăng ký topic sensor/# để nhận dữ liệu thời gian thực từ mọi nút.
- Kiểm tra ngưỡng: mỗi gói dữ liệu được so với ngưỡng (nhiệt độ, eCO_2, TVOC, trạng thái khói); khi vượt ngưỡng, bot khởi tạo sự kiện cảnh báo.
- Gửi thông báo: bot soạn và gửi tin nhắn cảnh báo kèm mã nút, loại chỉ số, giá trị đo và thời điểm tới người dùng hoặc nhóm Telegram.
- Triển khai liên tục: chạy trên Railway 24/7, độc lập với máy tính cá nhân và tự khởi động lại khi gặp sự cố.

Kênh Telegram mở rộng khả năng giám sát ra ngoài mạng nội bộ, tăng tính sẵn sàng của cảnh báo và phù hợp xu hướng giám sát di động trong hệ sinh thái nhà thông minh.

3.7 Kết luận chương

Chương 3 đã phân tích và thiết kế toàn diện hệ thống báo cháy dựa trên mạng Thread: sơ đồ khối ba lớp (thiết bị đầu cuối SED; mạng Router và Border Router; ứng dụng MQTT Broker và Web Dashboard) cùng các yêu cầu chức năng, phi chức năng (thu thập dữ liệu, phát hiện ngưỡng và cảnh báo, truyền thông Thread, kết nối MQTT, quản lý năng lượng SED).

Phần tính toán công suất và thời lượng pin cho SED dựa trên light-sleep kết hợp timer và GPIO wake-up, đảm bảo chạy bền trên pin AA. Phần cứng đã hoàn chỉnh sơ đồ nguyên lý cho Border Router, Router và SED (khối nguồn, vi điều khiển, giao tiếp cảm biến); phần mềm đã xây dựng các lưu đồ, cơ chế xử lý dữ liệu SHT31/CCS811, đóng gói JSON và truyền CoAP.

Giao diện giám sát được thiết kế trực quan, cập nhật thời gian thực và cảnh báo khi có sự cố; hệ thống cũng bổ sung kênh cảnh báo từ xa qua bot Telegram trên Railway để người dùng nhận thông báo kịp thời kể cả khi không theo dõi web.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1 Môi trường phát triển

4.1.1 Cài đặt ESP-IDF

ESP-IDF (Espressif IoT Development Framework) là nền tảng phát triển chính thức cho ESP32, cung cấp thư viện ngoại vi, toolchain, FreeRTOS và công cụ build/flash/monitor. Đồ án dùng ESP-IDF v5.5.4, cài trên Windows.

a, Yêu cầu hệ thống

- Máy tính chạy Windows (7/10/11).
- Kết nối internet ổn định để tải bộ cài và thư viện.
- Ổ cứng trống tối thiểu 5GB cho framework và toolchain.

Đảm bảo các yêu cầu này giúp cài đặt và vận hành ESP-IDF suôn sẻ.

b, Tải và giải nén ESP-IDF v5.5.4

- Truy cập trang chủ Espressif (tìm "ESP-IDF" trên Google).
- Chọn bản Offline Installer (v5.5.4) để tải về, đảm bảo ổn định khi cài.
- Chạy tệp .exe đã tải để giải nén và thiết lập môi trường.

Bộ cài offline giúp tránh sự cố mạng và tiết kiệm thời gian tải lại.

c, Cài đặt toolchain và công cụ

- Trình cài tự thiết lập các công cụ: Python, Git và compiler cho ESP32.
- Sau cài đặt xuất hiện hai phím tắt: ESP-IDF PowerShell và ESP-IDF CMD.
- Các công cụ này cho phép tương tác với chip qua dòng lệnh (kiểm tra phiên bản, quản lý thư viện).

Nhờ PowerShell/CMD chuyên dụng, có thể chạy idf.py build/flash/monitor mà không cần thiết lập biến môi trường thủ công.

d, Tích hợp ESP-IDF vào Visual Studio Code

- Tải và cài Visual Studio Code từ trang chủ.
- Trong VS Code, vào Extensions tìm "ESP-IDF Extension" để cài.
- Tích hợp này giúp lập trình, biên dịch và nạp code trực quan hơn so với chỉ dùng dòng lệnh.

Extension ESP-IDF cung cấp giao diện đồ họa, tự nhận diện dự án, gợi ý code và tích hợp lệnh build/flash/monitor.

e, Cấu hình Extension và đường dẫn

- Sau khi cài Extension, cấu hình để VS Code nhận đúng thư mục ESP-IDF và tools đã cài ở bước (c).
- Thiết lập tệp cấu hình (settings.json) chỉ định đường dẫn môi trường để VS Code gọi đúng lệnh build/flash.

Cấu hình đúng đường dẫn giúp VS Code kết nối toolchain và môi trường Python ảo của ESP-IDF.

f, Kiểm tra kết nối và cổng COM

- Kết nối board ESP32 với máy tính qua USB.
- Kiểm tra cổng COM thiết bị nhận (vd COM3) qua Device Manager.

- Chạy idf.py monitor -p [COM] trong VS Code để kiểm tra phản hồi từ ESP32.

Nhận được log từ ESP32 khẳng định cài đặt và cấu hình ESP-IDF hoàn tất, sẵn sàng phát triển.

4.1.2 Cài đặt các thư viện phụ thuộc

Sau khi thiết lập môi trường, thêm các thư viện cần thiết: I²Cdev (I²C đa luồng), ccs811 và sht3x (driver cảm biến), cJSON (xử lý JSON) và mqtt (kết nối broker). ESP-IDF quản lý qua IDF Component Manager (tệp idf_component.yml hoặc lệnh idf.py add-dependency).

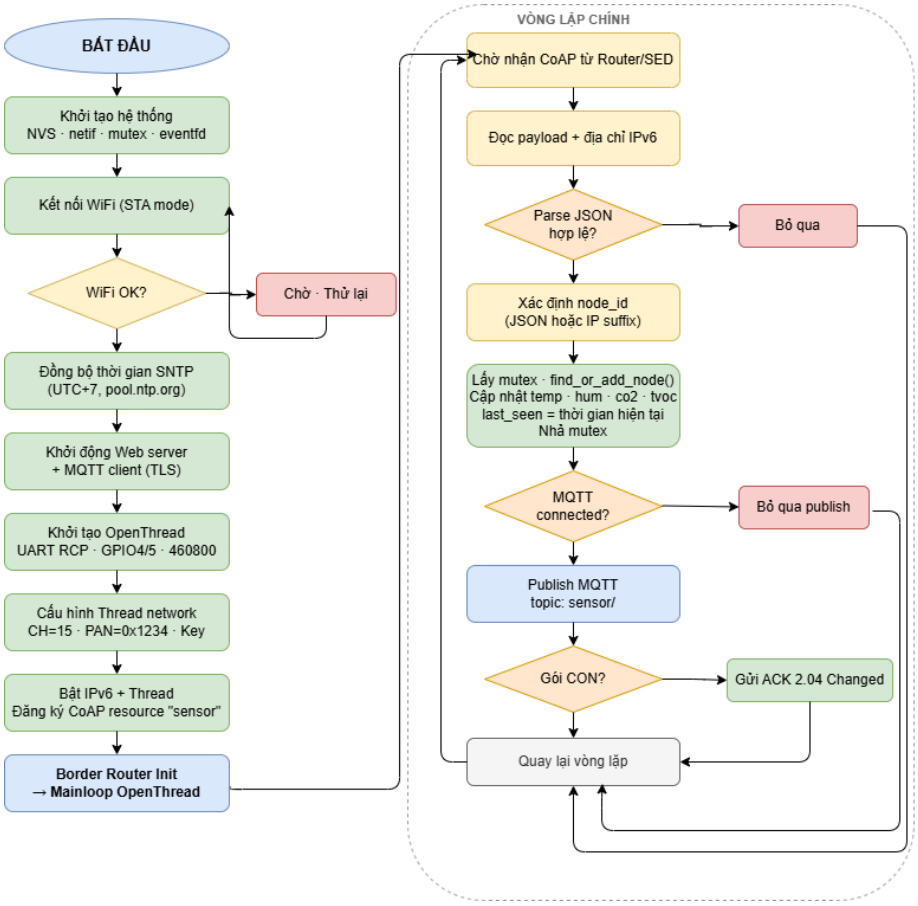
- I²Cdev – giao tiếp I²C an toàn đa luồng (có mutex), tương thích driver I²C mới (ESP-IDF ≥ 5.0): idf.py add-dependency "esp-idf-lib/I²Cdev^2.1.1".
- ccs811 – driver cảm biến CCS811 (đọc eCO_2, TVOC, cấu hình chế độ đo, ngắt ngưỡng), từ esp-idf-lib: idf.py add-dependency "esp-idf-lib/ccs811^1.0.7".
- sht3x – driver SHT3x (SHT30/31/35), hỗ trợ single-shot và periodic cho cấu hình ALERT: idf.py add-dependency "esp-idf-lib/sht3x^1.0.8".
- cJSON – tạo và phân tích JSON để đóng gói dữ liệu cảm biến trước khi gửi CoAP/MQTT: idf.py add-dependency "espressif/cjson^1.7.19".
- MQTT client – component chính thức của Espressif (MQTT over TCP/TLS/WebSocket), cho Border Router publish lên HiveMQ Cloud: idf.py add-dependency "espressif/mqtt^1.0.0".

Sau khi thêm dependency, quá trình build tự tải và tích hợp, đảm bảo đủ driver và thư viện cho firmware.

4.2 Lưu đồ thuật toán cho các thiết bị trong hệ thống

4.2.1 Lưu đồ thuật toán Border Router

Border Router (ESP32 WROOM 32) là cửa ngõ duy nhất nối mạng Thread với Internet, thực hiện ba nhiệm vụ: điều khiển RCP qua UART (tầng radio), vận hành web server hiển thị dữ liệu và kết nối MQTT lên HiveMQ Cloud. Lưu đồ thuật toán ở hình 4.1.



Hình 4.1 Lưu đồ thuật toán ESP32 WROOM 32 làm Border Router

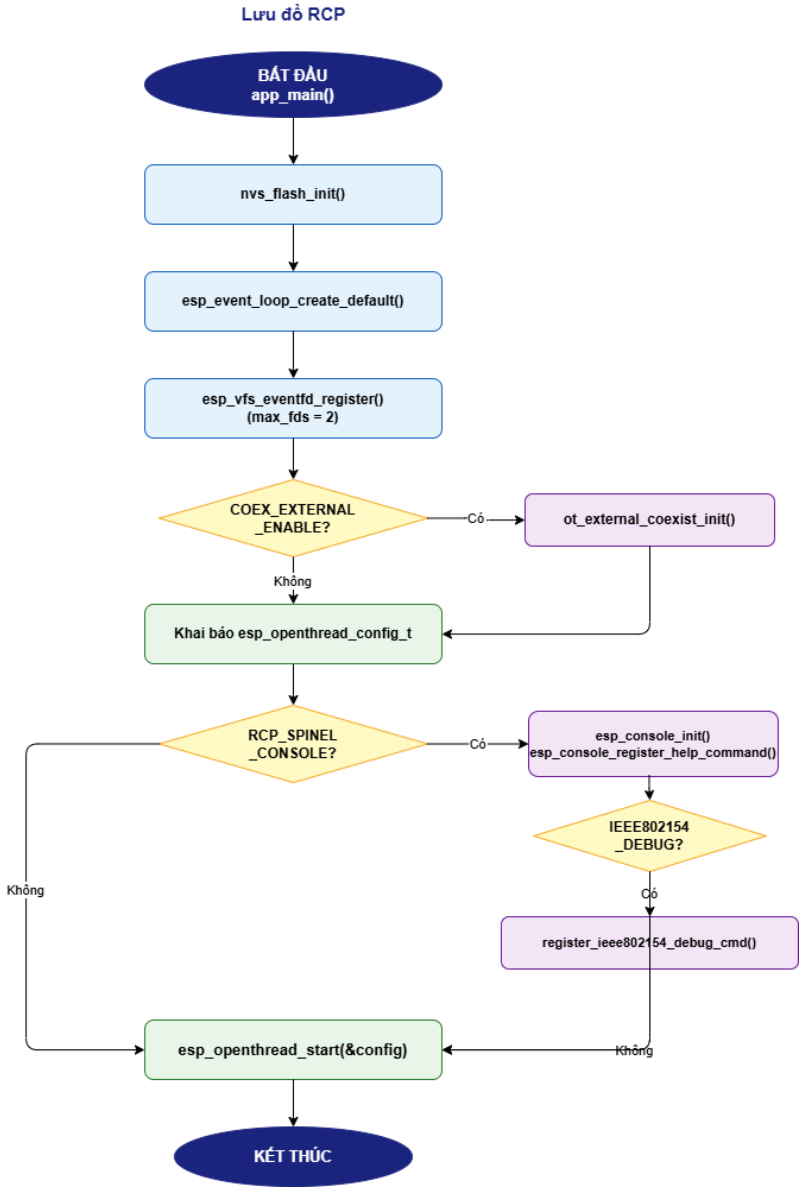
Lưu đồ mô tả hoạt động đồng bộ từ khởi tạo, giám sát nền đến phát hiện sự cố:

- Khởi tạo: cấu hình các thành phần lõi (NVS, netif, mutex, eventfd), kết nối Wi-Fi (STA), đồng bộ SNTP, khởi động Web server và MQTT client (TLS), thiết lập OpenThread (UART RCP qua GPIO4/5, CH=15, PAN=0x1234) trước khi vào vòng lặp chính.
- Xử lý và đồng bộ dữ liệu tại biên: trong vòng lặp, liên tục chờ CoAP từ Router/SED, trích thông tin (IPv6, JSON), định danh node_id và dùng mutex để cập nhật an toàn thông số cảm biến.
- Chuyển tiếp lên Cloud và phản hồi Thread: nếu MQTT ổn định, publish lên topic sensor/; nếu nhận gói CON từ mạng Thread, gửi lại ACK 2.04 Changed để hoàn tất chu kỳ.

Sự liên kết ba giai đoạn cho thấy hệ thống cân bằng tốt giữa tiết kiệm năng lượng khi giám sát nền và đảm bảo thời gian thực, độ chính xác khi có sự cố.

4.2.2 Lưu đồ thuật toán RCP

RCP (Radio Co-Processor) là firmware chạy trên ESP32-H2, làm bộ xử lý radio chuyên dụng cho Border Router: chỉ cung cấp tầng PHY/MAC của IEEE 802.15.4, giao tiếp với ESP32 WROOM 32 qua UART (Spinel), không xử lý logic ứng dụng. Lưu đồ ở hình 4.2.



Hình 4.2 Lưu đồ thuật toán ESP32-H2 chạy RCP

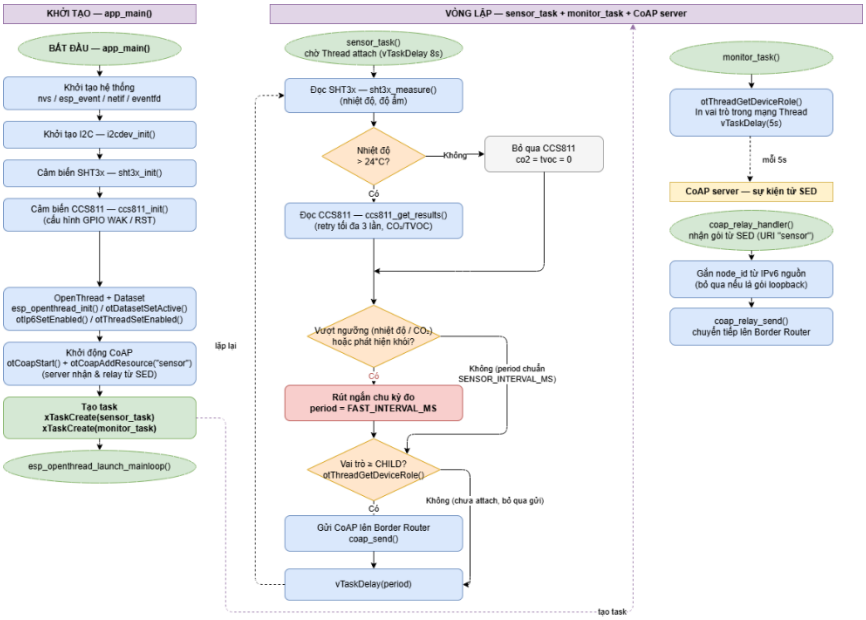
Lưu đồ khởi tạo và kích hoạt OpenThread trên RCP:

- Khởi tạo cơ bản: tại app_main() lần lượt khởi tạo NVS (nvs_flash_init), tạo vòng lặp sự kiện (esp_event_loop_create_default) và đăng ký eventfd (esp_vfs_eventfd_register).
- Cấu hình mở rộng: kiểm tra coexistence ngoại vi (COEX_EXTERNAL_ENABLE) để cấu hình driver, rồi khai báo cấu trúc esp_openthread_config_t.
- Console và kích hoạt OpenThread: kiểm tra bật console (RCP_SPINEL_CONSOLE), đăng ký lệnh debug IEEE 802.15.4 nếu cần, rồi gọi esp_openthread_start(&config) để khởi chạy mạng Thread.

Lưu đồ cụ thể hóa việc khởi tạo radio và kích hoạt OpenThread, đảm bảo luồng tin từ các nút biên được đồng bộ lên đám mây và giao diện với độ trễ thấp.

4.2.3 Lưu đồ thuật toán Router

Router (ESP32-H2) là bộ định tuyến trung gian, chuyển tiếp dữ liệu giữa SED và Border Router, đồng thời tự đọc cảm biến và gửi dữ liệu của mình. Lưu đồ ở hình 4.3.



Hình 4.3 Lưu đồ thuật toán Router

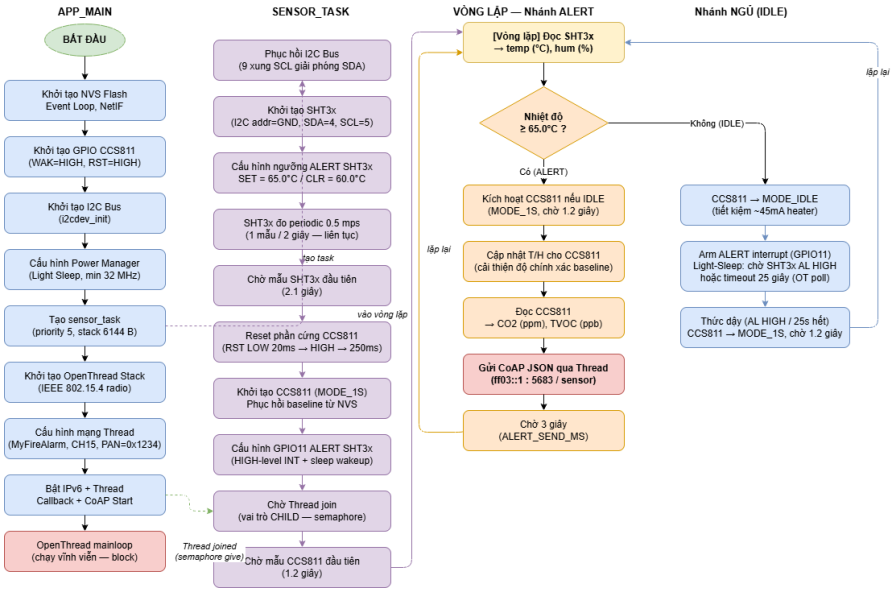
Lưu đồ xử lý luồng dữ liệu của Router:

- Khởi tạo (INIT_BLOCK): thiết lập NVS và Event loop, cấu hình ngoại vi cho SHT31, CCS811, kích hoạt OpenThread và khởi tạo các task chạy song song.
- Thu thập và phân luồng (sensor_task): SHT31 quét nền liên tục, làm điều kiện kích hoạt CCS811 (đọc eCO_2/TVOC khi nhiệt độ vượt ngưỡng), rồi đóng gói CoAP gửi lên Border Router.
- Giám sát mạng và xử lý sự kiện (monitor_task & CoAP handler): kiểm tra định kỳ vai trò thiết bị (Disabled, Detached, Child, Router, Leader); CoAP handler chèn node_id theo IPv6 nguồn và chuyển tiếp dữ liệu toàn vẹn.

Lưu đồ làm rõ quản trị đa tác vụ song song, tối ưu kích hoạt cảm biến để tiết kiệm năng lượng và đảm bảo luồng tin Thread thông suốt.

4.2.4 Lưu đồ thuật toán SED

Lưu đồ dưới đây chi tiết chu trình khởi tạo, thu thập dữ liệu và quản lý kết nối mạng Thread của nút SED:



Hình 4.4 Lưu đồ thuật toán SED

Luồng xử lý đa tác vụ của SED gồm các giai đoạn:

- Khởi tạo phần cứng và mạng: thiết lập tài nguyên nền, cấu hình I²C/GPIO cho cảm biến và kích hoạt OpenThread để chuẩn bị gia nhập mạng Mesh.
- Chu kỳ quét cảm biến thông minh dựa trên điều kiện môi trường: tác vụ sensor_task được tối ưu hóa năng lượng.
- Định tuyến và giám sát kết nối: monitor_task cập nhật vai trò thiết bị (Disabled, Detached, Child, Router, Leader); CoAP handler trích IPv6 nguồn, chèn node_id và chuyển tiếp dữ liệu lên Border Router.

Lưu đồ cụ thể hóa lập trình đa tác vụ song song, tối ưu kích hoạt cảm biến để giảm dòng tiêu thụ và đảm bảo tính đồng bộ, toàn vẹn dữ liệu.

4.3 Thiết lập mạng Thread

Sau khi thiết lập thư viện, bước tiếp theo là thiết lập mạng Thread. Mọi thiết bị trong cùng mạng phải dùng chung datasheet (kênh, PAN ID, tên mạng, network key); việc đồng bộ datasheet diễn ra tự động qua giao thức MLE khi Border Router khởi tạo mạng.

4.3.1 Đồng bộ datasheet

Border Router là Leader, thiết lập datasheet cho toàn mạng (định nghĩa trong code và ghi vào RCP). Khi Router, SED khởi động, chúng quét tìm mạng, nhận datasheet từ Border Router và tự tham gia.

a, Cấu hình datasheet trong Border Router

Các thông số mạng khai báo dạng hằng số trong code Border Router:

- Kênh hoạt động: #define THREAD_CHANNEL 15
- PAN ID: #define THREAD_PAN_ID 0x1234
- Tên mạng: #define THREAD_NETWORK_NAME “MyFireAlarm”
- Khóa bảo mật: mảng THREAD_KEY gồm 16 byte (ví dụ các giá trị hex từ 0x00 đến 0xff).

Khi khởi động, hàm ot_task() tạo biến otOperationalDatasheet, gán giá trị rồi gọi otDatasheetSetActive() để kích hoạt; tiếp theo otThreadSetEnabled(true) khởi động mạng Thread.

b, Đồng bộ datasheet cho các thiết bị khác

- Router và SED khai báo datasheet tương tự (cùng kênh, PAN ID, tên mạng, key); khi khởi động chúng quét kênh, tìm beacon từ Border Router và tự đồng bộ qua MLE, không cần can thiệp thủ công.
- Ban đầu khi chỉ có Border Router và Router, Router là Child rồi nâng cấp lên Router khi có SED tham gia.
- SED sau khi đồng bộ datasheet ở trạng thái Child và có thể gửi/nhận dữ liệu CoAP.

4.3.2 Ghép nối SED và Router

Để ghép nối SED với Router, cả hai cùng cấu hình mạng; SED tự tìm Router làm Parent và gửi dữ liệu qua CoAP, Router nhận và chuyển tiếp lên Border Router.

- Cùng cấu hình Thread: Router và SED dùng chung Network Name ("MyFireAlarm"), Channel 15, PAN ID 0x1234 và Network Key → tự kết nối; Router là Leader/Router, SED là Sleepy End Device tìm Router làm Parent.
- SED gửi CoAP đến Router: lấy địa chỉ Parent qua otThreadGetParentInfo(), xây địa chỉ IPv6 đích (RLOC16 của Router) và gửi JSON (temp, hum, co2, tvoc) lên cổng 5683, URI /sensor.
- Router nhận và chuyển tiếp: đăng ký CoAP resource /sensor với handler coap_relay_handler; khi nhận gói từ SED, trích 4 ký tự cuối IPv6 nguồn làm node_id (vd SED_abcd), thêm vào JSON rồi gửi lên Border Router qua multicast ff03::1. Router cũng tự gửi dữ liệu với node_id = "Router_H2".
- Cơ chế: dựa trên Thread (tự tìm Parent) và CoAP (gửi trực tiếp đến Parent), không cần giao thức riêng; SED chỉ thức theo chu kỳ poll hoặc khi có ngưỡng cảm biến (GPIO wake) để tiết kiệm điện.

Nhờ Thread tự tìm Parent và CoAP định tuyến trực tiếp, việc ghép nối SED–Router hoàn toàn tự động, không cần can thiệp thủ công hay giao thức phụ.

4.4 Triển khai xử lý thông số từ các cảm biến

Hệ thống dùng tác vụ sensor_task trong FreeRTOS để khởi tạo, đọc và xử lý dữ liệu cảm biến. Giá trị thô được cJSON_CreateObject() chuyển sang JSON, phân luồng theo vai trò Router và SED trước khi đẩy vào ngăn xếp giao thức.

4.4.1 Cảm biến SHT31

SHT31 kết nối qua I²C; trước khi đọc, hệ thống gọi I²Cdev_init() khởi tạo driver và sht3x_init_desc() cấu hình cổng I²C, chân SDA/SCL và địa chỉ slave.

a, Cơ chế thu thập dữ liệu trên Router

- Khởi tạo: trên Router, cấu hình GPIO_NUM_4 làm SDA và GPIO_NUM_5 làm SCL.

- Chu kỳ đọc: sensor_task chạy vòng lặp với vTaskDelay() tạo chu kỳ cố định; mỗi chu kỳ gọi sht3x_measure() trả về nhiệt độ, độ ẩm dạng số thực, không cần đưa CPU vào chế độ tiết kiệm.

b, Cơ chế thu thập dữ liệu trên SED

- Cấu hình ngủ và wake-up: để tối ưu pin cho SED, app_main dùng esp_pm_configure() với light_sleep_enable = true; chu kỳ thức thiết lập bằng esp_sleep_enable_timer_wakeup().
- Quy trình xử lý khi thức giấc (Wake-up):
  - Hết thời gian ngủ, phần cứng tự wake-up, khôi phục xung nhịp CPU.
  - sensor_task được giải phóng, gọi sht3x_measure() lấy nhiệt độ và độ ẩm.
  - Sau khi đọc và gửi gói CoAP bằng otCoapSendRequest(), khi gói rời bộ đệm, vi điều khiển quay lại Light Sleep để hạ dòng tiêu thụ, chờ chu kỳ ngắt tiếp theo.

Xử lý dữ liệu SHT31 trên SED trong sensor_task gồm: khởi tạo và cấu hình ngưỡng, đọc nhiệt độ/độ ẩm, xác định cảnh báo theo latch phần mềm và điều chỉnh tốc độ đo theo trạng thái. Lưu đồ chi tiết ở hình 4.5.

4.4.2 Cảm biến CCS811

CCS811 đo eCO_2 và TVOC, giao tiếp qua I^2C và dùng chân CCS811_WAK_PIN (GPIO 12) điều khiển thức/ngủ độc lập với CPU. Trước khi vận hành, gọi ccs811_init_desc() (đặt địa chỉ, cổng I^2C) và ccs811_init() khởi tạo phần cứng.

a, Cơ chế thu thập dữ liệu trên Router

- Khởi tạo và chân thức dậy: trên Router, chân WAK (GPIO_NUM_12) cấu hình output bằng gpio_config(), luôn kéo xuống mức thấp (gpio_set_level) để CCS811 hoạt động liên tục.
- Chu kỳ đọc: trong vòng lặp sensor_task, gọi ccs811_get_results() đọc CO_2 và TVOC sau mỗi vTaskDelay(); do cấp nguồn liên tục, cảm biến chạy Constant power mode.

b, Cơ chế thu thập dữ liệu trên SED

- Điều khiển nguồn phối hợp: để tối ưu dòng cho SED, CCS811 không thức liên tục mà đóng/ngắt đồng bộ với chu kỳ Light Sleep của ESP32 (qua esp_pm_configure() và esp_sleep_enable_timer_wakeup()).
- Quy trình xử lý phối hợp khi thức giấc (Wake-up):
  - Khi ESP32 thức giấc, sensor_task được giải phóng và gọi gpio_set_level(CCS811_WAK_PIN, 0) kéo chân WAK xuống thấp để đánh thức CCS811 khỏi ngủ sâu.
  - Do cảm biến cần thời gian kích hoạt lại buồng đo, gọi vTaskDelay() đưa tác vụ vào chờ ngắn, nhường CPU để tiết kiệm năng lượng.
  - Kiểm tra dữ liệu sẵn sàng qua ccs811_get_results(); sau khi đọc CO_2 và TVOC, đóng gói bằng cJSON và gửi qua otCoapSendRequest().

- Khi mạng Thread xác nhận gói đã rời bộ đệm, gọi gpio_set_level(CCS811_WAK_PIN, 1) kéo WAK lên cao ép CCS811 vào Sleep mode, trước khi ESP32 quay lại Light Sleep.

4.5 Triển khai truyền nhận dữ liệu giữa các thiết bị

Mạng hạ tầng Thread xử lý luồng thông tin môi trường thông qua kiến trúc phân tầng giao thức CoAP (Constrained Application Protocol) chạy trên nền UDP/IPv6. Tiến trình định tuyến và chuyển tiếp dữ liệu được thực hiện bất đồng bộ nhằm tối ưu hóa băng thông, đảm bảo tính toàn vẹn gói tin và giảm thiểu tỷ lệ thất lạc dữ liệu trong không gian mạng Mesh.

4.5.1 Triển khai truyền nhận dữ liệu giữa SED và Router

Truyền thông giữa SED và Router cha theo mô hình Unicast trực tiếp (Client–Server) để tránh lỗi kiểm tra toàn vẹn dữ liệu mạng.

- Đóng gói tại SED (Client): sau khi sensor_task thu thập xong, dữ liệu JSON được tạo bằng cJSON_CreateObject(); gói CoAP tạo qua otCoapNewMessage() với loại OT_COAP_TYPE_NON_CONFIRMABLE, mã OT_COAP_CODE_POST, đường dẫn /sensor (otCoapMessageAppendUriPathOptions). Gói gửi bằng otCoapSendRequest() tới địa chỉ RLOC của Router cha (lấy từ otThreadGetParentInfo).
- Tiếp nhận tại Router (Server): Router khởi động dịch vụ bằng otCoapStart(), đăng ký tài nguyên /sensor (otCoapResource + otCoapAddResource) gắn với coap_relay_handler(). Khi nhận gói từ SED, handler bóc tách JSON, trích địa chỉ IPv6 nguồn (mPeerAddr) để sinh node_id dạng SED_xxxx rồi gắn vào dữ liệu trước khi chuyển tiếp lên Border Router.

Sự phối hợp giữa đóng gói CoAP ở Client và xử lý sự kiện bất đồng bộ ở Server đảm bảo tính thời gian thực và độ tin cậy cho liên kết SED – Router.

4.5.2 Triển khai truyền nhận dữ liệu giữa Router và Border Router

Truyền nhận dữ liệu giữa Router và Border Router (BR) trung chuyển toàn bộ dữ liệu cảm biến từ mạng Mesh Thread để chuẩn bị đẩy lên đám mây.

- Chuyển tiếp và đóng gói dữ liệu tại Router:
  - Xử lý chuyển tiếp: khi nhận CoAP từ SED qua coap_relay_handler(), Router dùng cJSON_Parse() giải mã và thêm node_id; với dữ liệu tự thân, Router dùng cJSON_CreateObject() sinh gói với định danh “Router_H2”.
  - Phát lên Border Router: Router tạo gói CoAP bằng otCoapNewMessage() (loại OT_COAP_TYPE_NON_CONFIRMABLE), đặt đường dẫn /sensor (otCoapMessageAppendUriPathOptions) rồi phát bằng otCoapSendRequest() qua multicast ff03::1 về Border Router.

- Tiếp nhận và phân tích dữ liệu tại Border Router:
  - Khởi tạo máy chủ CoAP: Border Router kích hoạt dịch vụ bằng otCoapStart(), đăng ký tài nguyên /sensor (otCoapResource + otCoapAddResource) gắn với coap_recv_handler().
  - Tiếp nhận và trích xuất: khi có gói CoAP từ Router, coap_recv_handler() bóc tách payload JSON và trích địa chỉ IPv6 nguồn (mPeerAddr); cả payload và IP được đưa vào update_node_data() để chuẩn bị publish lên MQTT Broker.

Sự phối hợp giữa multicast của Router và bộ lắng nghe bất đồng bộ tại /sensor của Border Router giúp luồng CoAP/JSON luân chuyển liên tục, hoàn thiện hạ tầng dữ liệu trước khi kết nối mạng diện rộng.

4.6 Cấu hình MQTT Broker

4.6.1 Triển khai trên HiveMQ

Hệ thống dùng HiveMQ Cloud làm MQTT Broker trung tâm để quản lý và phân phối luồng sự kiện. Broker được cấu hình như sau:

- Tham số hạ tầng kết nối:
  - Địa chỉ máy chủ (URL): mqtts://a9261df0be8c4c78b660c609c1fa6e60.s1.eu.hivemq.cloud định tuyến gói tin về cụm máy chủ đám mây.
  - Cổng dịch vụ (Port): 8883, bắt buộc mọi truyền nhận qua SSL/TLS.
- Cơ chế xác thực và bảo mật:
  - Định danh Client: xác thực với Broker qua tài khoản cố định -username = “iot_gateway”, password = “Gateway123”.
  - Cấu trúc topic: phân cấp sensor/<node_id>; Front-end dùng wildcard sensor/# để thu thập dữ liệu toàn mạng linh hoạt.

HiveMQ Cloud sau khi cấu hình đã thiết lập hành lang an toàn dữ liệu và không gian phân luồng gói JSON cho toàn hệ thống.

4.6.2 Triển khai trên Border Router

Để chuyển tiếp dữ liệu từ mạng Thread ra diện rộng, Border Router cấu hình một Client MQTT bảo mật kết nối và publish dữ liệu lên HiveMQ Cloud.

- Cấu hình kết nối: dùng cấu trúc esp_mqtt_client_config_t; địa chỉ Broker đặt ở trường uri với mqtts:// qua cổng 8883, trường crt_bundle_attach gọi esp_crt_bundle_attach để tích hợp chứng chỉ TLS, kèm username và password xác thực.
- Khởi tạo và vận hành Client: gọi esp_mqtt_client_init() để tạo Client, esp_mqtt_client_register_event() đăng ký mqtt_event_handler(), rồi esp_mqtt_client_start() kích hoạt kết nối bất đồng bộ.
- Xử lý sự kiện kết nối: khi TLS thiết lập xong, mqtt_event_handler() bắt sự kiện MQTT_EVENT_CONNECTED và bật cờ s_mqtt_connected = true báo hiệu sẵn sàng.

– Đóng gói và đẩy dữ liệu: tại update_node_data(), nếu s_mqtt_connected hợp lệ, hệ thống tạo topic sensor/<node_id> từ định danh nút rồi gọi esp_mqtt_client_publish() với QoS = 1 để đẩy JSON lên cloud, đảm bảo bắt tay xác nhận, không thất lạc dữ liệu.

Để tường minh hóa trình tự khởi tạo kết nối TLS, chu trình quản lý sự kiện kết nối và tiến trình gọi hàm đẩy dữ liệu JSON từ tầng biên lên HiveMQ Cloud Broker, luồng logic thực thi được cụ thể hóa qua lưu đồ thuật toán dưới đây:

4.7 Triển khai Web Dashboard

Hệ thống triển khai một ứng dụng giao diện giám sát trung tâm (Web Dashboard) dựa trên các công nghệ tiêu chuẩn HTML5, CSS3 và JavaScript (ES6). Giao diện hoạt động theo cơ chế Single-Page Architecture (SPA) hướng sự kiện, được lập trình tập trung trên môi trường VS Code, quản lý mã nguồn bằng Git/GitHub và triển khai dịch vụ lưu trữ trực tuyến (Cloud Hosting) tự động qua nền tảng Render.

4.7.1 Quy trình quản lý mã nguồn và triển khai CI/CD

Chu trình đóng gói và phát hành Dashboard gồm các bước:

– Quản lý mã nguồn trên VS Code: các file tĩnh (index.html, style.css, script.js) được phát triển trên VS Code; dùng Git tạo kho cục bộ và đóng gói phiên bản qua git commit.

– Phát hành lên GitHub: đẩy mã nguồn lên GitHub qua nhánh chính:

```
git remote add origin <url_kho_luu_tru_github>
git branch -M main
git push -u origin main
```

Hình 4.5 Đẩy dữ liệu code lên Github

– Triển khai tự động trên Render (CI/CD): Render được liên kết với tài khoản GitHub và trỏ vào kho của đồ án.

- Cấu hình dịch vụ: ứng dụng khởi tạo dạng Static Site, Build Command để trống, Publish Directory là thư mục gốc chứa index.html.
- Kích hoạt tự động: mỗi khi git push lên nhánh main, Render bắt Webhook, tự build lại và phân phối giao diện qua HTTPS.

4.7.2 Triển khai cấu trúc không gian hiển thị (HTML/CSS)

Không gian hiển thị Dashboard được thiết kế để tối ưu khả năng bao quát thông tin:

– Bố cục (Layout Grid): dùng CSS Grid kết hợp Flexbox tạo lưới thẻ thiết bị (nodes-grid), ánh xạ phần cứng thành 4 thẻ giám sát: Thiết bị 01 (Router_H2), 02 (Node_ef7:c0c9), 03 (Node_ac9d:b03), 04 (Node_1e2:612b); tích hợp Responsive (media) co giãn trên cả PC và di động.

– Phân cấp chỉ số (Metrics Hierarchy): mỗi thẻ (node-card) phân định 4 vùng hiển thị qua các id phần cứng: Nhiệt độ (temp-), Độ ẩm (hum-), eCO_2 (co2-) và TVOC (tvoc-).

– Cảnh báo trực quan: CSS có sẵn lớp trạng thái động -#system-status quản lý safe (xanh) và danger (đỏ nhấp nháy); lớp .alert đổi màu nền ô chỉ số sang đỏ khi một đại lượng vượt ngưỡng.

4.7.3 Thiết lập kết nối và xử lý luồng sự kiện MQTT (JavaScript)

Mã điều khiển (script.js) dùng thư viện mqtt.min.js thiết lập kênh bất đồng bộ qua WebSocket bảo mật (WSS):

– Khởi tạo kết nối: dùng tài khoản webclient mặc định và đăng ký toàn bộ nút bằng wildcard client.subscribe('sensor/#'); Client khởi tạo qua hằng số cấu hình kết nối:

```
const client = mqtt.connect('wss://204e1cd42856497598802c289c184353.s1.eu.hivemq.cloud:8884/mqtt', options);
```

Hình 4.6 Thiết lập kết nối MQTT Broker với Render

– Xử lý gói tin (Message Handler): khi Broker đẩy dữ liệu, sự kiện client.on('message', ...) kích hoạt, xử lý theo các bước:

- JSON.parse(payload) chuyển chuỗi thô thành đối tượng JavaScript.
- Dùng NODE_MAP ánh xạ payload.node_id thành các khóa ngắn (h2, b6f, n3, n4) khớp DOM ID trên HTML.
- Cập nhật giá trị vào giao diện qua textContent kèm toFixed(1).

4.7.4 Thuật toán kiểm tra ngưỡng cảnh báo và trực quan hóa đồ thị

Ngoài cập nhật chỉ số, Dashboard chạy song song hai thuật toán lõi giám sát nâng cao:

– Đánh giá nguy cơ cháy (checkAlerts): thiết lập cứng hai ngưỡng kỹ thuật TEMP_THRESHOLD (ngưỡng nhiệt độ) và CO2_THRESHOLD (ngưỡng CO_2); mỗi khi nhận dữ liệu, hàm checkAlerts() duyệt mảng latestData và dùng classList.toggle('alert', condition) ép thẻ chỉ số hoặc cả Dashboard sang trạng thái danger khi có nút vượt ngưỡng nhiệt độ hoặc CO_2, đồng thời nhấp nháy cảnh báo.

– Đồ thị thời gian thực (Chart.js): dựng biểu đồ đường riêng cho từng thiết bị; khi có dữ liệu, pushPoint() đẩy điểm vào bộ đệm và giới hạn MAX_CHART_PTS = 60 để tránh tràn bộ nhớ, rồi chart.update('none') làm mới mượt mà.

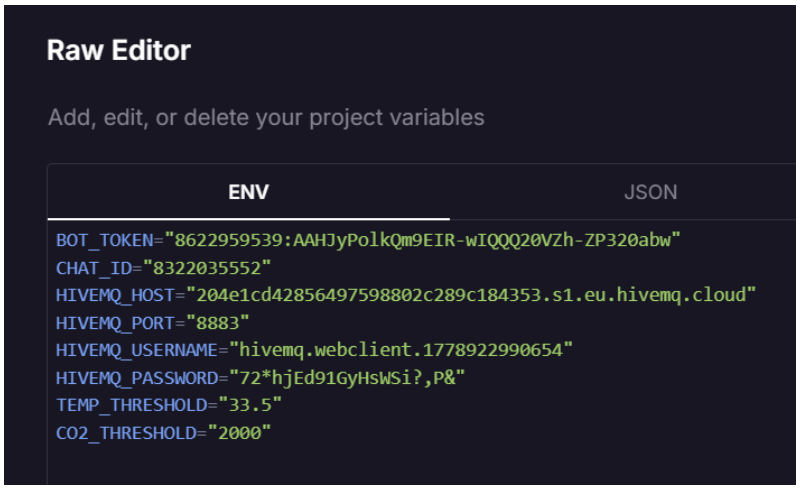
Quy trình phát triển đồng bộ (VS Code → GitHub → Render) kết hợp trình điều khiển JavaScript hướng sự kiện CoAP/MQTT đã hoàn thiện tầng ứng dụng giám sát trực quan, đảm bảo thời gian thực và cảnh báo sớm tường minh cho hệ thống.

4.8 Triển khai cảnh báo qua Telegram

Sau khi cấu hình MQTT Broker trên HiveMQ Cloud, nhóm xây dựng dịch vụ cảnh báo qua Telegram: mã nguồn quản lý trên Github và triển khai tự động lên Railway, đăng ký kết nối tới MQTT của HiveMQ Cloud qua biến môi trường để chạy thường trực và liên tục lắng nghe dữ liệu. Quy trình gồm các bước:

- Phát triển mã nguồn dịch vụ: Xây dựng chương trình cho bot Telegram đảm nhận đồng thời hai vai trò -làm MQTT client đăng ký (subscribe) dữ liệu cảm biến từ HiveMQ Cloud và gọi Bot API của Telegram để gửi tin nhắn cảnh báo tới người dùng.
- Quản lý mã nguồn trên Github: Đẩy toàn bộ source code lên một repository trên Github nhằm quản lý phiên bản và làm nguồn triển khai tự động.
- Triển khai lên Railway: Liên kết repository Github với Railway để nền tảng tự động build và chạy dịch vụ; nhờ đó bot hoạt động thường trực 24/7, độc lập với máy tính cá nhân và tự khởi động lại khi gặp sự cố.
- Đăng ký địa chỉ MQTT và cấu hình qua biến môi trường: khai báo HIVEMQ_HOST, cổng 8883 (HIVEMQ_PORT), HIVEMQ_USERNAME/HIVEMQ_PASSWORD, BOT_TOKEN, CHAT_ID cùng các ngưỡng (TEMP_THRESHOLD và CO2_THRESHOLD = 2000 ppm) ngay trên Railway. Lưu dưới dạng biến môi trường giúp tách thông tin nhạy cảm khỏi mã nguồn và dễ thay đổi mà không cần build lại.

Các biến môi trường phục vụ kết nối và cấu hình ngưỡng của dịch vụ bot trên Railway được minh họa trong hình dưới đây:



Hình 4.7 Triển khai cấu hình trên Railway

Nhờ được triển khai trên nền tảng Railway, kênh cảnh báo qua Telegram hoạt động ổn định, liên tục và sẵn sàng gửi thông báo tới người dùng mọi lúc, hoàn thiện cơ chế cảnh báo từ xa song song với web dashboard và nâng cao đáng kể tính kịp thời của hệ thống báo cháy.

4.9 Biên dịch và nạp firmware

Sau khi viết code cho các thành phần (Border Router, RCP, Router, SED), tiến hành biên dịch và nạp firmware. Quy trình giống nhau cho mọi board, chỉ khác cổng COM và log: mở dự án trong VS Code (đã cấu hình ESP-IDF), chạy idf.py build để biên dịch (tạo file .bin trong build/), kết nối board qua USB, xác định cổng COM rồi chạy idf.py -p COMx flash để nạp. Khác biệt cho từng thiết bị:

- Border Router (ESP32 WROOM 32): sau khi nạp, mở Serial Monitor (idf.py -p COMx monitor) xem log kết nối Wi-Fi, đồng bộ NTP, khởi tạo

OpenThread và kết nối MQTT; địa chỉ IP in ra dùng để truy cập web dashboard.

- RCP (ESP32-H2): nạp firmware RCP, log hiển thị giao thức Spinel (không có log ứng dụng); sau đó cắm vào ESP32 WROOM 32 qua UART (TX, RX, GND).
- Router (ESP32-H2): nạp firmware router; log hiển thị quá trình tham gia mạng Thread, đọc SHT31/CCS811 và gửi CoAP (Joined Thread as CHILD rồi nâng cấp lên Router).
- SED (ESP32-H2): nạp firmware SED; log hiển thị khởi tạo cảm biến và light-sleep, in cảnh báo khi vượt ngưỡng. Ở light-sleep monitor có thể ngắt tạm thời nên cần debug pin để theo dõi.

Như vậy, quy trình nạp firmware cho mọi board theo các bước chung, chỉ khác cổng COM và log, giúp triển khai và gỡ lỗi dễ dàng.

4.10 Kết luận chương

Chương 4 đã trình bày toàn bộ quá trình triển khai từ môi trường phát triển đến hiện thực hóa thiết kế ở Chương 3. Môi trường dùng ESP-IDF v5.5.4 trên VS Code cùng các thư viện I²Cdev, ccs811, sht3x, cJSON và MQTT client.

Các lưu đồ thuật toán cho Border Router, RCP, Router và SED đã được xây dựng chi tiết (từ khởi tạo, giám sát nền đến xử lý sự cố). Mạng Thread thiết lập thành công với đồng bộ datasheet và ghép nối tự động SED–Router qua CoAP.

Xử lý dữ liệu SHT31 và CCS811 được thực hiện với cơ chế tiết kiệm năng lượng phù hợp từng thiết bị; truyền nhận CoAP SED–Router và Router–Border Router ổn định. Cấu hình MQTT Broker trên HiveMQ Cloud và Border Router đã hoàn tất, Web Dashboard triển khai CI/CD tự động trên Render. Ngoài ra, kênh cảnh báo từ xa qua bot Telegram đã được xây dựng, đẩy lên Github và triển khai thường trực trên Railway, kết nối HiveMQ Cloud để tự gửi cảnh báo khi vượt ngưỡng.

Cuối cùng, quy trình biên dịch và nạp firmware đã hoàn tất, sẵn sàng cho kiểm thử và đánh giá ở Chương 5.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Kết quả giao tiếp mạng Thread

5.1.1 Thời gian join mạng của SED

Thời gian gia nhập mạng (join) của SED quyết định tính sẵn sàng và khả năng khôi phục liên lạc sau sự cố mất nguồn. Thực nghiệm cấp nguồn cho SED trong vùng đã có mạng Mesh của Router, dùng UART (Terminal) để giám sát thay đổi trạng thái và timestamp của vi điều khiển.

Quá trình chuyển trạng thái của SED từ khởi động đến khi gia nhập mạng được ghi nhận qua chuỗi log Debug từ cổng UART của ESP32 (xem Hình 5.1).

Phương pháp xác định: trích dấu thời gian phần cứng (ms) qua debug UART (baudrate 115200), t_{start} ghi khi OpenThread bắt đầu quét mạng, $t_{success}$ ghi khi hoàn tất bắt tay và chuyển sang vai trò nút con (CHILD). Thời gian join:

$$t_{join} = t_{success} - t_{start}$$

Dữ liệu chuyển trạng thái mạng của SED trích từ log UART được tổng hợp ở hình 5.1.

```
> I(503) OPENTHREAD:[N] Mle-----------: Role disabled -> detached
I (513) OT_STATE: netif up
I (523) main_task: Returned from app_main()
I(673) OPENTHREAD:[N] Mle-----------: Role detached -> child
```

Hình 5.1 Trạng thái của SED trong mạng Thread

Áp dụng số liệu đo thực tế vào công thức:

$$t_{join} = 673 - 503 = 170\ ms = 0.17\ s$$

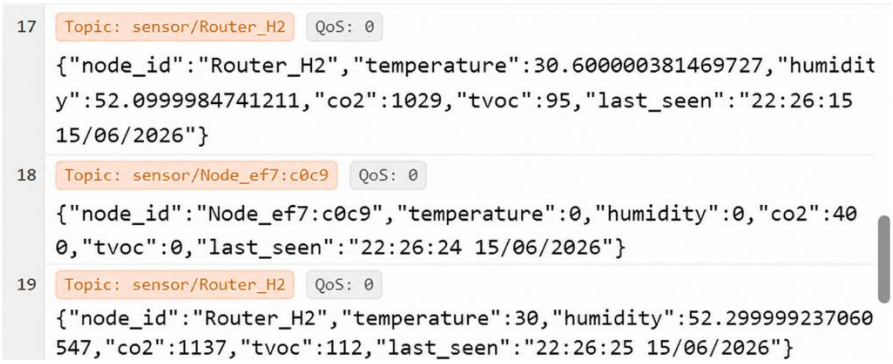
Nhờ lưu thông tin mạng trước đó trong bộ nhớ không biến đổi (NVM), SED chỉ mất 170 ms (0,17 giây) để tái lập liên lạc và chuyển sang vai trò nút con trong mạng Mesh Thread.

Tốc độ 0,17 giây phản ánh hiệu năng xử lý thời gian thực xuất sắc của OpenThread, vượt yêu cầu của hệ thống cảnh báo khẩn cấp: SED ngay sau khi được cấp nguồn lại lập tức trực tuyến, sẵn sàng gửi gói CoAP/JSON cảnh báo về Border Router.

5.1.2 Độ ổn định kết nối

Độ ổn định kết nối của mạng Mesh Thread là chỉ số cốt lõi đánh giá khả năng vận hành bền bỉ trong môi trường nhiều nhiễu vô tuyến. Thực nghiệm dựa trên hai tiêu chí: tỷ lệ duy trì gói tin (PDR) và thời gian tự cấu hình lại đường truyền (self-healing) khi một nút mất nguồn đột ngột.

Về PDR, các nút SED được cấu hình phát liên tục dữ liệu JSON (nhiệt độ, độ ẩm, khí độc) về Border Router theo chu kỳ cố định.

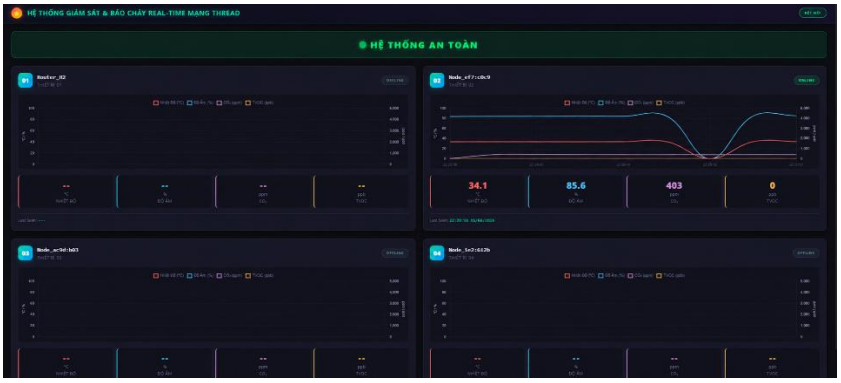


Hình 5.2 Dữ liệu đẩy lên MQTT broker hiển thị trên HiveMQ

Giám sát dài hạn cho thấy luồng CoAP đạt độ tin cậy rất cao, với tỷ lệ truyền nhận gói tin (PDR) đạt hiệu suất cao và tỷ lệ thất thoát gói không đáng kể, chứng tỏ liên kết vô tuyến OpenThread ổn định, kháng nhiễu tốt và nằm trong ngưỡng an toàn cho hệ thống cảnh báo cháy.

Về tự phục hồi, kịch bản giả lập ngắt nguồn nút Router trung gian (đang bắc cầu) cùng 2 node; ngay sau đó các nút SED mất kết nối với trạm cha hiện tại.

Thuật toán định tuyến MLE của OpenThread tự kích hoạt: các nút con quét kênh tìm mạng mới rồi thiết lập liên kết qua Router cha khác hoặc thẳng về Border Router. Toàn bộ quá trình tìm tuyến thay thế và khôi phục dữ liệu diễn ra gần như tức thời, không cần can thiệp thủ công.



Hình 5.3 Mô phỏng hệ thống khi Router mất kết nối

Kết quả giả lập trực quan ở hình 5.3: sau khi Router_H2 và hai nút lân cận OFFLINE, nút Node_ef7:c0c9 tự định tuyến lại, thiết lập liên kết mới để giữ trạng thái ONLINE và tiếp tục cập nhật dữ liệu lên Dashboard mà không làm sập hệ thống.

Kết quả về tính ổn định dòng dữ liệu và khả năng tự thích ứng của topo Mesh OpenThread khẳng định độ tin cậy vượt trội: cơ chế self-healing đảm bảo luồng thông tin cứu hỏa luôn thông suốt, đáp ứng yêu cầu vận hành nghiêm ngặt của hệ thống phòng cháy chữa cháy thực tế.

5.2 Kết quả đọc cảm biến

Mục này kiểm chứng năng lực thu thập dữ liệu thời gian thực từ các cảm biến trên nút đầu cuối. Thực nghiệm đặt nút trong các điều kiện giả lập (gia nhiệt chủ động, tạo khói bằng giấy đốt) để kiểm tra độ nhạy, độ chính xác và tương tác giữa SHT31 và CCS811. Dữ liệu được trực quan hóa trên giao diện quản lý ở hình 5.4.



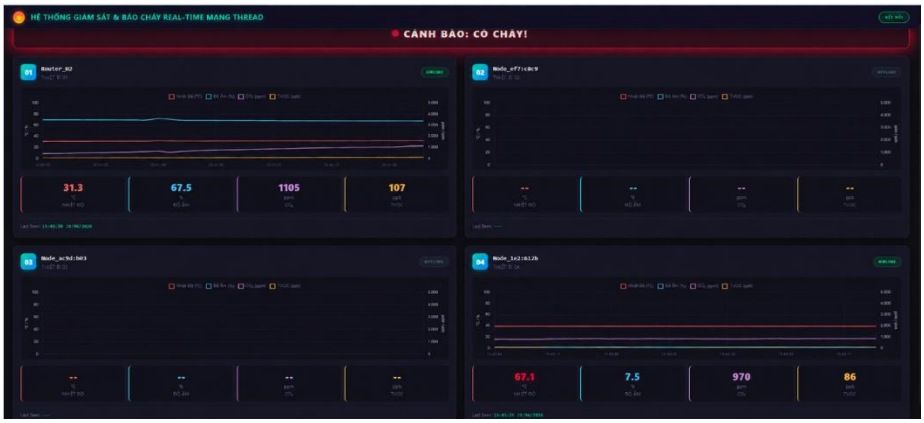
Hình 5.4 Kết quả đọc cảm biến tại SED

Kết quả cho thấy chu trình xử lý dữ liệu chính xác và đồng bộ: SHT31 (qua I²C) phản hồi nhạy với thay đổi vi khí hậu, khi gia nhiệt giá trị nhiệt độ tăng vọt, khớp với thông số nhà sản xuất. Dữ liệu SHT31 được dùng bù nhiệt độ/độ ẩm cho CCS811, triệt tiêu trôi điểm gốc; nhờ đó CCS811 ghi nhận chính xác biến động eCO_2 và TVOC trong thử nghiệm khói, nhanh chóng vượt ngưỡng để kích hoạt báo động.

5.3 Kết quả trong các trạng thái kiểm thử

a, Tình huống 1: Kích hoạt cảnh báo khi nhiệt độ tăng cao vượt ngưỡng

Khi dùng nguồn nhiệt chủ động tiếp cận nút mạng, SHT31 ghi nhận nhiệt độ tăng liên tục và nhanh chóng vượt ngưỡng an toàn.



Hình 5.5 Giao diện Web Dashboard chuyển trạng thái cảnh báo nguy hiểm màu đỏ khi nhiệt độ vượt ngưỡng

Sự kiện vượt ngưỡng được đóng gói thành gói CoAP khẩn cấp truyền về trung tâm. Như ở hình 5.5, giao diện chuyển từ xanh an toàn sang đỏ, thanh trạng thái hiển thị "HỆ THỐNG NGUY HIỂM" và ô nhiệt độ của thiết bị sự cố nhấp nháy đỏ, định vị chính xác nguồn nhiệt để người vận hành nhận biết tức thì.

b, Tình huống 2: Kích hoạt cảnh báo khi nồng độ khí CO_2 vượt ngưỡng

Tiếp theo, đưa nút mạng vào môi trường có nồng độ eCO_2 tăng cao do khói tích tụ. Khi CCS811 ghi nhận vượt ngưỡng, dòng gói JSON liên tục đẩy cờ cảnh báo lên đám mây.



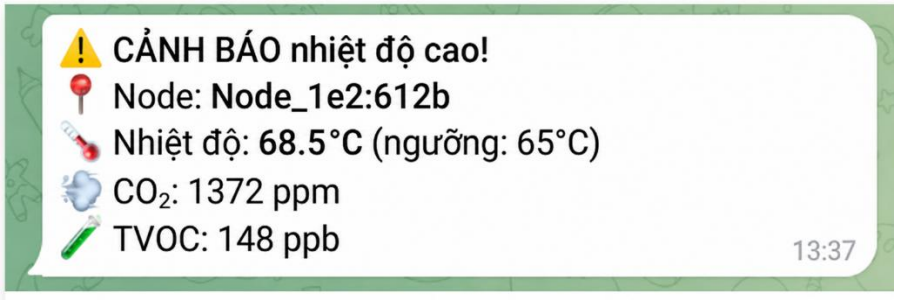
Hình 5.6 Giao diện Web Dashboard chuyển trạng thái cảnh báo nguy hiểm màu đỏ khi eCO_2 vượt ngưỡng

Minh chứng tại hình 5.6 cho thấy dù nhiệt độ đã hạ về bình thường, chỉ số CO_2 (ppm) vẫn neo ở mức cao vượt ngưỡng; dashboard tiếp tục duy trì cảnh báo đỏ để người quản lý không bỏ sót nguy cơ ngạt khí trong sự cố cháy âm ỉ.

5.4 Kết quả cảnh báo qua Telegram

Để kiểm chứng chức năng cảnh báo từ xa, nhóm tiến hành thử nghiệm bằng cách dùng nguồn nhiệt tác động lên một nút cảm biến cho đến khi nhiệt độ vượt ngưỡng cài đặt. Ngay khi phát hiện sự kiện vượt ngưỡng, dịch vụ bot Telegram (chạy thường trực trên Railway và đăng ký dữ liệu từ HiveMQ Cloud) lập tức gửi tin nhắn cảnh báo về ứng dụng Telegram của người dùng.

Nội dung tin nhắn được trình bày trực quan, bao gồm: tiêu đề cảnh báo, mã định danh nút gặp sự cố, giá trị nhiệt độ đo được kèm ngưỡng kích hoạt, nồng độ eCO_2, chỉ số TVOC và mốc thời gian ghi nhận. Hình 5.7 dưới đây minh họa một tin nhắn cảnh báo nhiệt độ cao thực tế nhận được trên Telegram:



Hình 5.7 Kết quả gửi từ Telegram

Trong tình huống minh họa, hệ thống ghi nhận nút Node_1e2:612b đạt nhiệt độ 68,5 °C -vượt ngưỡng kích hoạt 65 °C, kèm theo nồng độ eCO_2 1372 ppm và TVOC 148 ppb. Tin nhắn được gửi gần như tức thời sau khi phát hiện sự cố (trong vài giây) với đầy đủ thông tin, giúp người dùng nhanh chóng xác định vị trí và mức độ nguy hiểm. Như vậy, kênh cảnh báo qua Telegram hoạt động chính xác và ổn định, bổ sung hiệu quả cho giao diện Web Dashboard và đảm bảo người dùng luôn nhận được thông báo kịp thời ngay cả khi không trực tiếp theo dõi giao diện web.

5.5 Hiệu quả tiết kiệm năng lượng

Với các nút cảm biến chạy pin, tối ưu công suất tiêu thụ là yếu tố then chốt quyết định tuổi thọ và tính thực tế của sản phẩm. Thực nghiệm dùng thiết bị đo dòng chuyên dụng để giám sát dòng tiêu thụ của nút SED (nguồn 2 pin AA nối tiếp, ~3V) qua các giai đoạn của một chu kỳ làm việc: ngủ, thức dậy đọc cảm biến, cấu hình RF và truyền gói CoAP/JSON về Border Router.

Kết quả đo cho thấy chế độ SED tiết kiệm năng lượng tốt nhờ phối hợp quản lý nguồn của vi xử lý và giao thức OpenThread. Khoảng 95% chu kỳ nút ở light-sleep với dòng rất thấp (khoảng 200 µA) khi tắt khối RF và ngoại vi không cần thiết, chỉ giữ RTC Timer để đếm thời gian thức.

Khi tới chu kỳ thức (mỗi 25 giây), nút tích cực trong khoảng 80–120 ms với dòng đỉnh 100–150 mA để bật lại tầng PHY, đọc cảm biến và phát gói CoAP. Nhờ cơ chế truyền gián tiếp (Indirect Transmission) của Thread -Router cha giữ tạm gói tin hộ nút con -SED không cần giữ sóng chờ phản hồi mà tắt RF và ngủ lại ngay sau khi truyền xong.

Theo tích phân dòng trung bình một chu kỳ, với cặp pin AA ~2000 mAh, nút cảm biến vận hành liên tục 9–12 tháng tùy tần suất báo động. Kết quả này minh chứng giải pháp tối ưu năng lượng kết hợp cấu trúc SED đạt hiệu quả cao, đáp ứng yêu cầu vận hành dài hạn của thiết bị phòng cháy chữa cháy.

5.6 Đánh giá về độ trễ và độ tin cậy của hệ thống

- Đánh giá về độ trễ: Độ trễ tổng thể từ lúc cảm biến phát hiện sự cố đến khi hiển thị lên Web Dashboard dao động trong khoảng từ 0,5 giây đến 1,2 giây. Trong đó, trễ truyền dẫn nội bộ mạng Mesh Thread cực kỳ nhỏ (chỉ vài chục mili giây) nhờ băng thông 250 kbps của tầng vật lý IEEE 802.15.4. Thời gian chiếm tỷ trọng lớn nhất là quá trình chuyển đổi giao thức từ CoAP sang MQTT tại Gateway và truyền lên HiveMQ Cloud. Tốc độ phản hồi dưới mức giây (Sub-second latency) này hoàn toàn đáp ứng vượt trội các yêu cầu khắt khe về thời gian thực của tiêu chuẩn an toàn phòng cháy chữa cháy (PCCC). Riêng với kênh cảnh báo qua Telegram, do tin nhắn được định tuyến qua máy chủ Telegram trên Internet nên thời gian nhận cảnh báo nhỉnh hơn (khoảng vài giây), song vẫn đảm bảo tính kịp thời cho mục đích cảnh báo từ xa.
- Đánh giá về độ tin cậy: Hệ thống đạt độ tin cậy vận hành cao nhờ kiến trúc mạng Mesh phi tập trung, không có điểm lỗi đơn nhất (Single Point of Failure). Thực nghiệm đã minh chứng khi nút định tuyến trung gian gặp sự cố mất nguồn, thuật toán tự chữa lành (Self-healing) kích hoạt giúp các nút con tự động tái cấu hình đường truyền sang Border Router chỉ trong vòng dưới 2 giây để duy trì dòng dữ liệu.

5.7 Khả năng mở rộng

- Mở rộng số lượng nút (Scale-out): định tuyến động của OpenThread cho phép thêm cảm biến mới vào mạng Mesh mà không phải chỉnh cấu hình nút

cũ; khi phạm vi tăng, các Router mới tự kết nối, tự lập bản đồ định tuyến để bắc cầu và mở rộng phủ sóng cho SED, hỗ trợ hàng trăm thiết bị.

- Mở rộng tính năng ứng dụng: phần cứng mạnh và cảm biến giao tiếp I^2C cho phép nâng cấp linh hoạt -ở tầng thiết bị có thể thêm cảm biến khí độc khác hoặc điều khiển cơ cấu chấp hành (van khóa gas, phun nước, quạt hút khói); ở tầng ứng dụng, luồng JSON từ MQTT Broker sẵn sàng tích hợp với hệ thống BMS hoặc ứng dụng giám sát bên thứ ba.

5.8 Kết luận chương

Chương 5 đã trình bày kết quả thực nghiệm và đánh giá. Về mạng Thread, thời gian join của SED ~170 ms, tự phục hồi dưới 2 giây khi mất Router và tỷ lệ truyền nhận gói tin đạt hiệu suất cao.

Các cảm biến SHT31 và CCS811 hoạt động ổn định, phản hồi chính xác. Ba kịch bản kiểm thử (nhiệt độ cao, eCO_2 vượt ngưỡng, phát hiện khói) đều phản hồi kịp thời với độ trễ dưới 1,2 giây; kênh cảnh báo Telegram (trên Railway) hoạt động chính xác, tự gửi tin nhắn kèm đầy đủ thông số khi vượt ngưỡng.

Về năng lượng, light-sleep giúp SED chạy bền trên pin AA, đáp ứng vận hành lâu dài. Các kết quả khẳng định tính khả thi và hiệu quả của giải pháp.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Tổng kết những kết quả đạt được

Sau quá trình nghiên cứu lý thuyết, thiết kế phần cứng và thực nghiệm, đề tài "Thiết kế hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh" đã cơ bản đạt được các mục tiêu, nhiệm vụ đề ra; bên cạnh đó vẫn còn một số hạng mục chưa hoàn thiện, đặc biệt là tích hợp hoàn chỉnh cảm biến khói RE46C194 và đo kiểm thời lượng pin dài hạn được phân tích ở mục 6.2. Các kết quả chính đạt được gồm:

- Về nghiên cứu lý thuyết: làm chủ lý thuyết cốt lõi của mạng Mesh OpenThread, cơ chế quản lý năng lượng của nút SED và các chuẩn giao tiếp ngoại vi của cảm biến chuyên dụng cho bài toán cứu hỏa.
- Về thiết kế phần cứng: hoàn thiện sơ đồ nguyên lý và thi công bo mạch các nút (Router trung gian và nút cảm biến đầu cuối), tích hợp đồng bộ SHT31, CCS811 chạy ổn định bằng pin AA.
- Về phần mềm và hệ thống trung tâm: cấu hình thành công OpenThread Border Router làm cầu nối CoAP/UDP nội bộ sang MQTT lên HiveMQ Cloud; xây dựng Web Dashboard hiển thị thời gian thực và quản lý logic cảnh báo; bổ sung kênh cảnh báo từ xa qua bot Telegram (trên Railway) tự gửi thông báo tức thời khi có sự cố.
- Về thực nghiệm và kiểm thử: hệ thống vượt qua các kịch bản thực tế với độ trễ dưới 1,2 giây, tỷ lệ truyền nhận đạt hiệu suất cao và tự chữa lành (self-healing) dưới 2 giây khi mất nguồn trạm trung gian.

Nhìn chung, các khối chức năng từ phần cứng đến ứng dụng giám sát đều vận hành đồng bộ, chính xác và tin cậy cao, chứng minh tính đúng đắn của giải pháp và năng lực làm chủ công nghệ mạng không dây tiên tiến của sinh viên.

6.2 Hạn chế và nguyên nhân

Bên cạnh kết quả đạt được, hệ thống còn một số hạn chế về kỹ thuật và điều kiện triển khai; nhìn nhận thẳng thắn các khuyết điểm và nguyên nhân là cơ sở để tối ưu sản phẩm:

- Cảm biến khói RE46C194 chưa tích hợp và kiểm thử hoàn chỉnh: đã có sơ đồ nguyên lý và cơ chế giao tiếp, nhưng do thời gian và kinh phí chưa có buồng khói quang học hoàn chỉnh để đo đạc, hiệu chỉnh; hệ thống chưa đưa tín hiệu khói vào luồng cảnh báo.
- Cự ly vô tuyến suy giảm trong môi trường nhiều vật cản: băng tần 2.4 GHz bước sóng ngắn nên đâm xuyên kém, dễ bị hấp thụ/phản xạ qua tường bê tông dày hoặc kết cấu phức tạp.
- CCS811 còn độ trễ 3-5 giây: do đặc tính cảm biến màng oxit kim loại (MOX) cần thời gian nung nóng để khí khuếch tán và phản ứng nên không cho kết quả tức thời.

– Giám sát từ xa qua Web Dashboard phụ thuộc hoàn toàn vào Internet (WAN): hiện kết nối qua Cloud bên thứ ba (HiveMQ Cloud) qua MQTT, chưa có máy chủ cục bộ dự phòng khi mất mạng diện rộng.

Các hạn chế này chủ yếu do đặc tính linh kiện và giới hạn thời gian, kinh phí; chúng không ảnh hưởng tính đúng đắn của cấu trúc logic và hoàn toàn có thể khắc phục bằng các nâng cấp ở giai đoạn sau.

6.3 Đề xuất hướng phát triển tương lai

6.3.1 Tích hợp học máy để dự đoán cháy

Để chủ động phòng ngừa và giảm báo động giả, việc tích hợp trí tuệ nhân tạo ở tầng ứng dụng là bước đi tất yếu, chuyển hệ thống từ "phản ứng khi sự cố xảy ra" sang "dự đoán nguy cơ trước khi bùng phát":

– Triển khai học máy dự đoán: thu thập chuỗi dữ liệu lịch sử từ SHT31 và CCS811, áp dụng các mô hình học có giám sát (LSTM, Random Forest) trên máy chủ đám mây để phân tích xu hướng đồng thời của nhiệt độ, độ ẩm và eCO_2, đưa ra chỉ số nguy cơ cháy theo thời gian thực ngay cả khi chưa chạm ngưỡng nguy hiểm.
– Tối ưu báo động giả: huấn luyện mô hình phân biệt nhiễu môi trường (khói thuốc, khói nấu ăn, thời tiết oi) với dấu hiệu cháy thực, chỉ kích hoạt báo động khi khớp hành vi dữ liệu, nâng cao độ tin cậy.

Ứng dụng học máy không chỉ tối ưu thuật toán cảnh báo mà còn là cơ sở xây dựng hệ thống phòng cháy thông minh, tự thích ứng với mọi điều kiện thực địa.

6.3.2 Mở rộng thêm các loại cảm biến

Để nâng cao năng lực nhận diện và an toàn cho các môi trường đặc thù, cần mở rộng hệ sinh thái cảm biến ở tầng đầu cuối với các khối phần cứng chuyên dụng:

– Cảm biến khí gas (LPG) và CO: bổ sung module như MQ-2 hoặc cảm biến điện hóa công suất thấp để phát hiện sớm rò rỉ gas hoặc khí CO độc -khí không màu, không mùi thường gây ngạt trong các đám cháy kín.
– Cảm biến ngọn lửa (Flame Sensor): cảm biến hồng ngoại/tử ngoại phát hiện bức xạ từ ngọn lửa; kết hợp với dữ liệu nhiệt độ, khói tạo bộ quy tắc xác thực chéo, giúp báo động cháy tức thời và chính xác hơn.

Đa dạng hóa cảm biến đầu vào giúp mở rộng phạm vi bảo vệ trước nhiều nguy cơ và là nền tảng cho giải pháp an toàn toàn diện cho công trình hiện đại.

6.3.3 Nâng cấp giao diện quản lý đa nền tảng

Để tối ưu trải nghiệm người dùng và tăng tính linh hoạt, cần mở rộng khả năng tương tác của hệ thống lên đa nền tảng, đa dạng hóa các kênh tiếp nhận cảnh báo:

– Ứng dụng di động độc lập (Mobile App): xây dựng app iOS/Android (Flutter hoặc React Native) với thông báo đẩy (Push Notification) thời gian thực, hú còi báo động trên điện thoại kèm bản đồ định vị nút bị kích hoạt.

- Mở rộng kênh cảnh báo OTT và SMS: trên cơ sở kênh Telegram đã triển khai, mở rộng thêm Zalo API để gửi tin nhắn báo động kèm thông số hiện trường tới nhóm cứu hỏa nội bộ; đồng thời tích hợp SMS Gateway để đảm bảo cảnh báo vẫn truyền được ngay cả khi mất kết nối Internet.

Nâng cấp quản lý đa nền tảng giúp phá bỏ rào cản không gian, đảm bảo cảnh báo cháy luôn tiếp cận người có trách nhiệm nhanh nhất qua mọi phương tiện.

6.4 Kết luận chương

Chương 6 đã tổng kết kết quả của đồ án: thiết kế và triển khai thành công hệ thống báo cháy mạng Thread từ phần cứng, phần mềm nhúng đến giao diện giám sát và kênh cảnh báo từ xa qua Telegram. Các chỉ số về độ trễ, độ tin cậy và hiệu quả năng lượng khẳng định giải pháp đáp ứng tốt yêu cầu thực tế.

Chương cũng nêu các hạn chế còn tồn tại (chưa tích hợp hoàn chỉnh cảm biến khói RE46C194, chưa đo dòng tiêu thụ thực tế dài hạn) và đề xuất ba hướng phát triển: tích hợp học máy dự đoán cháy, mở rộng cảm biến và nâng cấp quản lý đa nền tảng, hướng tới sản phẩm có giá trị thương mại.

KẾT LUẬN

1. Kết luận

Đề tài "Thiết kế hệ thống báo cháy không dây tích hợp trong hệ sinh thái nhà thông minh" đã hoàn thành đầy đủ nội dung, nhiệm vụ và yêu cầu kỹ thuật đề ra. Kết quả thực nghiệm phản ánh tính chính xác của mô hình lý thuyết và chứng minh giải pháp mạng Mesh OpenThread kết hợp nút SED đáp ứng tốt thực tế: truyền dẫn ổn định, phản hồi báo động 0,5–1,2 giây, tự phục hồi mạng dưới 2 giây khi mất nguồn trạm trung gian. Tối ưu dòng ở chế độ ngủ sâu giúp thiết bị chạy lâu dài trên pin AA, khẳng định tính khả thi khi triển khai vào hạ tầng PCCC thực tế.

Qua quá trình thực hiện đồ án, sinh viên đã tích lũy nhiều kiến thức và kỹ năng:

- Về kiến thức: làm chủ kiến trúc mạng truyền thông không dây, các giao thức IoT (Thread, CoAP, MQTT), quản lý năng lượng trên vi điều khiển Core và nguyên lý các cảm biến vi khí hậu, IC chuyên dụng cho bài toán an toàn.
- Về kỹ năng: nâng cao kỹ năng đọc hiểu tài liệu kỹ thuật tiếng Anh, phân tích và giải quyết bài toán kỹ thuật; rèn luyện thiết kế sơ đồ nguyên lý, thi công PCB, lập trình hệ thống nhúng và xây dựng giao diện; đồng thời trau dồi kỹ năng viết báo cáo khoa học đúng quy chuẩn.

Kết quả đồ án là nền tảng để sinh viên tiếp tục nghiên cứu chuyên sâu về hệ thống nhúng và tự động hóa sau khi tốt nghiệp.

2. Hướng phát triển của đồ án trong tương lai

Từ kết quả đạt được và các hạn chế, đề tài định hướng ba giải pháp mở rộng trong tương lai nhằm nâng cao tính toàn diện và khả năng thương mại hóa:

- Tích hợp học máy dự đoán cháy sớm: dùng các thuật toán AI (LSTM, Random Forest) ở tầng ứng dụng phân tích xu hướng nhiệt độ, độ ẩm và khí độc để dự báo nguy cơ trước khi bùng phát và giảm báo động giả.
- Mở rộng hệ sinh thái cảm biến: thêm cảm biến khí gas (LPG), CO và cảm biến ngọn lửa (Flame Sensor) để xác thực chéo và mở rộng phạm vi cảnh báo cho môi trường đặc thù.
- Nâng cấp quản lý đa nền tảng: phát triển app di động iOS/Android với push notification thời gian thực; mở rộng kênh Telegram hiện có sang Zalo API và tích hợp SMS Gateway để cảnh báo vẫn truyền được khi mất Internet diện rộng.

Các hướng phát triển này giúp tối ưu nhược điểm hiện tại và là lộ trình nâng cấp mô hình thử nghiệm thành sản phẩm PCCC thông minh, có giá trị thương mại và ứng dụng cao.

TÀI LIỆU THAM KHẢO

[1] A. Qandour, D. Habibi, and I. Ahmad, "Wireless Sensor Networks for Fire Emergency and Gas Detection," in Proceedings of the IEEE International Conference on Communications, 2012, pp. 250-255.

[2] H.-S. Kim, S. Kumar, and D. E. Culler, "Thread/OpenThread: A Compromise in Low-Power Wireless Multihop Network Architecture for the Internet of Things," IEEE Communications Magazine, vol. 57, no. 7, pp. 55-61, Jul. 2019.

[3] J. W. Hui and D. E. Culler, "Extending IP to Low-Power, Wireless Personal Area Networks," IEEE Internet Computing, vol. 12, no. 4, pp. 37-45, Jul./Aug. 2008.

[4] S. Quincozes, E. Tubino, and J. Kazienko, "MQTT Protocol: Fundamentals, Tools and Future Directions," IEEE Latin America Transactions, vol. 17, no. 9, pp. 1439-1448, Sep. 2019.

[5] M. A. Tariq, M. Khan, M. T. R. Khan, and D. Kim, "Enhancements and Challenges in CoAP-A Survey," Sensors, vol. 20, no. 21, Art. no. 6391, Nov. 2020.

[6] Espressif Systems, ESP32-H2 Datasheet (Bluetooth Low Energy and IEEE 802.15.4 Combo SoC), Pre-release v0.4.8, 2023.

[7] Espressif Systems, ESP32 Series Datasheet (2.4 GHz Wi-Fi and Bluetooth/Bluetooth LE SoC), Version 5.2, 2023.

[8] Sensirion AG, Datasheet SHT3x-DIS: Humidity and Temperature Sensor, Version 7, Dec. 2022.

[9] ams AG, CCS811 Datasheet: Ultra-Low Power Digital Gas Sensor for Monitoring Indoor Air Quality, v1-00, 2016.

[10] Microchip Technology Inc., RE46C194 — Low-Power CMOS Photoelectric Smoke Detector ASIC with Interconnect, Timer Mode and Alarm Memory, 2014.